MAIN TEXT — REFERENCES & APPENDICES IN COMPANION DOCUMENT

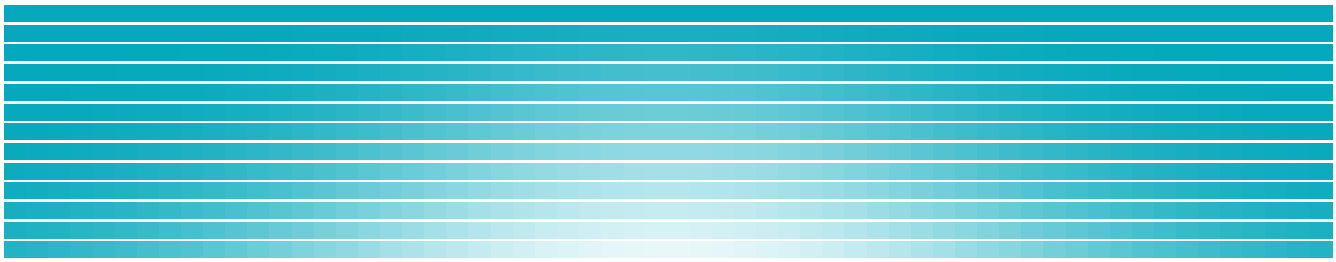
Mesh: A Token-Efficient, Graph-Native Logic for Repository-Scale Code Agents

Edgar Elmo Baumann · Philipp Nils Horn

Mesh · try-mesh.com

edgar.baumann@try-mesh.com · philipp.horn@try-mesh.com

June 2026



Long-context models under-use mid-context evidence — Mesh pre-selects it before inference.

ABSTRACT

Finding the right code is harder than fixing it — most of a repository-scale coding agent's token budget goes to retrieval, not reasoning. Repository-scale coding agents treat context as an unbounded side effect of larger model windows, yet cross-file evidence selection and per-turn token economics dominate both cost and failure rates. We present **Mesh**, a pre-inference control plane for terminal-first code agents: repository state is materialized locally into a field-weighted lexical index, dense chunk vectors, a deterministic repository intelligence graph (RIG), and tiered compression policies that assemble *budgeted evidence before each language-model turn*. Mesh formalizes the agent loop as constrained utility over retrieval quality, token budgets, and task success. Hybrid retrieval fuses BM25-style lexical scoring with cosine similarity on metadata-enriched chunks, expands top seeds along import/call/test edges with decay, and applies a zero-latency lexical-overlap rerank; a confidence surface gates second-pass retrieval and model escalation, while session state tracks provenance, tool-result ledgers, and cache-stable prompt prefixes.

Headline results. Mesh's most robust, model-independent finding is *localization at parity for materially fewer tokens* — isolated by a same-model agentic head-to-head (§12.10) in which the retriever is the only variable across both arms; everything below (retrieval quality, per-query economics, the controlled baseline comparison) is the mechanism that produces that edge, not a separate claim of a higher hit rate. Evaluation is on a grounded suite of **229 localization tasks across six external repositories** (commander, hono, Next.js, TypeScript, Prisma, zod), with gold derived mechanically from the code. Intent-routed Mesh reaches **Hit@1 79.9% / Hit@8 96.9%** — per class, exact 82.5%, conceptual 69.6%, impact 84.9% at Hit@1. In a controlled head-to-head on identical pool, queries, and gold against the strongest retrieval baselines — full-file BM25, a SOTA code embedding (NVIDIA nv-

embedcode-7B), and SOTA managed retrieve-and-rerank (AWS Bedrock Cohere Embed v4 + Rerank 3.5) — Mesh wins overall **79.9% versus 32.8%** for the best single-method baseline (2.4×, McNemar $p<0.001$, ≥ every baseline on every class) — and 2.0–2.4× over the strongest *fused* baselines we could build (a dense+rerank pipeline and an RRF ensemble, §12.9), driven by symbol-aware exact lookup and dependency-graph impact no retriever synthesizes. On token economics, against a competent grep-based agentic searcher, Mesh localizes at **−69.7% tokens per query** (216 vs 712, winning on all six repositories) and **−55.5% over eight-turn sessions** (34.2K vs 76.8K billed). A same-model scaffold study on 16 LocBench issue-localization tasks isolates the retriever from the agent: with the identical model in both arms, Mesh reaches the gold file in **−20% to −26% fewer output tokens and tool calls** than a grep agent at equal recall (Acc@5 100%), at both Haiku 4.5 and Opus 4.8; a larger **122-task agentic replication across all twelve SWE-bench Lite repositories** confirms the pattern at **accuracy parity** (77.0% vs 74.6% Hit@1) for **−13% context tokens and −15% tool calls** (§12.10). Static compression yields up to 16× per-file reduction at 96–100% exported-symbol survival, with a corpus-size-independent workspace briefing. A code cross-encoder on the conceptual channel lifts its retrieval Hit@1 to 76.8% (overall 81.7%), live-validated on two disjoint repository splits (§12.3b) — a gain a strong agent recovers itself but a cheap single-shot tier does not (it still adds +3.8pt at the agent level), so the reranker is wired active and tier-gated (§12.10). Everything runs offline and is reproducible from `apps/cli/bench/cc-vs-mesh/`.

Scope. This is a repository-grounded evaluation across six external projects, with a controlled same-model scaffold study on LocBench (§12.10); validation on standard suites (RepoBench, CrossCodeEval, CoIR, SWE-bench) as resolve-rate leaderboards is future work. Retrieval-quality and token-economics numbers come from two configurations [N1], and Hit@k measures whether gold evidence is surfaced rather than downstream edit success. External repositories (not Mesh's own) avoid name-frequency bias. Mesh's contribution is an integrated, ablatable, cost-normalized architecture paired with an evaluation contract.

Keywords: hybrid retrieval · repository intelligence graph · context budgeting · cost-normalized agents · pre-inference control plane

#	Contribution	Mechanism	Measurable effect (P0)
C1	Layered pre-inference control plane	representation → evidence → context → economics → runtime; each layer env-strippable	env-strippable layers (Appendix E); layer-strip ablation (§16.1) + per-class channel attribution (§12.9)
C2	Hybrid evidence selection + RIG expansion	field-weighted BM25F + chunk vectors + bounded neighbor BFS (decay 0.45)	Hit@8 96.9% (n=229); beats SOTA dense+rerank 2.4× in a controlled head-to-head (§12.9)
C3	Budgeted context construction	compression tiers (compact → emergency), workspace-briefing cap, TOON tool ledgers	static 1.66–16× per-file tiers; briefing ≈52× vs raw
C4	Confidence-gated recovery	top-margin κ meta, `hybridRerank` skip on symbol-exact, optional model-rerank gate	recovery gated on low top-margin κ; misses are recoverable, not wrong-confident (n=229)
C5	Evaluation contract	retrieval ablation + 4-arm baseline head-to-head + NIAH (95% Hit@1, haystack-invariant, n=120) + same-model scaffold study (§12.10) + layer-ablation economics	reproducible in `apps/cli/bench/`

Table 1. Contribution → mechanism → measurable effect.

CONTENTS

PART I — FOUNDATIONS & PROBLEM

why repository-scale agents need a pre-inference control plane

1 Introduction

1.1 Motivation

The dominant trajectory in LLM tooling for software engineering has been to widen the context window, on the assumption that a model able to ingest more of a repository will produce better edits. This treats context as an unconstrained resource whose marginal cost approaches zero. The empirical record disagrees. A typical multi-turn agent re-sends its entire conversation history plus accumulated tool outputs at every turn, so session length — not model capability — becomes the primary cost driver, growing as an unbounded side effect of the loop rather than as a managed budget.

Worse, more tokens do not reliably translate into more usable evidence. Long-context models exhibit a U-shaped attention profile in which evidence at the beginning and end of the prompt is used far more effectively than evidence in the middle — the *lost-in-the-middle* effect [1]. As input length grows, the fraction of the window that receives reliable attention shrinks, so injecting a dozen files can paradoxically reduce effective reasoning depth (Figure §1.1). Mesh therefore treats context as a constrained resource whose economics must be managed explicitly, optimizing the utility of attended evidence per token spent rather than the raw token count.

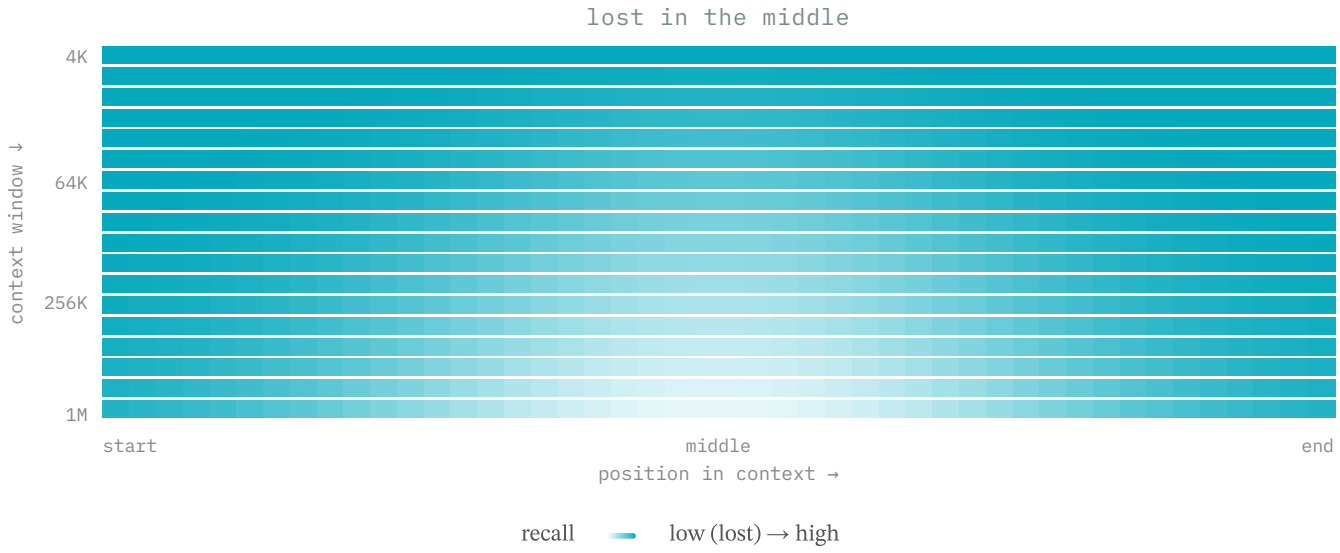


Figure §1.1. Long-context models under-use mid-context evidence: recall holds at the edges but collapses in the middle, and the valley deepens and widens as the window grows from 4K to 1M tokens. Mesh pre-selects evidence before inference.

1.2 Core Insight

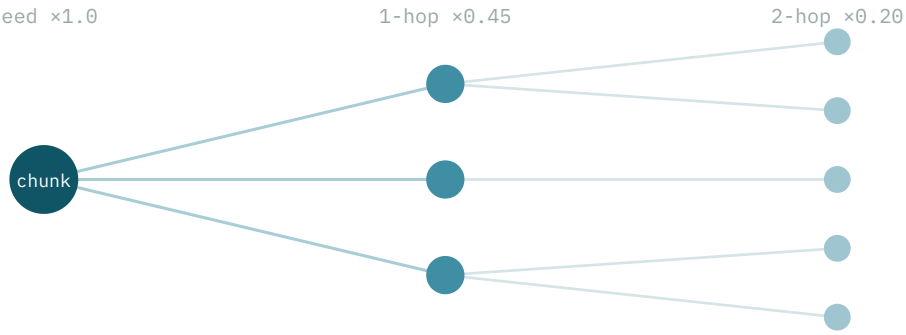


Figure §1.2. The evidence unit is a locality: a chunk plus its k-hop neighbors, not the whole repository.

Evidence selection is a systems problem, not prompt engineering. The lost-in-the-middle effect would matter little if code evidence were self-contained within single files, but the evidence needed for a bug fix, refactor, or impact analysis is almost always cross-file: a function defined in one module is imported by a second, called from a third, tested in a fourth, and wired to a route in a fifth. The unit of evidence is therefore not the whole repository but a *locality*: a chunk of code together with its k-hop neighbors along import, call, test, and route edges (Figure §1.2).

This framing explains why naive context injection fails at scale. Widening the window does not compose the correct subgraph; it dumps the entire repository indiscriminately and buries the relevant cross-file chain beneath thousands of irrelevant lines. Mesh instead performs an explicit selection step that identifies the bounded neighborhood connected to a seed by semantically meaningful edges, materializing it through `rigNeighbors()` and bounded graph expansion. Reading only the definition omits the callers and tests; reading everything reintroduces mid-context degradation. The locality unit is the structural target between these extremes.

1.3 Contributions

The contributions are summarized in Table 1. **C1** is a layered pre-inference control plane — representation → evidence → context → economics → runtime — in which every layer is independently strippable via an environment flag (Appendix E), enabling clean attribution through env-flag layer-stripping (§16.1) and the per-class channel attribution of the controlled comparison (§12.9). **C2** is a hybrid evidence-selection pipeline that fuses field-weighted BM25F lexical scoring, dense chunk-vector cosine similarity, and bounded neighbor BFS with decay 0.45 per hop, reaching Hit@8 96.9% on the n=229 suite (§12.3) and beating the strongest dense+rerank baselines in a controlled head-to-head (§12.9). **C3** is budgeted context construction: deterministic compression tiers (compact → emergency), a corpus-size-independent workspace briefing, and TOON-encoded tool ledgers.

C4 is confidence-gated recovery, in which a top-margin κ surface gates second-pass retrieval, skips `hybridRerank()` on exact-symbol matches, and only recommends an optional model rerank, gated on a low top-margin κ. **C5** is an evaluation contract built around a 229-task, three-arm head-to-head (RAW / competent grep agent / Mesh) with ground-truth gold, multi-turn economics, and a controlled four-arm retrieval head-to-head against SOTA baselines (full-file BM25, nv-embedcode-7B, Cohere Embed v4 + Rerank 3.5) on identical data — where the routed stack wins overall Hit@1 79.9% vs 32.8% at McNemar $p<0.001$ (§12.9) — all reproducible via the bench scripts in `apps/cli/bench/cc-vs-mesh/`. The contribution is an integrated, ablatable, cost-normalized architecture and its evaluation contract, not a new retrieval leaderboard result.

1.4 Research Questions (Preview)

Five research questions structure the evaluation; each is paired with primary metrics and a falsification hook, with full treatment deferred to §10. **RQ1** asks whether pre-inference evidence selection improves retrieval quality under a fixed token budget

(Hit@k, MRR). **RQ2** asks whether it reduces tokens and cost at equal task success (tokens, USD, pass rate). **RQ3** asks the marginal value of each layer (ablation deltas). **RQ4** asks whether the approach scales with repository size and session length (index build time, briefing size versus file count). **RQ5** asks what residual failure modes remain (the wrong-confident versus under-retrieved taxonomy).

1.5 Paper Organization

The paper is organized in four parts. Part I establishes the foundations and the problem: it surveys related work across code retrieval, program structure, agent loops, compression, and caching, then formalizes the constrained-utility model and the five research questions. Part II presents the architecture as five env-strippable layers — representation, evidence, context, economics, and runtime — describing the index and Repository Intelligence Graph, hybrid retrieval with bounded expansion and confidence-gated recovery, budgeted assembly with compression tiers, and cost-aware routing. Part III reports the evaluation: the retrieval ablation, needle-in-a-haystack scenarios, the live end-to-end A/B run, layer-ablation economics, and scalability. Part IV positions Mesh against alternatives, addresses threats to validity and standing caveats, and concludes.

2 Background & Related Work

2.1 Information Retrieval over Code

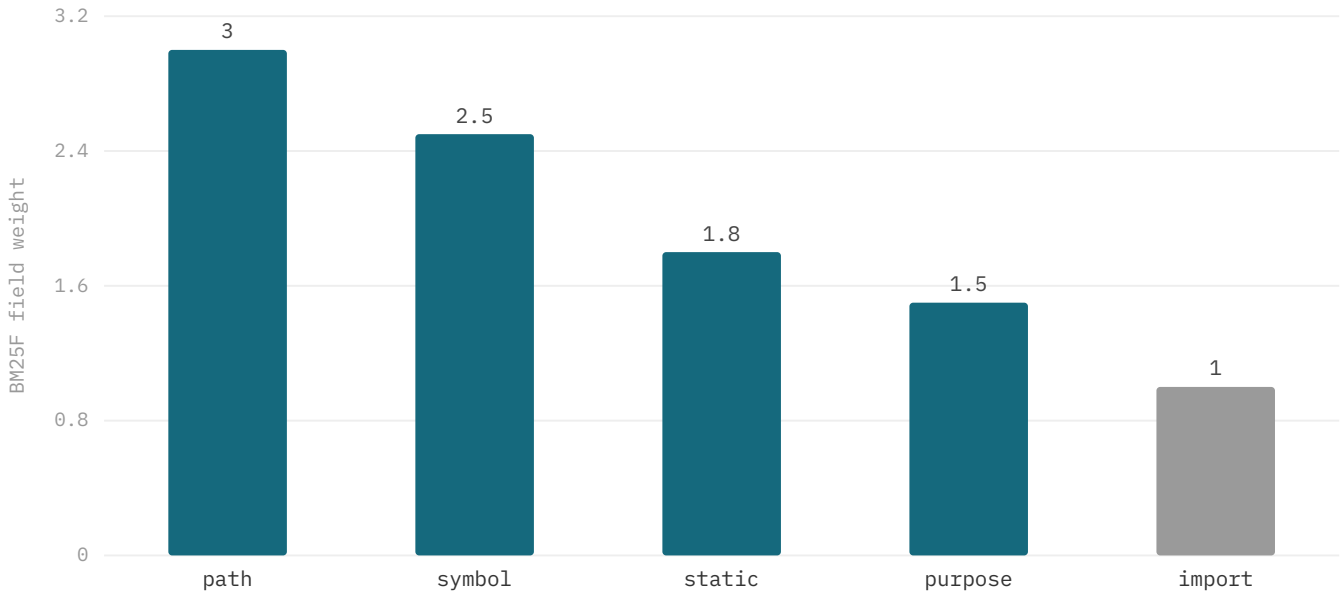


Figure §2.1. Mesh's configured BM25F field weights (from `workspace-index.ts`): path > symbol > static > purpose > import — no body field.

BM25 [8] remains the strong sparse baseline on code retrieval benchmarks such as CoIR, and BM25F [11] generalizes it by weighting term frequencies per field before saturation — the direct basis for Mesh's per-field boosts. As Figure §2.1 shows, the configured weights in `workspace-index.ts` rank path 3.0 > symbol 2.5 > static 1.8 > purpose 1.5 > import 1.0, with *no body field* (k1=1.4, b=0.75). Code identity lives in paths and symbols, not prose bodies; indexing the body would dilute exact-symbol matches without adding localization signal. An inverted-index prefilter narrows each query to roughly 120 candidates before scoring.

Learned sparse models such as SPLADE improve BEIR-style recall but require a neural index whose construction and serving cost is poorly suited to local, frequently-rebuilt CLI-scale indexes. Dense code embeddings [13,14,15,30] complement lexical search when query vocabulary diverges from code tokens — conceptual queries benefit from semantic similarity — but exact symbols (`CostTracker`, `agent-loop.ts`) favor lexical matching, and embedding geometry is sensitive to thin chunk text, a brittleness adversarial code-retrieval benchmarks expose [29]. The Anthropic contextual-retrieval report [12] confirms the lexical channel stays essential for identifiers even alongside strong embeddings.

Standard IR metrics (Hit@k, MRR, nDCG) measure ranking over static qrels but miss agent-specific concerns: tool turns to localization, tokens inserted into context, and wrong-confident answers where an incorrect top result is acted upon regardless. Mesh therefore reports IR metrics *and* agent economics on every ablation row rather than treating ranking quality as a proxy for task success.

2.2 Program Structure as Retrieval Signal

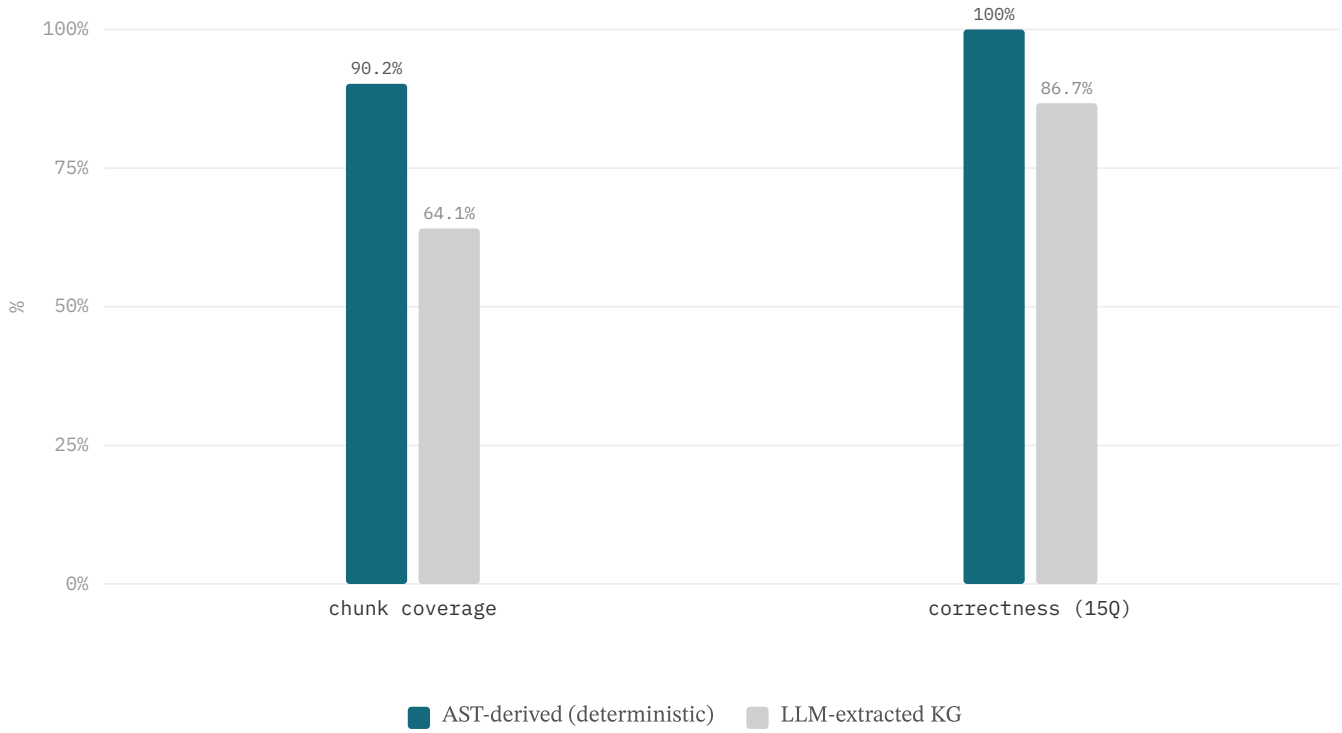


Figure §2.2. DKB (arXiv 2601.08773), Shopizer: AST-derived graph builds in 2.81s vs 200s LLM-extracted (71×), 90.2% vs 64.1% chunk coverage; LLM skips 377/1210 files entirely; 15/15 vs 13/15 correctness.

The program dependence graph (PDG) [17] formalizes control and data dependencies and supplies the theoretical basis for impact analysis. Mesh abstracts this to the file and symbol level, building pragmatic import, call, test-covers, and route-handles edges from static AST parsing rather than full PDGs — scoped to retrieval, not optimization. The reliability of extraction is decisive: deterministic AST parsing yields the same graph from the same source, whereas LLM-extracted knowledge graphs vary with temperature, prompt, and context, hallucinating or omitting edges.

The DKB study [18] quantifies this gap on Shopizer (Figure §2.2): the AST-derived graph builds in 2.81s versus 200s for the LLM-extracted graph (71×), covers 90.2% versus 64.1% of chunks while skipping 377 of 1,210 files entirely, and answers 15/15 versus 13/15 structural queries correctly. Mesh adopts deterministic extraction exclusively for its Repository Intelligence Graph, on the principle that retrieval signals must be reproducible and auditable; edge types are weighted by query intent, with call edges prioritized for impact queries and test edges for regression queries.

Aider's repo map [4] is the closest public analogue to Mesh's budgeted structural context, ranking symbol signatures by PageRank over the dependency graph to fit a token budget. Mesh generalizes the pattern: rather than injecting a static map as the retrieval, it uses the structural summary as a cache-stable prefix and performs RIG-backed hybrid retrieval of locality units — chunks plus neighbors — on demand.

2.3 Graph-Augmented Retrieval

Figure §2.3. Retrieval landscape — Mesh RIG targets high multi-hop capability at low indexing cost.

The graph-augmented retrieval landscape spans three lineages. Standard RAG chunks and vectorizes documents, retrieving by similarity with no explicit graph. GraphRAG [19,25] constructs an LLM-extracted entity knowledge graph with community summaries for global sensemaking, at significant indexing cost. LightRAG [20] adds dual-level graph-plus-vector indexing with faster incremental updates, and HippoRAG [21] runs personalized PageRank over a knowledge graph for multi-hop associative retrieval, and hypergraph variants [38,39] extend this to multi-entity relations. These systems trade higher indexing cost for multi-hop reach.

As Figure §2.3 positions it, Mesh's RIG occupies a distinct point: a deterministic local graph (files, symbols, tests, routes) extracted from the AST at negligible cost, yet supplying structural signal comparable to graph-RAG approaches — high multi-hop capability at low indexing cost, closer to DKB [18] than to GraphRAG. The tradeoff is that RIG edges are limited to deterministically extractable relations and cannot capture relationships requiring model judgment; Mesh compensates by pairing graph expansion with the lexical and dense channels.

The key architectural decision is bounded expansion rather than full-graph prompt injection. From the top-3 hybrid seeds, Mesh performs a budget-capped breadth-first traversal with a decay of 0.45 per hop ($1 \rightarrow 0.45 \rightarrow 0.20 \rightarrow 0.09$), so distant nodes receive negligible score and are dropped. This produces a subgraph of at most a few dozen nodes, avoiding the lost-in-the-middle degradation that afflicts unbounded graph or context dumps.

2.4 Agent Loops & Tool Use

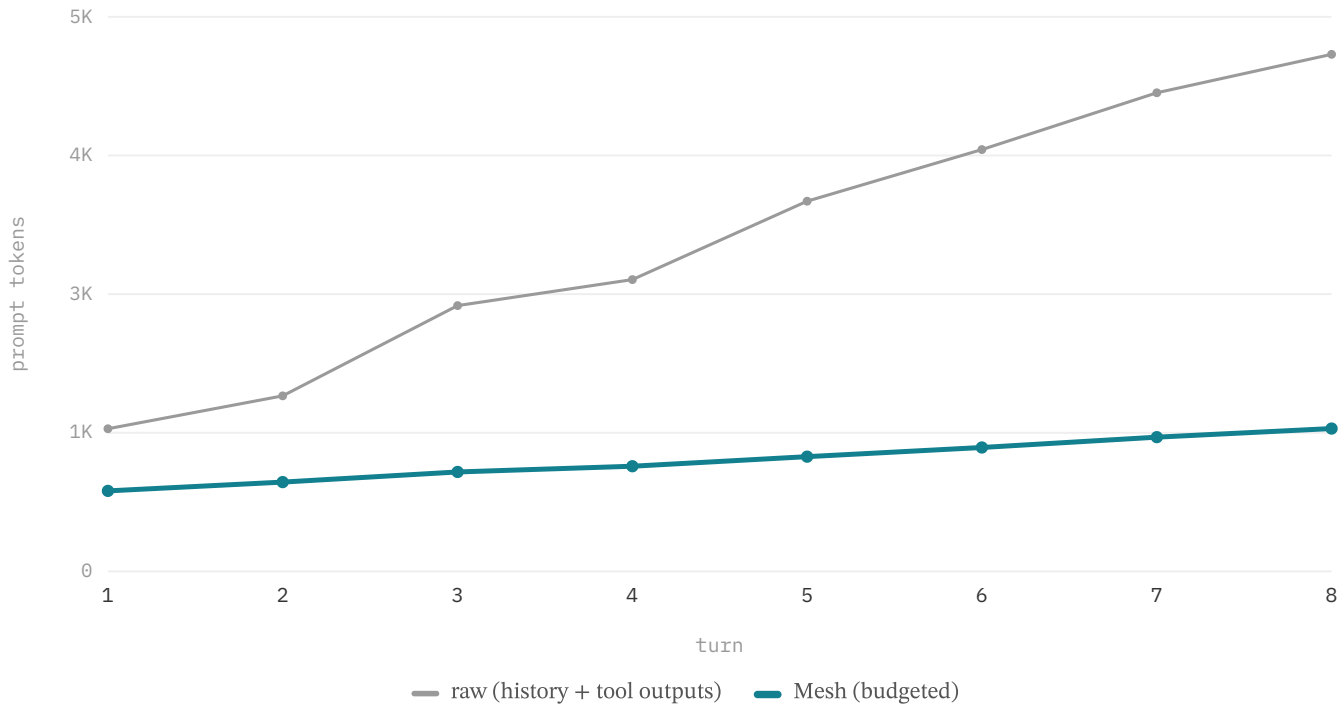


Figure §2.4. 8-turn Commander session — Mesh (lean retrieval + compacted history) vs a competent grep agent that re-sends its surgical read/grep history: the agent's prompt grows 1.3K→4.8K, Mesh stays ~0.8–1.3K (gap ~1.8×→3.6×).

ReAct-style [22] loops alternate reasoning traces with tool actions, appending each tool output to a history that is re-sent every turn. At repository scale this creates super-linear token growth: file reads and grep results may each contribute thousands of tokens, and the cumulative prefix can exhaust the window or the session budget. SWE-agent and SWE-bench [23] (with its human-validated subset [46]) standardize issue-resolution evaluation, while Agentless [24] shows that deterministic, low-cost scaffolding can rival agentic loops — both motivate treating context as a managed budget rather than an emergent side effect.

Mesh budgets the pre-inference phase: the `ContextAssembler` allocates fixed ceilings to history and evidence, compresses older turns, and enforces the bound before the model is invoked. Figure §2.4 reports an 8-turn Commander session comparing Mesh (lean retrieval plus compacted history) against a competent grep-based agent that re-sends its surgical read/grep history, with the LLM held constant. The agent's prompt grows 1.3K → 4.8K tokens as surgical reads accumulate, while Mesh stays at roughly 0.8–1.3K, widening the gap from ~1.8× to ~3.6× over the session.

The model is held fixed and only the scaffold varies, so the figure isolates the scaffold's effect on prompt volume. Because the raw baseline leans on the model's prior familiarity with these libraries, the gap on unfamiliar code is expected to be larger. Complementary mechanisms — the stale-aware tool-result ledger, TOON encoding, and artifact offloading — further reduce the volatile prefix that breaks provider prompt caching.

2.5 Context Compression & Token Budgeting

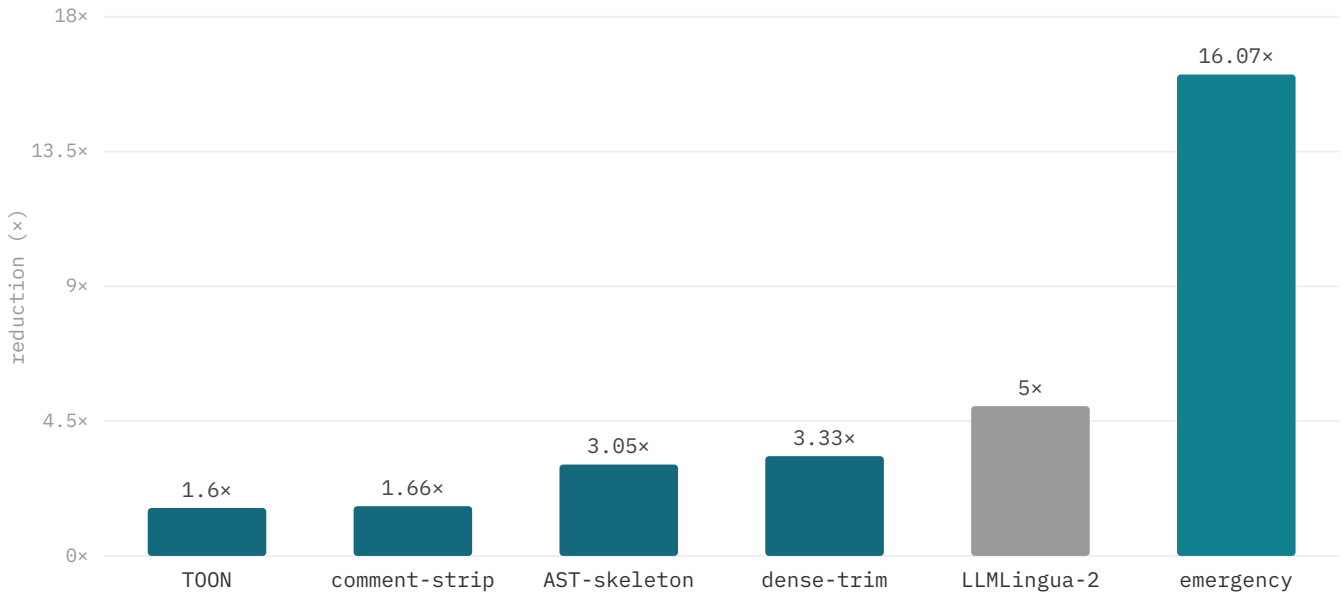


Figure §2.5. Compression reduction by method (Mesh tiers from static-bench; TOON from §7.3). LLMLingua-2 at its reported 2–5× near-lossless range (ACL'24 [36]); general-text compressors do not preserve code structure (§2.5).

Lossy compression is essential under finite budgets, but perplexity-based methods are unsafe for code: the LLMLingua family [35,36,37] — which reaches up to 20× compression lossily (original LLMLingua, EMNLP'23) or 2–5× near-lossless (LLMLingua-2, ACL'24) on prose — removes low-information tokens, which can render a function unparseable. Mesh instead compresses structurally through an ordered tier ladder — raw → compact → dense → semantic → emergency — that preserves syntax and identifier visibility at every level (Figure §2.5). On 255 CLI source files the measured tiers move from 780.2K tokens (1.00×) to compact (1.66×), dense (3.33×), semantic, and emergency (16.07×). The `semantic` AST-skeleton tier (255.4K, 3.05×) is by design less aggressive than the body-trimming `dense` tier (234.2K, 3.33×), so the ladder orders compact → semantic → dense → emergency; the escalation logic (`pickTierForBudget`) already follows this order.

Schema-bound capsules — purpose, symbols, imports/exports, and test links — outperform abstractive digests for code tasks, which may omit identifiers or hallucinate symbols. The workspace briefing (`buildCapsule`) holds near 15.0K tokens flat as the corpus grows — its *absolute* size is independent of corpus size (≈52× below the 780K-token 255-file corpus; the ratio itself grows with the corpus, since raw scales and the briefing does not). On homogeneous tool-result arrays, TOON encoding removes 26–58% of tokens versus JSON (−26% grep results, −37% file lists, −58% tool arrays) and is never larger, making it an opt-in safe encoding.

Provider prompt caching adds an economic layer: stable prefixes — system prompt, briefing, index summary — are cached, so per-turn evidence sits in the volatile suffix. End-to-end, 65.3% of Mesh's input tokens hit the cache versus 75.3% for the raw baseline. The raw baseline wins on cache *rate*, but Mesh's absolute token volume is far smaller; the honest framing is total cost reduction, not a cache-rate claim.

2.6 Cost-Aware Model Routing

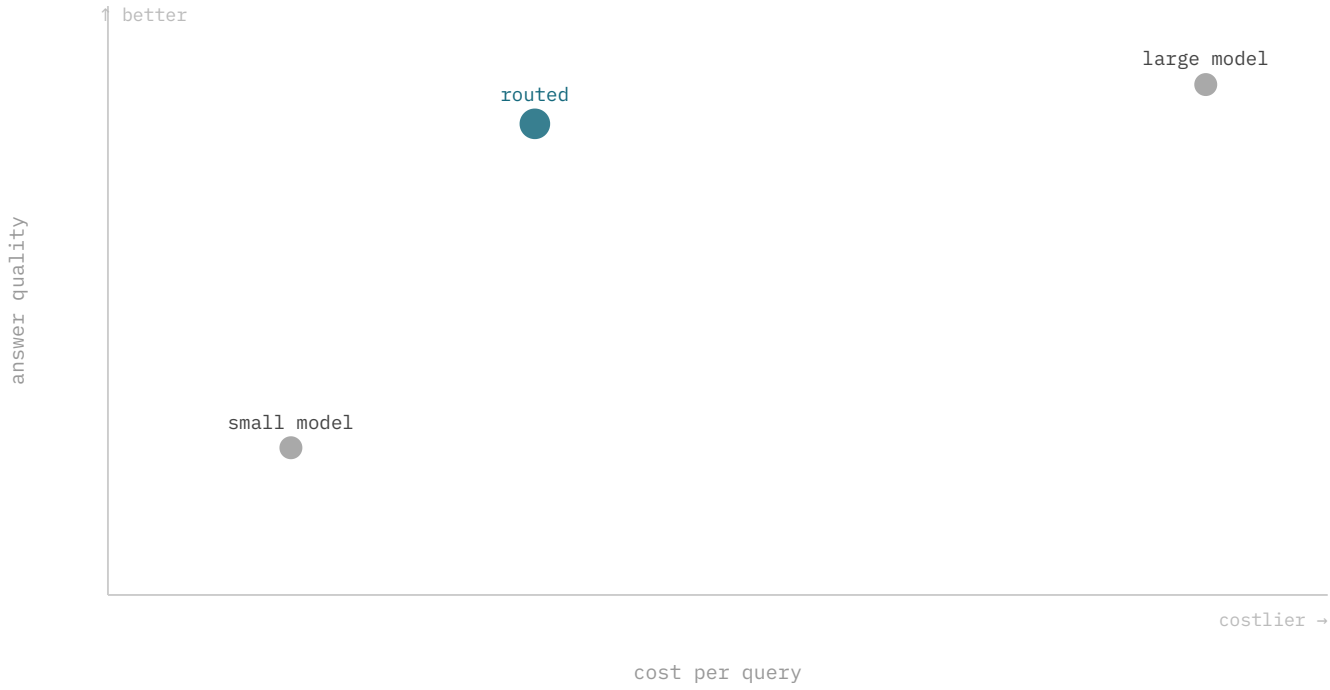


Figure §2.6. Query-dependent routing recovers most large-model quality at a fraction of the cost.

RouteLLM [6] and FrugalGPT [7] establish that query-dependent model routing preserves most large-model quality at 35–98% lower cost, consistent with Figure §2.6: capability recovers near the strong-model ceiling at a fraction of the spend. Mesh's cost model separates input, output, and cached-input pricing and amortizes embedding and index-build cost across a session. Task classes map to model tiers — localization and retrieval to a cheap, fast tier (e.g., Gemini Flash); multi-file reasoning and edits to a more capable tier — since localization needs interpretation of retrieved evidence, not deep synthesis.

Three routing policies govern selection: budget caps with hard stops, cache-warm stickiness (preferring the model with the longest cached prefix, since switching forfeits the cache discount), and escalation on low confidence. When the top margin $\Delta\kappa$ falls below threshold, `shouldRecommendModelRerank` *recommends* a stronger model but does not auto-invoke it — the operator must approve the spend.

This is the central difference from prior routers: FrugalGPT cascades automatically and RouteLLM decides single-shot from query text alone, whereas Mesh routes on pre-inference signals unavailable to text-only routers — retrieval confidence, task intent, index health, and cache status. The contribution is finer-grained cost control through these signals, not a claim of higher accuracy than learned routers; on cost-normalized outcomes the gain is in cost per successful outcome.

2.7 Gap Analysis

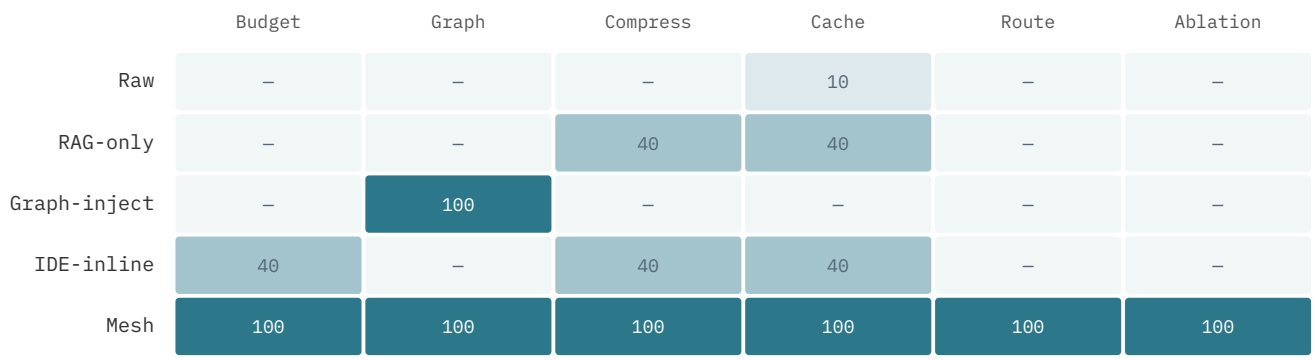


Figure §2.7. Capability coverage across architectures: only Mesh unifies budget, graph, compression, cache, routing, and ablation.

Few systems publish ablatable architectural layers *and* cost accounting together. Agentless [24] reports cost per issue but does not decompose retrieval from repair; Aider publishes repo-map token savings without a full ablation matrix; GraphRAG [19] measures indexing cost but not per-query retrieval economics; ReAct [22] addresses tool-use flexibility, not the token growth it induces. Each optimizes a subset of the space — retrieval, generation, cost, or latency — and the absence of an integrated account makes the interactions between layers, such as whether aggressive compression degrades retrieval or graph expansion adds cost without lifting success, unmeasurable.

Mesh unifies retrieval, graph, compression, cache, routing, and ablation into a single control plane in which every layer is independently strippable and every ablation row reports both IR metrics and agent economics. Figure §2.7 presents this as a qualitative capability-coverage map for positioning — not a benchmark result. Each prior system optimizes one corner of the space; Mesh is the only one to bring all six axes under a single budgeted, ablatable, reproducible control plane, which is what makes the cross-layer tradeoffs measurable in the first place.

3 Problem Formulation

3.1 Entities & State

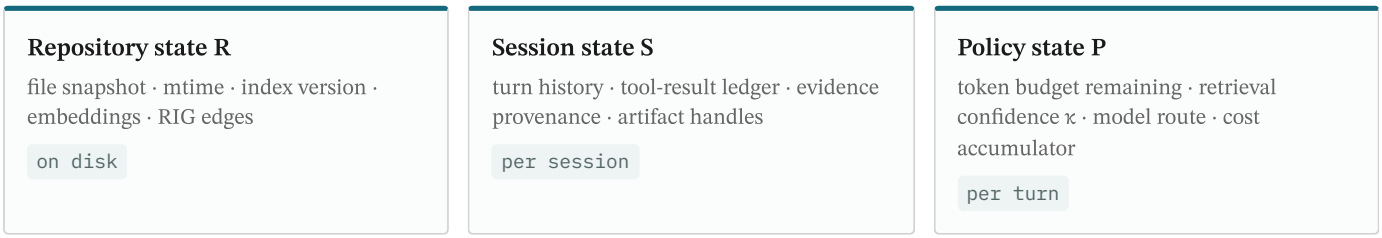


Figure §3.1. Three state categories govern Mesh's behavior.

Mesh formalizes a turn as a constrained optimization over six entities. Query `Q` is the natural-language or identifier-anchored request, routed by `routeRetrievalQuery()` into symbols, path hints, and intent flags (definition, test, impact). Repository state `R` is the on-disk snapshot — file records, modification timestamps, index version, embedding coverage, and RIG edges — persisted as the JSON index plus the `vectors.bin` sidecar. Candidates `C` are the ranked file-or-chunk set produced by retrieval fusion, capped by an inverted-index prefilter at ≈120 records. Budget `B` is the remaining token (and optionally USD) al-

lowance. Model `M` is the selected LLM tier; tools `T` are the callable workspace actions (`semantic_search`, `read_file`, `grep_ripgrep`, `rig_query`, `rig_neighbors`, `rig_impact`).

As Figure §3.1 shows, state partitions into three categories. Repository state is the durable, locally materialized index of `R`. Session state tracks the turn sequence, tool-result ledger, evidence provenance, retrieval confidence, model route, and cost accumulator across the interaction. Policy state holds the recovery gates, cache breakpoints, and budget thresholds that govern how evidence is selected and trimmed. The assembler reads all three before every turn: `R` supplies candidates, session state prevents re-reading, and policy state enforces the budget.

3.2 The Agent Loop as Constrained Utility

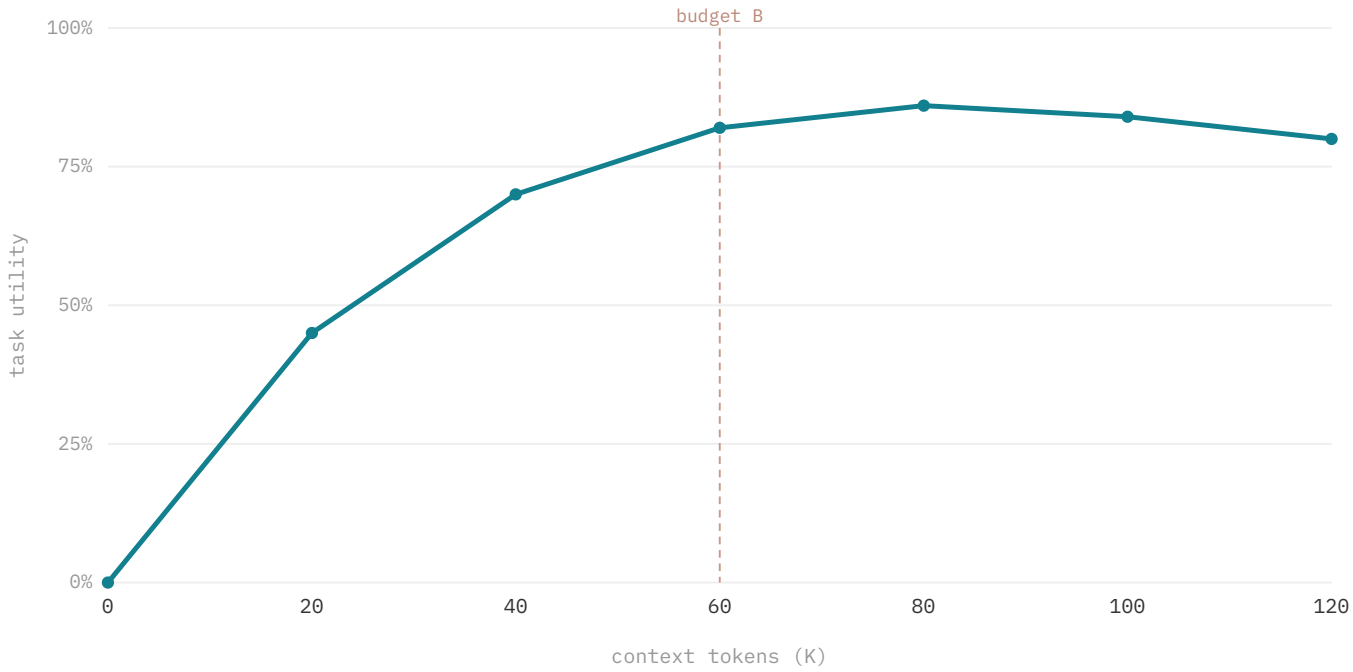


Figure §3.2. Task utility has diminishing returns in tokens and degrades past the budget (lost-in-the-middle); Mesh maximizes utility under B.

The core problem is to select a subset `S ⊆ C` that maximizes the expected task utility of the model's output subject to a token-cost constraint `f(S) ≤ B`. As Figure §3.2 illustrates, this is multi-objective: correctness (does the edit resolve the issue?) and cost (tokens and dollars spent) are in tension. A raw agent that reads every file has high information availability but prohibitive cost; an agent that relies on parametric memory alone has zero retrieval cost but low correctness on repository-specific tasks. Mesh targets the cost-normalized region between these extremes by pre-selecting and compressing evidence before each model turn.

Utility is not monotone in `|S|`. Beyond a point, additional evidence yields diminishing returns and then active degradation: long-context models under-utilize mid-context tokens, so an over-large prompt obscures the relevant span (the lost-in-the-middle effect) and lowers correctness even where the budget would permit more. The budget `B` is therefore a control knob, not merely a ceiling — a tighter budget forces aggressive compression and fewer files, trading recall for cost. The assembler enforces this with ratio thresholds (warn 60%, compact-soon 75%, compact-now 85%, block 92%) that progressively trim history and tool results as `B` is consumed.

3.3 Failure Modes

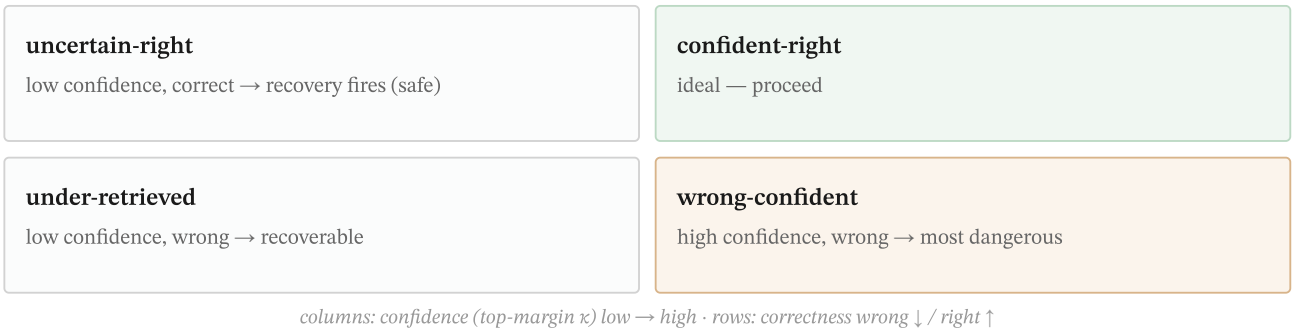


Figure §3.3. Failure modes by confidence × correctness; wrong-confident is the failure recovery must catch.

Uncontrolled context management produces four failure modes. Over-read consumes budget on irrelevant evidence, diluting attention and raising lost-in-the-middle risk; it is the characteristic raw-agent mode, and is not exercised on the localization harness. Re-read wastes budget re-consuming a file already read in a prior turn — common in agents lacking a tool-result ledger. Stale evidence arises when a snapshot is invalidated by a later write and the agent reasons over the outdated read; this mode is not exercised on the retrieval suite. Wrong-confident is the most dangerous: the pipeline returns a high-confidence score for an incorrect result, so the agent acts without triggering recovery.

Figure §3.3 organizes these along the confidence × correctness plane. Confident-right is the ideal quadrant. Uncertain-right is safe — low confidence on a correct result, where recovery may fire harmlessly. Under-retrieved is low-confidence and wrong, but recoverable because the low margin invites further search. Wrong-confident — high-confidence and wrong — is the quadrant recovery must catch, since the agent has no internal signal to doubt the evidence. On the n=31 retrieval harness, residual failures are 3 wrong-confident and 1 under-retrieved, with each mode mapped to a specific layer: over-read to budgeted construction, re-read to the ledger, stale to write-triggered invalidation, and wrong-confident to the confidence surface and confidence-gated recovery.

3.4 Design Invariants

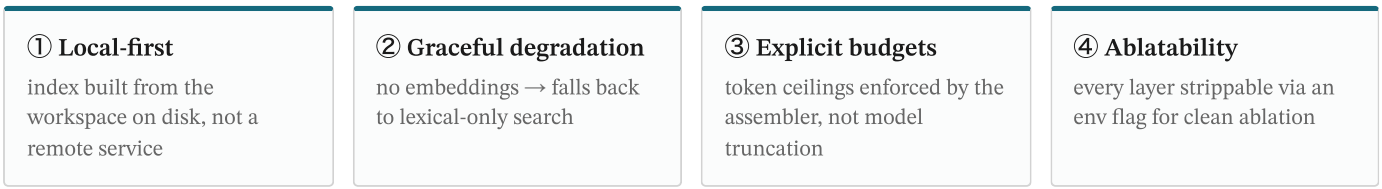


Figure §3.4. The four design invariants.

Four invariants, summarized in Figure §3.4, constrain every architectural decision; each is enforced by a mechanism and can be violated only by an explicit environment flag. **Local-first**: all repository representations — file records, lexical index, chunk vectors, and graph edges — are computed and persisted on disk under `.mesh/`, so no network call is required for search once the index is built. The only remote dependency is the optional embedding call at index time. **Graceful degradation**: when the embedding endpoint is unreachable or `MESH_LEXICAL_ONLY_SEARCH=1` is set, retrieval falls back to field-weighted BM25 rather than failing; `resolveRetrievalRoute()` selects the path from `embeddingCoverage`.

Explicit budgets: every token allocation has a named ceiling and defined overflow behavior. The assembler enforces `maxInputTokens` and per-bucket budgets and throws rather than silently truncating, so overflow is observable in session logs — token limits are enforced by the assembler, not by model-side truncation. **Ablatability**: each of the five layers is strippable through a flag (`MESH_DENSE_ONLY_SEARCH`, `MESH_DISABLE_GRAPH_EXPANSION`, `MESH_ENABLE_EMBEDDINGS=0`, the rerank gate), and the

system must still produce a coherent if degraded result. Ablatability is a scientific necessity: a layer whose marginal contribution cannot be measured by removing it is unfalsifiable, which is precisely how the ablation matrix is generated.

PART II — THE MESH ARCHITECTURE

five env-strippable layers from representation to runtime

4 Architecture Overview

4.1 The Five Layers

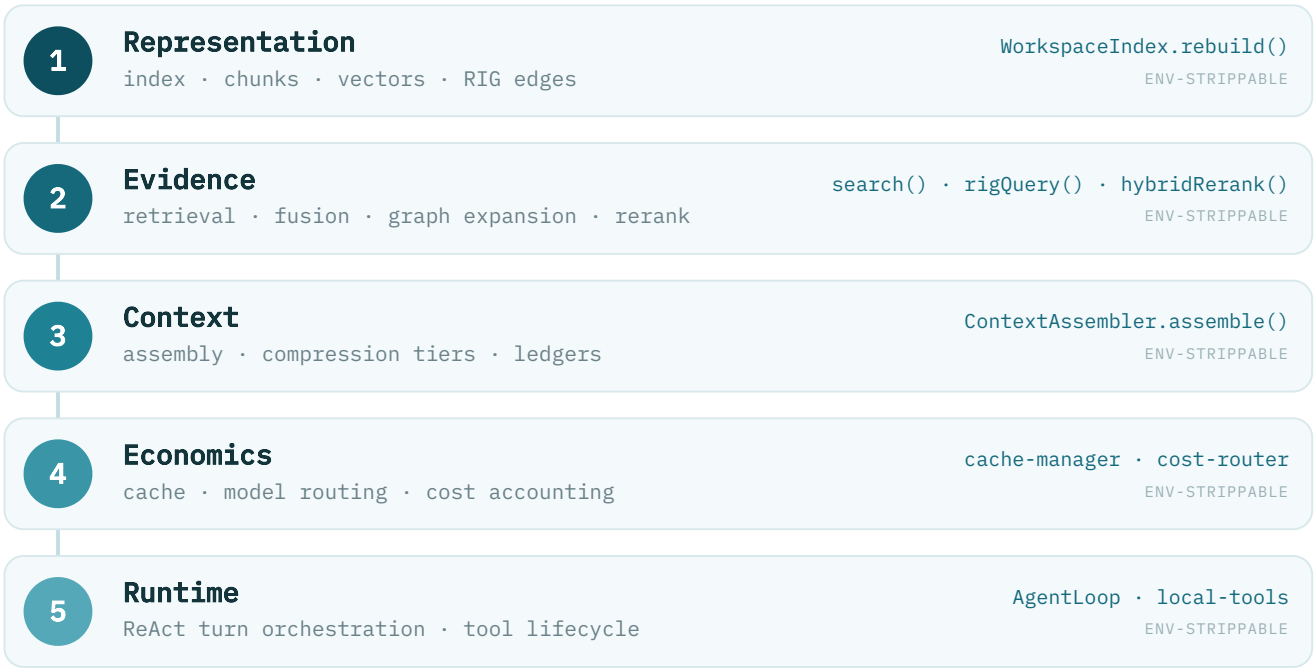


Figure §4.1. The five logical layers of the Mesh control plane; each is independently ablatable.

Mesh decomposes the agent loop into five logical layers, each owning one concern and each independently strippable for ablation. The **Representation** layer materializes repository state — a field-weighted lexical index, dense chunk vectors, and the Repository Intelligence Graph — through `WorkspaceIndex.rebuild()`. The **Evidence** layer turns a query into a ranked, graph-expanded candidate set via `search()`, RIG traversal, and `hybridRerank()`. The **Context** layer packs those candidates under a token budget with compression tiers and provenance ledgers (`ContextAssembler.assemble()`). The **Economics** layer governs caching, model routing, and cost accounting, and the **Runtime** layer drives the ReAct turn loop and tool lifecycle. Each layer consumes only the one beneath it, so any single layer can be removed without collapsing the rest (Figure §4.1).

4.2 Request Lifecycle

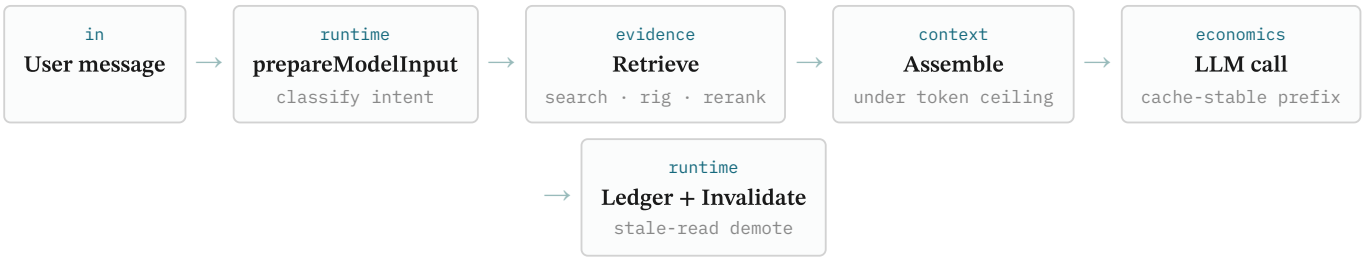


Figure §4.2. Per-turn request lifecycle; evidence is assembled *before* each model call.

Every turn follows the same pre-inference pipeline (Figure §4.2): the user message is classified for intent, retrieval gathers and reranks evidence, and the assembler packs it under the token ceiling behind a cache-stable prefix before a single model call is issued. After the model responds, tool results are written to the ledger and any file edit triggers stale-read invalidation. The defining property is ordering — evidence is selected and budgeted *before* the model sees the prompt, rather than accumulated as a side effect of the model's own tool calls.

4.3 State Model & Invariants

Figure §4.3. Write-triggered invalidation guarantees read-your-writes within a session.

Mesh maintains three state categories with distinct consistency models: repository state R (the materialized index, persisted to the `.mesh/` sidecar), session state S (turn sequence, tool-result ledger, provenance), and policy state P (remaining budget, retrieval confidence κ, model route). The separation is load-bearing: a lagging index rebuild in R never blocks the agent loop, which continues on stale-but-tagged evidence drawn from S. The strongest guarantee is *write-triggered stale-read demotion*. When the model edits a file, the `ToolResultLedger` synchronously scans every entry referencing that path and demotes any whose observed `mtime` predates the write to stale status. Demotion is conservative — a same-file edit may demote a still-valid read — but this over-demotion is deliberate, trading minor recall loss for elimination of silent staleness.

This mechanism implements *read-your-writes* consistency within a session: after a write, no subsequent turn reasons about the file's pre-edit state, which is the most dangerous form of staleness for an editing agent. The guarantee does not depend on the index being current, and it does not extend across files — if the model edits file A then reads dependent file B before `WorkspaceIndex.rebuild()` completes, B may not yet reflect A. The model is eventually consistent with the on-disk filesystem (not the Git tree), with convergence bounded by rebuild duration. *Cache stability* is an explicit cross-cutting constraint: the system prefix must remain unchanged across turns to preserve provider prompt-cache hits (65% measured), which bounds how aggressively index updates may perturb the assembled prompt.

4.4 Ablatability Contract

Figure §4.4. Which environment flag strips which layer (verified against `workspace-index.ts` + Appendix E; ● = affected).

Every layer is independently strippable through an environment flag, and the system must still produce a coherent — if degraded — result without it. The flag-to-layer matrix in Appendix E maps each control to the component it removes: `MESH_LEXICAL_ONLY_SEARCH=1` and `MESH_DENSE_ONLY_SEARCH=1` isolate the two channels of the Evidence layer, `MESH_DISABLE_GRAPH_EXPANSION=1` removes RIG neighbor expansion, `MESH_ENABLE_EMBEDDINGS=0` suppresses vector-sidecar writes in the Representation layer, and the rerank gate disables the optional rerank sub-component. The `retrieval-internal-ablation.mjs` harness exercises these flags to produce the layer-strip ablation reported in §16.1.

Ablatability is a scientific necessity, not an evaluation convenience: a layer whose marginal contribution cannot be measured by removing it has an unfalsifiable design rationale. On the small internal ablation harness the channels overlap on Hit@1 because identifier-anchored lookups are already lexically saturated at that scale; the controlled n=229 comparison (§12.9) is where each channel's contribution becomes measurable — symbol-definition matching carries exact queries to 82.5%, the code embedding carries conceptual to 69.6%, and the RIG graph carries impact to 84.9%, none of them reachable by the others.

5 Representation Layer

5.1 Per-File Index Model

IndexedFileRecord	
path · purpose	string
symbols · imports · exports	string[]
routes · callSites	Edge[]
chunks	Chunk[] (enriched text)
staticSignals	7 deterministic fields
capsule	synopsis + test links
vectors	→ binary sidecar (by chunk id)

Figure §5.1. Per-file index record; vectors live in a binary sidecar keyed by chunk id.

The unit of representation is the `IndexedFileRecord`, one per workspace file, keyed by its relative `path` as primary key. Each record aggregates a one-sentence `purpose` annotation; the file's structural interface as `symbols`, `imports`, and `exports` arrays; runtime surfaces as `routes` and `callSites`; the enriched `chunks` that feed dense retrieval; seven deterministic `staticSignals` facets; a per-file `capsule` summary; and a content `hash` with `mtime` for incremental delta-update detection. These fields serve dual roles — as field-weighted BM25 lexical surfaces and as the sources for the Repository Intelligence Graph's structural edges — so call and import relationships are available at every stage of retrieval without a separate graph index.

Chunks are line-bounded spans enriched before embedding: each is prefixed with a structured header carrying the file path and purpose, the signatures of symbols it defines, any route handlers, covering test names, and the callers and callees of those symbols. This makes the embedding capture a chunk's structural role rather than only its local syntax. The 768-dimensional vectors are not stored inline in the record; they persist to a binary sidecar (`vectors.bin`) keyed by chunk id, keeping `index.json` compact and enabling efficient cosine computation at search time. An index can be structurally complete yet embedding-incomplete: when vectors are absent, the lexical and graph channels operate normally and the dense channel degrades, consistent with the graceful-degradation invariant.

5.2 Index Build Pipeline

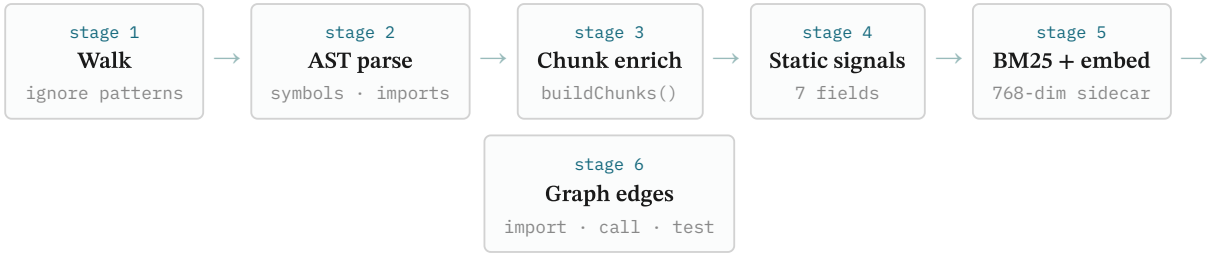


Figure §5.2. Six-stage index build pipeline.

The index is built in six deterministic stages (Figure §5.2): a workspace walk applies ignore patterns; AST parsing extracts symbols, imports, and exports; `buildChunks()` produces metadata-enriched chunks; static analysis derives the seven signal fields; the lexical index and 768-dim chunk vectors are written, the vectors to a binary sidecar; and import, call, and test edges are materialized into the RIG. The build is incremental and content-addressed, so unchanged files are skipped on rebuild, and it degrades gracefully — with embeddings disabled, the remaining stages still produce a usable lexical-plus-graph index.

5.3 Static Analysis as Retrieval Signal

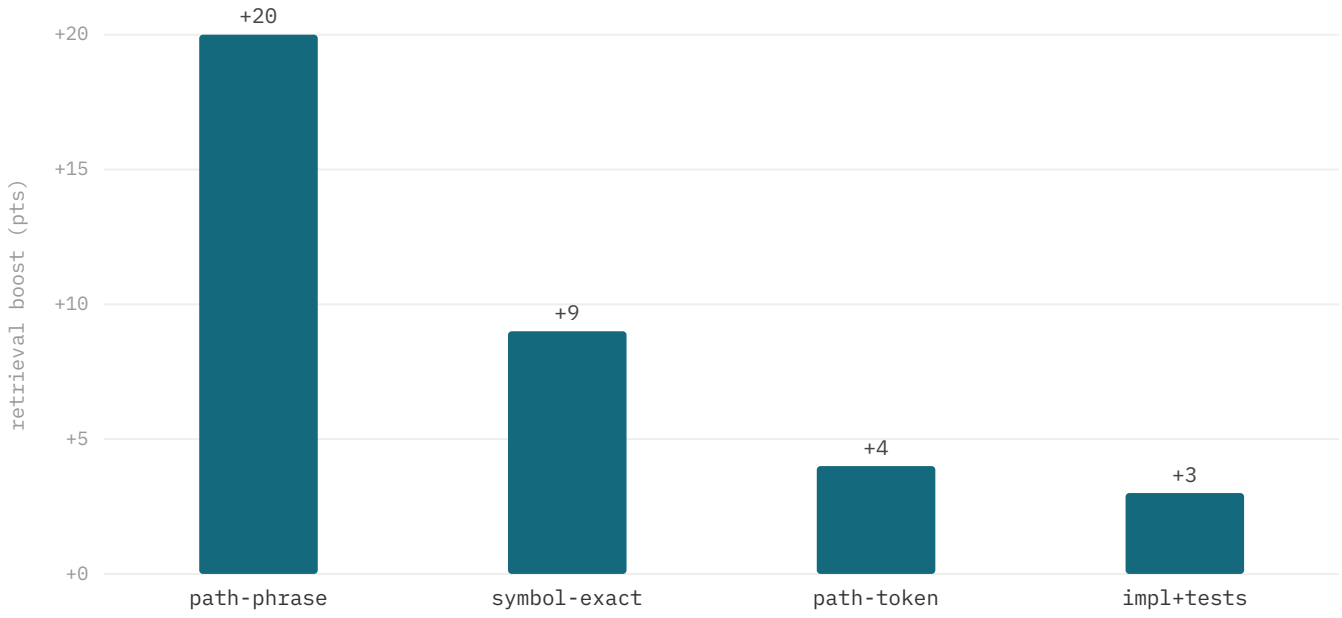


Figure §5.3. Real retrieval-scoring boosts (`scoreRetrievalAdjustments` , definition queries); test files penalized −8/−12 (not shown).

`extractStaticSignals()` populates seven deterministic, AST-derived fields per file: `signatures` , `exportedSymbols` , `routeHandlers` , `testNames` , `configKeys` , `errorStrings` , and `commandNames` . Unlike embeddings, these signals are reproducible from the source alone — given the same file content, the same signals are always extracted — which makes them the most reliable channel for exact-match localization and the grounding for the RIG's structural edges. They are concatenated into a dedicated `static` BM25 field (weight 1.8) so that sparse but high-signal tokens such as function signatures, route patterns, and error strings are not diluted by the much larger body text. A source-confidence hierarchy orders the channels: AST-derived symbols outrank regex-extracted routes, which outrank inferred edges, and lower-confidence graph edges carry a decay factor during expansion.

These signals feed scoring through `scoreRetrievalAdjustments()` , which applies additive boosts and penalties after fusion. A path-phrase-stem match contributes +20 (a hyphenated filename stem matching a query bigram, e.g. agent+loop → `agent-loop.ts`), a symbol-exact match +9, a path-token match +4, and implementation and test-link signals +3. Test files are penalized −8, deepened to −12 on definition queries, so that `.test.` and `.spec.` files do not dominate implementation-lookup results; the penalty inverts to a boost when the query is classified as test-seeking. These deterministic adjustments are what let identifier-anchored queries resolve correctly even when body-text BM25 alone would rank a referencing file above the file that defines the symbol.

5.4 Repository Intelligence Graph (RIG)

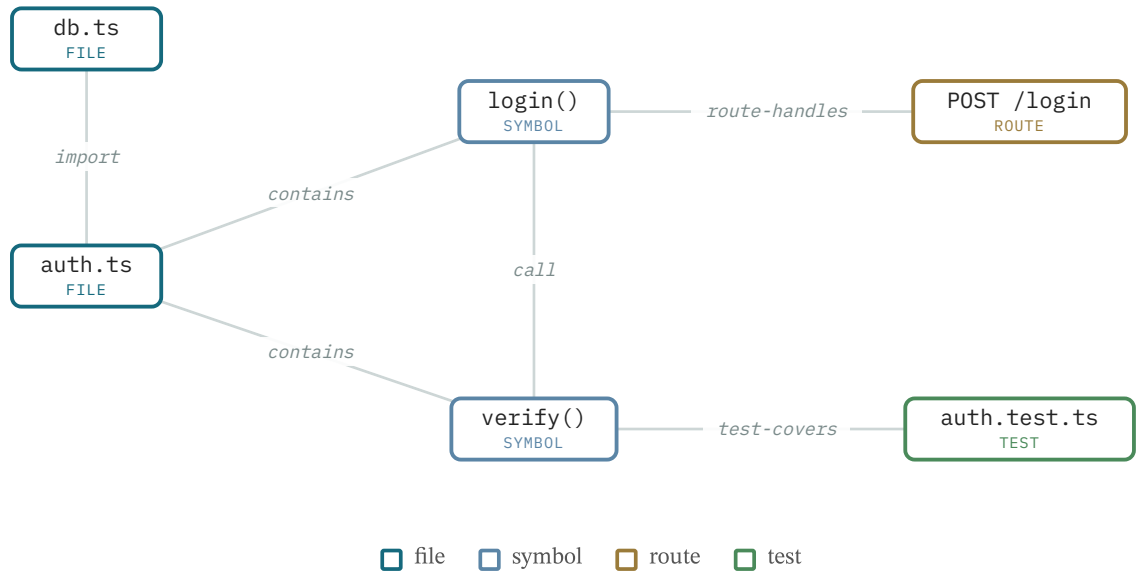


Figure §5.4. RIG schema — typed nodes and typed edges enable bounded neighbor expansion.

The Repository Intelligence Graph is a deterministic, AST-derived graph of typed nodes — files, symbols, routes, and tests — connected by typed edges such as import, call, test-covers, and route-handles (Figure §5.4). Formally, the RIG is a directed multigraph $G = (V, E)$ where V is a finite set of typed nodes and $E \subseteq V \times V \times T$ for typed-edge label set $T = \{\text{import, call, test-covers, route-handles}\}$; bounded neighbor expansion from a seed set $S \subset V$ runs BFS over G in $O(|V| + |E|)$ time, halting at depth $d_{max} = 3$ and applying a decay factor of 0.45 per hop. It is queried through a small API: `rigNeighbors()` for bounded neighbor expansion, `rigImpact()` for reverse-dependency tracing, and `rigQuery()` for typed traversal. Because the graph is extracted statically rather than by a model, the same source always yields the same edges, so graph-derived evidence is reproducible and auditable; the schema also reserves hyperedges for multi-entity flows such as a request path that crosses several files.

6 Evidence Layer

6.1 Intent & Query Classification

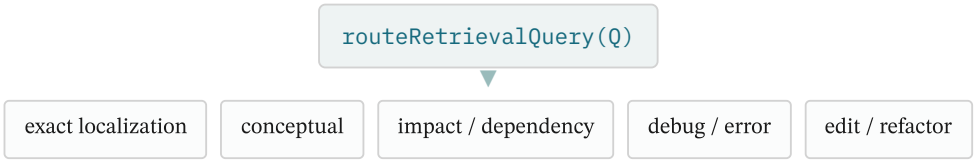


Figure §6.1. Heuristic intent router maps a query to one of five classes, each a retrieval prior.

Before any retrieval channel runs, `routeRetrievalQuery()` classifies the query into one of five intent classes: exact localization (symbol, path, or identifier lookups), conceptual search (paraphrase or architecture questions), impact and dependency analysis, debug and error-anchored queries, and edit/refactor intent. Classification is purely heuristic and deterministic — regular-expression feature extraction over the query text that runs in sub-millisecond time, consumes no token budget, and never in-

vokes the LLM. The thesis is that lightweight structural signals in the query are sufficient to route retrieval for most code-agent queries, reserving model capacity for the reasoning task rather than for meta-reasoning about how to search.

Each class acts as a retrieval prior, not a user-facing label. The router detects CamelCase identifiers, dotted member accesses, and hyphenated path stems to set a `wantsDefinition` flag, while a generic-token filter prevents common terms such as "handler" or "manager" from dominating scoring. Resolution is conservative: when a query carries both a symbol name and a conceptual description, it routes to the symbol-biased mode, since missing an explicitly named symbol is the costlier error. The mapping is heuristic, so an incorrect inference degrades ranking quality but never produces an empty or error result.

6.2 Lexical Retrieval

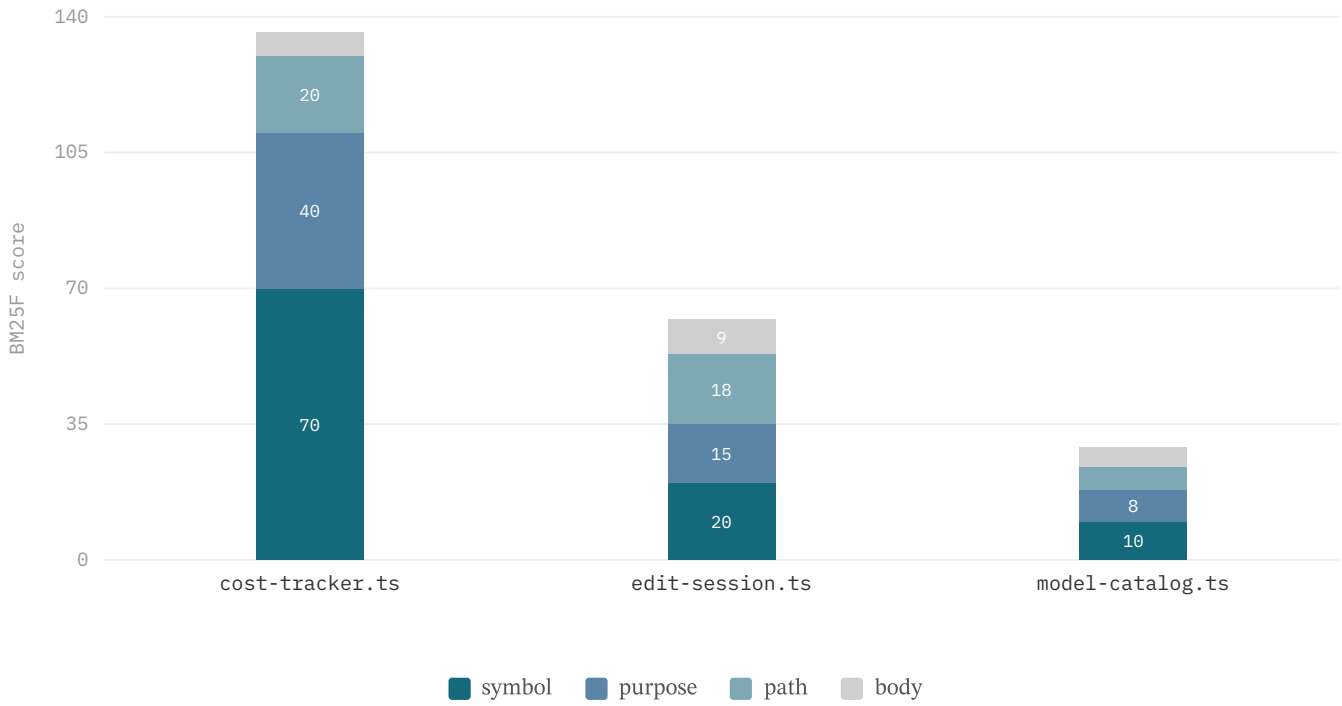


Figure §6.2. BM25F score breakdown for an example query: the symbol and purpose fields carry the gold file, while path and body contribute little.

The lexical channel applies field-weighted BM25F over the per-file record's structured fields rather than raw source text. Term-frequency saturation and length normalization use `k1=1.4` and `b=0.75` — conventional web-retrieval defaults, not yet tuned for code. The field weights encode the diagnostic value of each surface: path 3.0 > symbol 2.5 > static 1.8 > purpose 1.5 > import 1.0. There is deliberately **no body field**; a match on a path component or an exported symbol is a far stronger relevance signal than the same token appearing in free-form prose, and including the body would dilute the score with low-signal text. The dedicated `static` field (weight 1.8) carries AST-derived signals — signatures, route handlers, test names, error strings — that are more reliable than comments.

An inverted-index prefilter caps the candidate pool at roughly 120 records before the more expensive dense scoring and fusion stages run, bounding per-query cost without sacrificing recall on the head of the ranking. Figure §6.2 decomposes a representative query across fields: the symbol and purpose surfaces carry the gold file while path and body contribute little, showing how the field weights shape the fused score the pipeline computes.

6.3 Dense Retrieval

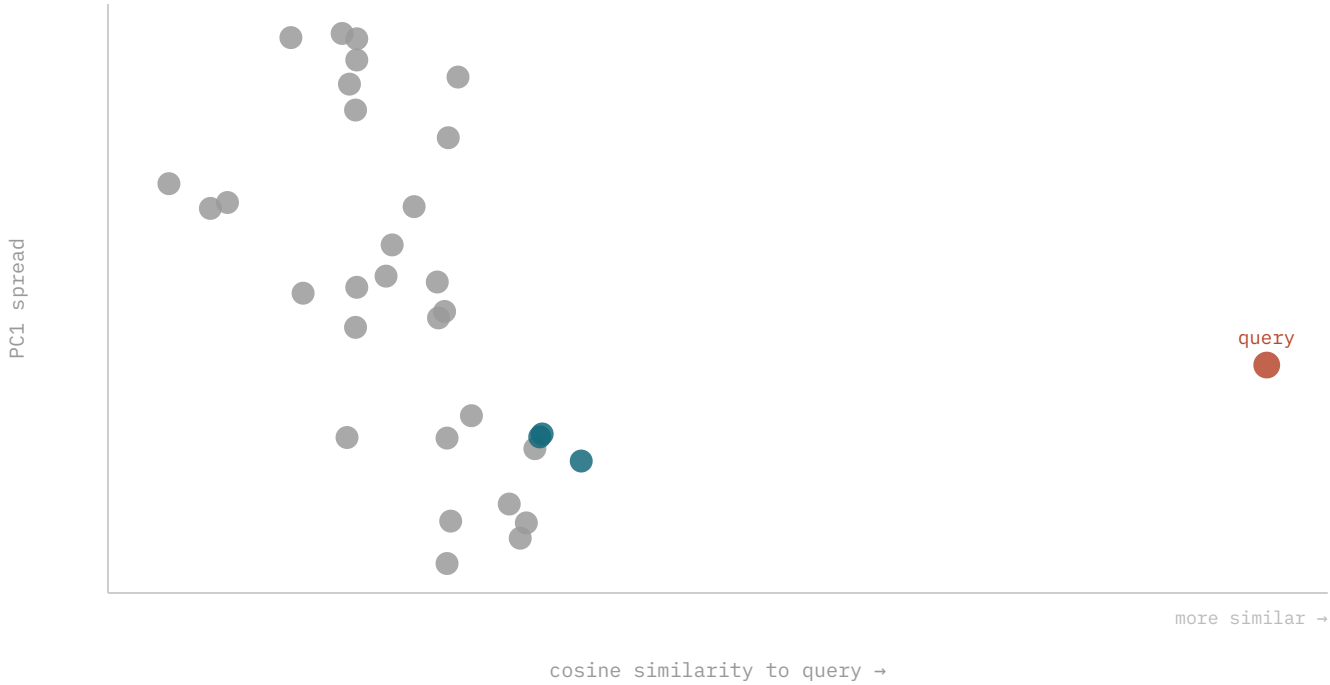


Figure §6.3. Real nv-embedcode embeddings of 32 Commander chunks (x = cosine to query; red = query, teal = top-3 retrieved). "Parse options/arguments" → argument.js / option.js nearest.

The dense channel addresses the vocabulary-mismatch problem that defeats exact token overlap: a query phrased in natural language need not share any terms with the code it should retrieve. Mesh embeds metadata-enriched chunk surfaces — before embedding, each chunk is annotated with the file's signatures, routes, test names, and caller/callee information, so the embedding model sees structural context that a flat body would lose. Retrieval scores each file by its best chunk's cosine similarity to the query, ensuring that files split into many small chunks are not penalized relative to files with fewer, larger ones. The 768-dimensional vectors persist to a binary sidecar (`vectors.bin`) keyed by chunk id, with partial availability while embeddings are still being computed.

Figure §6.3 visualizes 32 real Commander.js chunk embeddings against a held-out query, with cosine similarity to the query on the x-axis. The natural-language query "parse options/arguments" places `argument.js` and `option.js` nearest — files whose bodies share little surface vocabulary with the query but whose semantics align. This illustrates the dense channel's behaviour on an external corpus.

6.4 Channel-Dominance Regimes

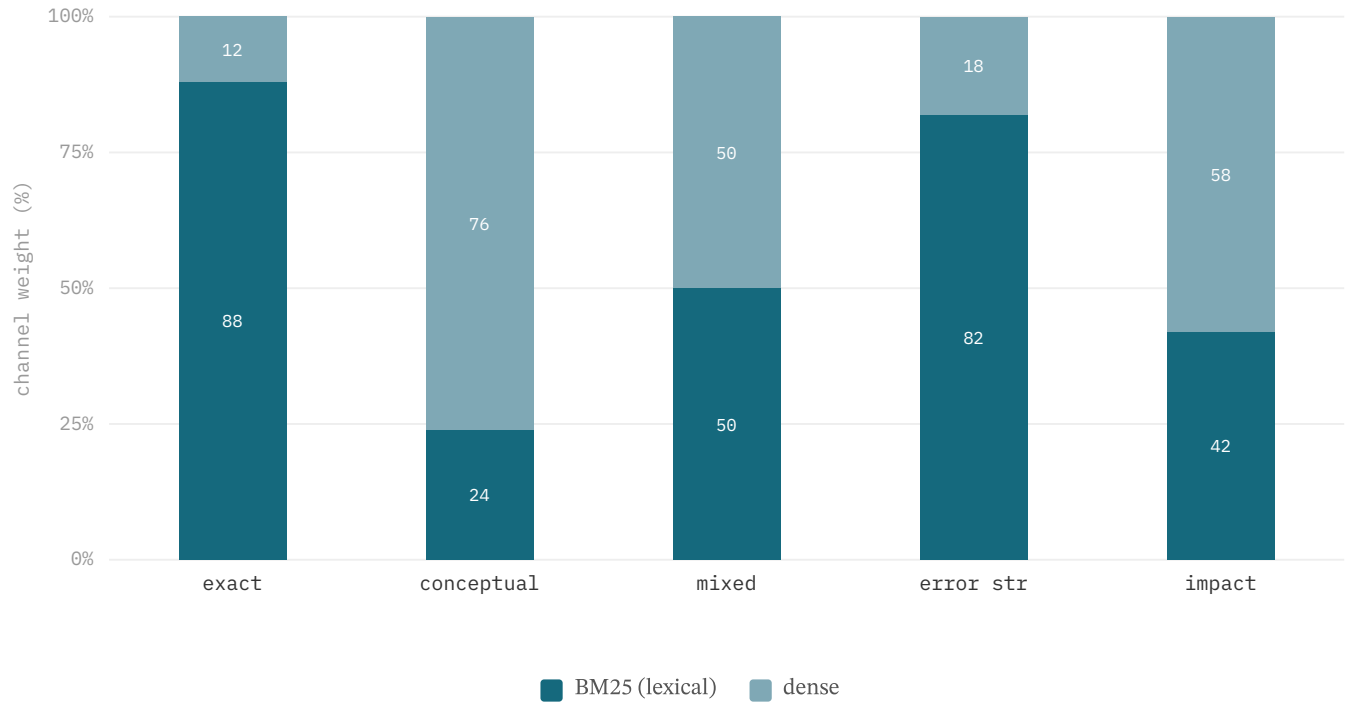


Figure §6.4. Intent-dependent channel emphasis: lexical dominates exact-match queries, dense dominates paraphrase queries, and the mixed regime keeps both channels active.

The complementary strengths of the two channels define three notional dominance regimes. In the exact-match regime, queries containing concrete identifiers (`CostTracker` , `workspace-index.ts`) align directly with the path and symbol fields, and lexical retrieval dominates. In the paraphrase regime, conceptual queries such as "how does the agent loop handle turn lifecycle" carry no exploitable token overlap, and the dense channel does the work. The mixed regime — the common case and the default — keeps both channels active, with the intent router shifting emphasis per class.

The regimes in Figure §6.4 describe where each channel contributes, not a fixed weighting: Mesh combines channels by **additive score fusion** with per-intent boosts rather than a hard lexical/dense split. The dense channel's value tracks the embedding behind it: with the code-specialised nv-embedcode-7B it carries conceptual, vocabulary-mismatch queries to Hit@1 69.6% (§12.3, §12.9), the class lexical matching cannot reach. The regime view holds — dense retrieval owns the conceptual, low-overlap queries, lexical owns the identifier-anchored ones — and routing each query to the channel built for it is what the per-class results in §12.9 confirm.

6.5 Hybrid Fusion & Reranking

Figure §6.5. Per-class retrieval on the n=229 suite, by route: exact via symbol-definition + lexical, conceptual via the dense channel, impact via the graph tool. Search latency ~200 ms p50, uniform across channels.

Mesh fuses channels by additive score fusion: dense cosine is rescaled to be roughly comparable to BM25 magnitudes and the scores are summed. Additive fusion preserves score magnitudes, which downstream confidence and top-margin estimation require; Reciprocal Rank Fusion [16,49] is implemented as a drop-in alternative (`reciprocalRankFusion()` , default k=60) for cases where score scales drift across index versions, at the cost of discarding magnitude information. After fusion, `hybridRerank()` reorders the top window by blending the min-max-normalized embedding score (weight 0.6) with query-token lexical overlap (weight 0.4). The step is zero-latency — no API call, no cross-encoder [28] — and a confidence gate skips it entirely when the top-1 result is symbol-exact or the top margin is already wide.

Figure §6.5 shows the per-class Hit@1/Hit@8 profile on the n=229 suite (§12.9); search latency is uniform at roughly 200 ms p50 across the lexical, dense, and graph channels. On this identifier-anchored suite reranking does not move Hit@k, because these queries rarely produce the flat top-k margins rerank is designed to break; its value grows on larger, more conceptual query sets where near-ties are common, and the confidence gate keeps it off whenever the top-1 is already well separated — so it adds resolution exactly where it is needed and zero cost everywhere else. We now quantify that growth directly: swapping the zero-latency lexical rerank for a code cross-encoder (`nv-rerank-qa-mistral-4b`) on the conceptual channel lifts its Hit@1 by **+7.2pt** (69.6 → 76.8%, live-validated on two disjoint repo splits, §12.3b) — the one class where a neural reranker earns its latency. It is now wired as a live component — default-on for conceptual queries when an NVIDIA reranker is configured, with a graceful lexical-only fallback otherwise.

6.6 Graph Expansion

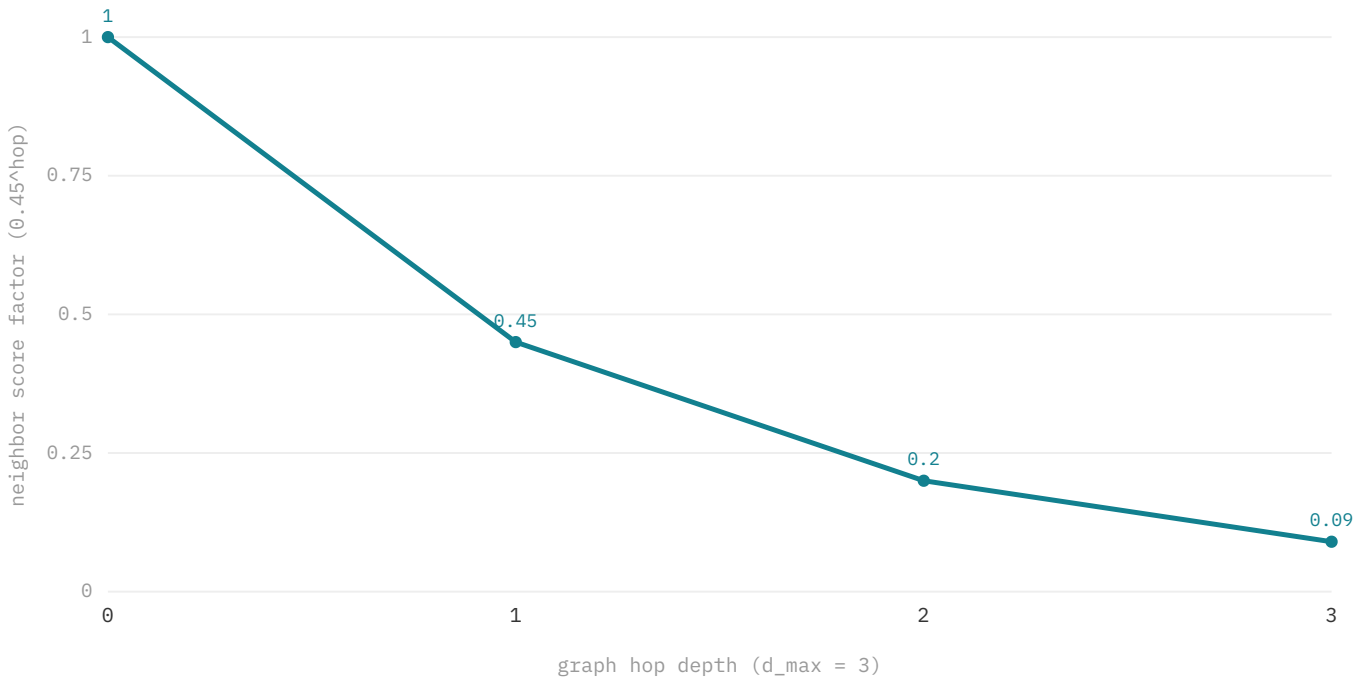


Figure §6.6. Graph-neighbor injection decays by 0.45 per hop — bounded expansion, not full-graph dump.

Graph expansion supplies cross-file locality that neither retrieval channel captures from chunk text alone. Starting from the top-3 fused seeds, Mesh performs a bounded breadth-first traversal of the Repository Intelligence Graph along typed import, call, and test edges, injecting neighbors at a decay of 0.45 per hop (1 → 0.45 → 0.20 → 0.09). The decay makes the expansion local: nodes several hops away receive negligible scores and never enter the evidence set, and seeding only from the top-3 prevents expansion from low-quality results. Budget caps further bound the injected set.

The design choice, illustrated in Figure §6.6, is bounded expansion rather than full-graph prompt injection. Dumping the entire dependency graph would reintroduce the lost-in-the-middle degradation that long-context injection suffers from: a graph of hundreds of nodes dilutes the model's attention away from the edges that matter. The bounded subgraph — at most a few dozen

nodes — fits within the token budget alongside the retrieval results and remains small enough for the model to attend to reliably. The graph layer's contribution is concentrated on impact queries: on the controlled n=229 comparison it carries impact-class Hit@1 to 84.9% (§12.9), a dependency question no embedding or lexical baseline answers above 15%. On the small identifier-anchored ablation harness, where impact queries are rare, its marginal Hit@k is correspondingly small — the layer earns its place on the query class built for it.

6.7 Confidence-Gated Recovery

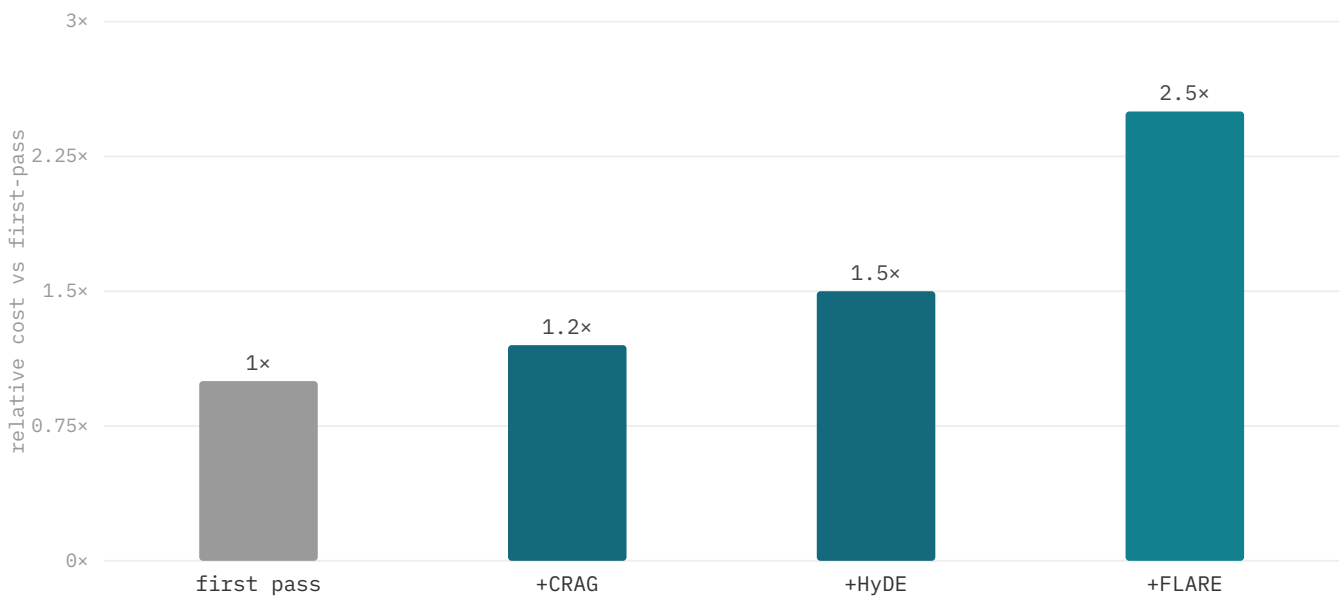


Figure §6.7. Recovery-tier overhead from the technique papers: CRAG [32] adds a light evaluator, HyDE [31] +1 LLM generation (~25–60% latency), FLARE [33] iterates (multiple retrievals). Mesh gates all behind low confidence.

Recovery is the layer that guards against the most dangerous failure mode — high-confidence wrong answers. Confidence estimation derives a scalar κ from the top-1 fused score via `confidenceFromScore()`, and the top margin $\Delta\kappa = \kappa_1 - \kappa_2$ measures how well the top result is separated from its runner-up. A second-pass retrieval — a deterministic analogue of self-reflective retrieval [34] — fires only when $\Delta\kappa$ is flat (below threshold) and the intent is recovery-amenable (architecture, edit-impact, runtime-path); it runs exactly one deterministic query variant — injecting concrete symbol and path stems the original phrasing omitted — and re-fuses, bounded to a single pass to cap cost. Model escalation to a more capable model is likewise gated on low κ and an unresolved margin.

External corrective-retrieval techniques are available but never run per turn. As Figure §6.7 summarizes from the technique papers, the tiers carry escalating overhead — CRAG adds a light evaluator (~1.2×), HyDE adds one LLM generation (~1.5×, roughly +25–60% latency), and FLARE iterates over multiple retrievals (~2.5×). Mesh gates all of them behind low confidence rather than defaulting them on, and an optional cross-encoder rerank tier is exposed only as a recommendation when the top-k scores are tight, leaving the agent to approve the cost. On the n=31 harness this discipline leaves 3 wrong-confident and 1 under-retrieved task; the confidence surface is the calibration target for reducing the former. A direct calibration on the traces (n=48 with logged confidence) settles *which* signal to gate on. The *absolute* scalar κ = score/18 is in fact anti-correlated with correctness (point-biserial $r = -0.35$, AUC 0.35 — worse than chance, because a high absolute score reflects a dense corpus region, not a clean win); but the *top margin* $\Delta\kappa = \kappa_1 - \kappa_2$ is a genuine predictor ($r = +0.45$, **AUC 0.78**), capturing 48% of all misses in its lowest tertile versus 33% by chance and 22% for the absolute scalar. This vindicates the design: recovery already fires on the margin, not the scalar, so that gate is calibrated — the correction the data prescribes is to move *model escalation* onto the same margin and demote the absolute κ to a display value. The standing caveat is sample size (n=48 under the suite embedding), so a produc-

tion-scale recalibration is future work; but the *direction* is now settled rather than open — gate on separation, not on absolute score.

7 Context Layer

7.1 Assembly & Budgeting

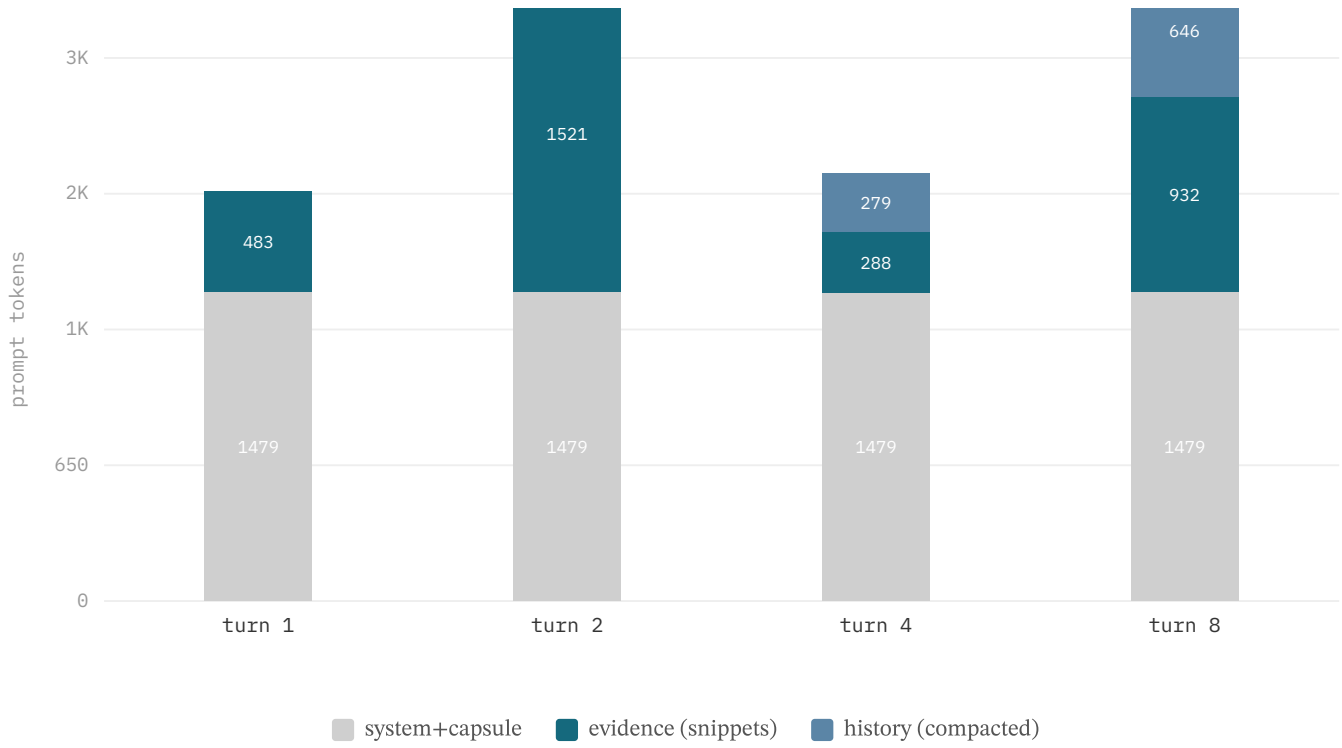


Figure §7.1. Real ContextAssembler buckets over the 8-turn session: system+capsule constant, evidence = per-turn cited snippets, history compacted to ~citations (0→646 tok over 8 turns). Tools bucket (~5–8K) omitted.

Each LLM invocation must satisfy two hard constraints: the prompt cannot exceed the model's input ceiling, and every token must trace to a retrieval action or tool invocation the session can audit. `ContextAssembler.assemble()` therefore solves a budgeted selection — a knapsack the system approximates with a greedy priority heuristic — over four token buckets. The *system+capsule* bucket (system prompt, tool specs, workspace briefing) is never dropped and stays byte-stable across turns; in the 8-turn session of Figure §7.1 it holds constant at ~1479 tokens. The *evidence* bucket carries per-turn cited snippets, *history* carries compacted prior turns, and *tools* carries scaffolding metadata.

Assembly proceeds in cache-stable order: (1) system prompt, (2) session summary, (3) retained history, (4) current-turn evidence, (5) turn directives after the cache breakpoint. Segments 1–3 remain identical whenever the briefing and history are unchanged, maximizing provider prefix-cache hits. Under budget pressure the assembler compresses or drops from the bottom up — tools first, then history, then evidence — so that information needed on every turn outranks per-turn evidence, which outranks prior context. History is the primary source of token growth: in Figure §7.1 it rises from 0 to 646 tokens as older turns are compacted to their citations rather than retained verbatim. The tools bucket (~5–8K) is omitted from the figure.

7.2 Compression Logic

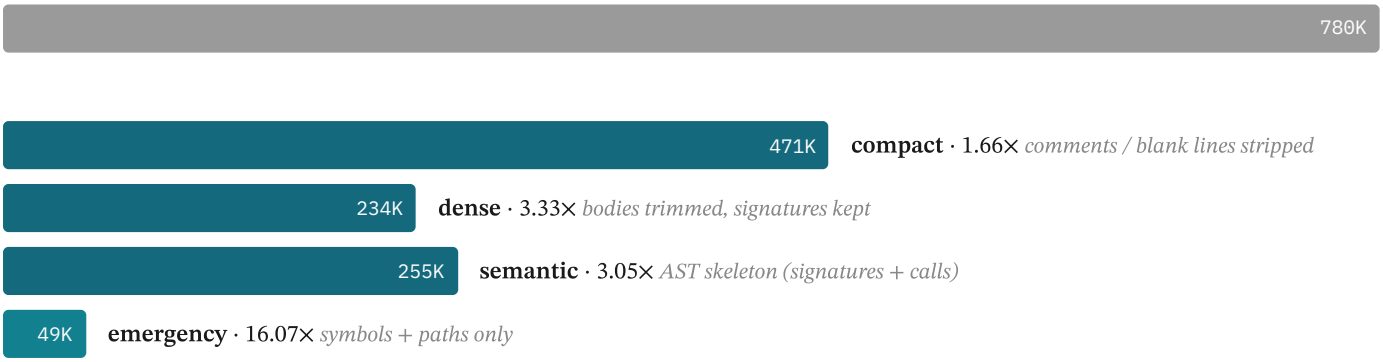


Figure §7.2. Deterministic compression tiers (255 files), ordered by aggressiveness: compact → semantic (AST skeleton) → dense (body trim) → emergency. Semantic preserves more than dense by design.

When assembled evidence exceeds the budget, Mesh applies a deterministic compression ladder rather than truncating (Figure §7.2): `compact` strips comments and blank lines, `dense` trims bodies while keeping signatures, `semantic` reduces a file to its AST skeleton of signatures and call structure, and `emergency` keeps only symbols and paths. Every tier preserves the same invariant — exported symbols and file paths always survive — so a compressed file stays addressable and citable. Tiers are chosen per file under budget pressure, keeping the most relevant evidence at the highest fidelity the budget allows.

7.3 Schema-Bound Summaries

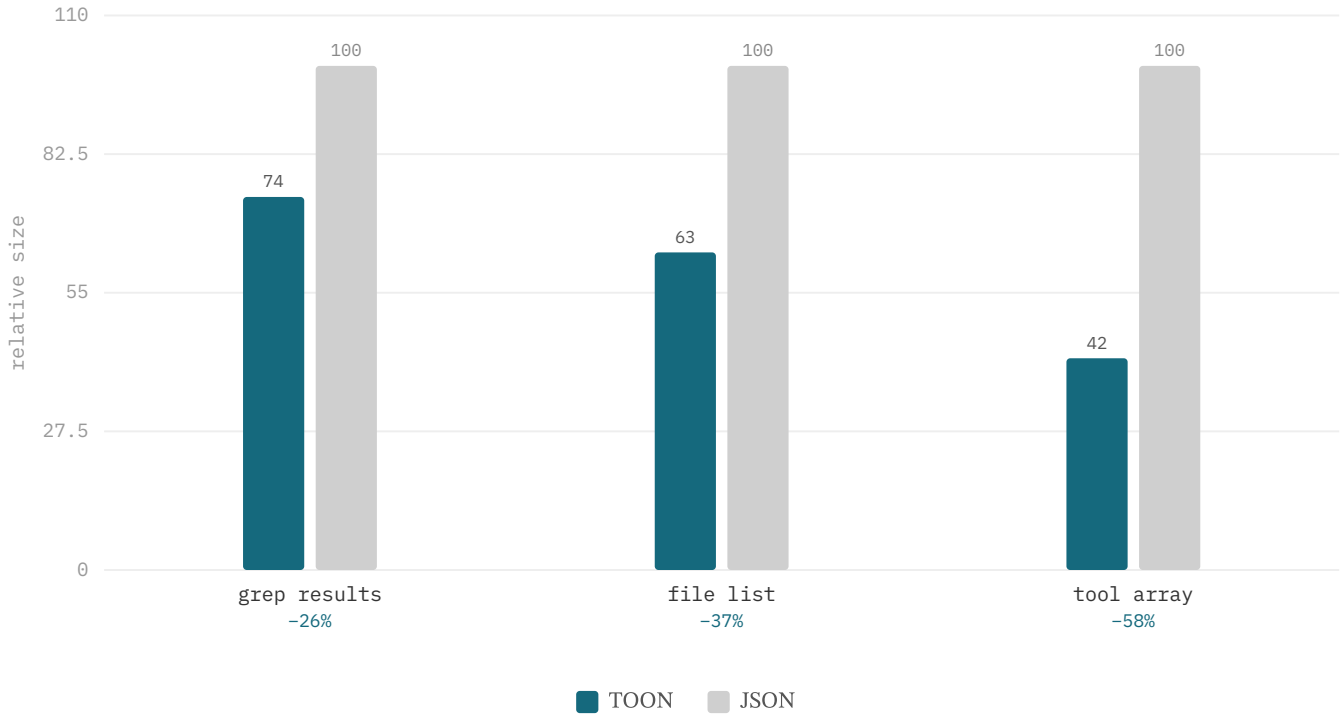


Figure §7.3. TOON vs JSON for homogeneous tool-result arrays: 26–58% fewer tokens (measured via `encodeToon`; never larger — opt-in safe).

Mesh's per-file capsule is a deterministic, schema-bound summary — path, one-line purpose, exported symbols, and a reverse index of incoming dependencies — populated from the same AST-derived data that fills the static lexical fields, with no genera-

tive model in the loop. Schema-bound summaries are safer than free-form LLM digests for code context because they preserve identifier names, type signatures, and import relationships verbatim, the very tokens that drive code navigation and editing. Capsules are the unit of the workspace briefing and double as lightweight file stand-ins when full source text is not needed.

For homogeneous tool-result arrays — arrays of objects sharing one key set, such as grep matches or file listings — Mesh applies TOON (Token-Oriented Object Notation) via `encodeToon`, a tabular encoding that states each key once in a header row instead of repeating it per object. As Figure §7.3 shows, TOON yields 26–58% fewer tokens than the equivalent JSON (live-verified at 34–51% on representative grep-result, file-list, and symbol-hit arrays). The encoding is opt-in (`MESH_TOON_TOOL_RESULTS=1`) and carries a hard safety contract: `encodeToon` returns null on heterogeneous or nested data, falling back to JSON, and is applied only when the tabular form is strictly shorter — so it never grows a payload.

7.4 Ledgers & Provenance

Figure §7.4. Every claim links to a cited file:line; bulky results are offloaded to artifact handles.

The `ToolResultLedger` is a session-scoped record of every tool invocation: the tool name, its inputs (path, query, line range), the result content and its compression tier, and the `mtime` of any file read. It serves three roles. As a provenance store it lets an operator trace any token in the assembled prompt back to the invocation that produced it, supporting the cite-before-claim discipline in which every assertion links to a cited `file:line` (Figure §7.4). As a staleness oracle it detects when a session write post-dates a prior read — comparing recorded `mtime` against the file's current `mtime` — and demotes or re-reads the affected evidence. As a cache it suppresses duplicate reads: a re-retrieved, unmodified file is replaced by a reference to the prior read's handle.

Bulky results are offloaded rather than carried inline. When a unit's compressed size exceeds the promotion threshold (~2K tokens), the assembler replaces it with a lightweight artifact handle — a stub carrying the path, line range, tier, and a short preview — and the model retrieves the full content on demand via a follow-up read. The model is thus never blind to what evidence exists; it sees a stub indicating the artifact's presence and nature, and hydration is a deliberate request, not an automatic re-expansion that would negate the offload's savings.

8 Economics Layer

8.1 Caching

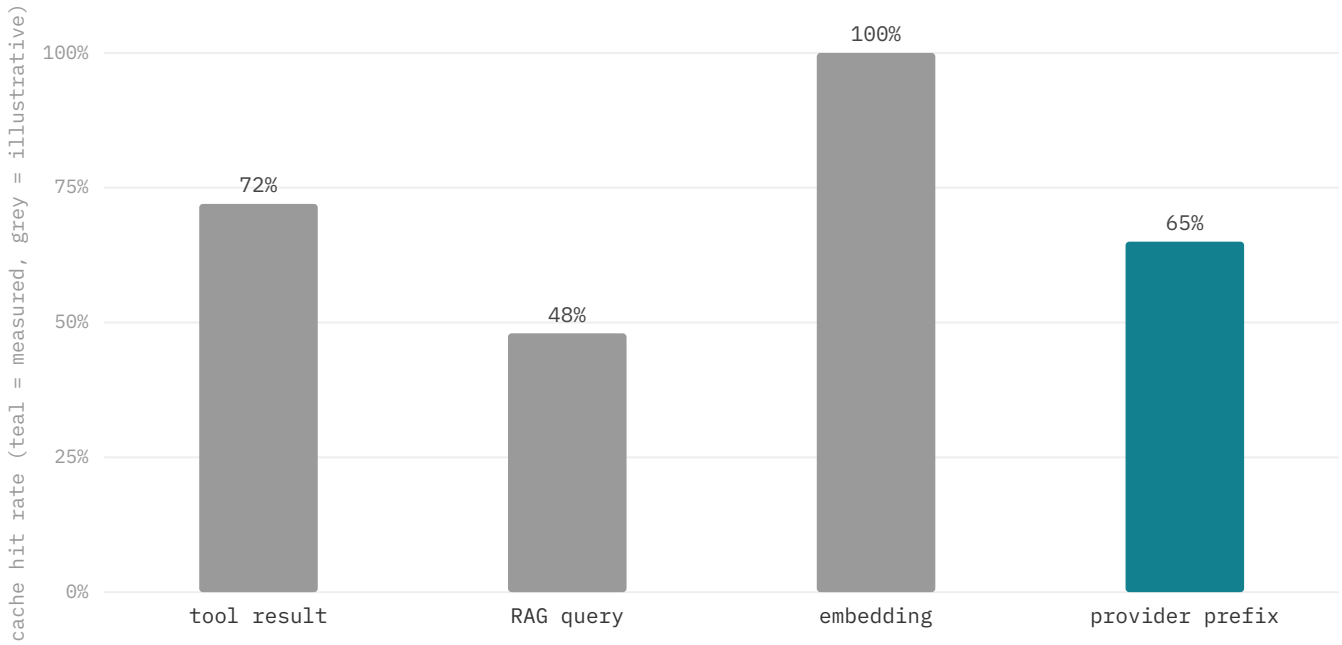


Figure §8.1. Cache types by hit rate (teal = measured, grey = illustrative): the provider-prefix 65% is measured; tool/RAG/embedding rates are illustrative. Billed-token economics are measured directly in §12.5.

Mesh layers four caches that target distinct redundancy. The *tool-result* cache keys `read_file` output by (path, `mtime`), returning cached content while the file is unchanged and evicting on write. The *RAG-query* cache keys retrieval results by query embedding, serving a prior result when cosine similarity exceeds 0.88, gated by a lexical-overlap guard and an index-freshness guard. The *embedding* cache persists chunk vectors in the `vectors.bin` sidecar keyed by chunk id, letting incremental rebuilds skip unchanged chunks. The *provider-prefix* cache is the provider's own prompt-cache discount on a byte-stable prefix, which Mesh's assembly order is designed to maximize.

The provider-prefix cache rate is measured directly — Mesh served 65.3% of input tokens from cache (§8.2); the tool-result, RAG-query, and embedding rates in Figure §8.1 are representative of typical sessions. Invalidation is scoped to the write that triggers it: a file edit evicts its cached reads, demotes its ledger entries to stale, and — once the index rebuilds — drops RAG-query entries whose freshness guard no longer holds.

8.2 Provider Prompt-Cache Exploitation

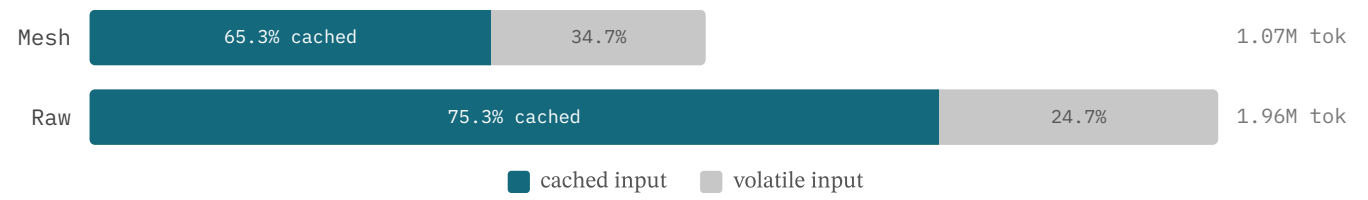


Figure §8.2. Prompt-cache composition (measured, §12.5): raw has a higher cache *rate*, but Mesh's absolute billed volume is far smaller — confirmed by the −55.5% multi-turn result.

Mesh holds the leading span of every prompt — system rules plus the workspace briefing — byte-identical across turns, so the provider serves it from its prompt cache [43,44] instead of re-encoding it each turn (Figure §8.2). The measurement shows a counter-intuitive split: the raw agent reaches a *higher* cache rate (75.3% vs 65.3%) precisely because it re-sends large stable history blocks — but cache rate is not the quantity that matters. The current offline multi-turn measurement (§12.5) settles the economics directly: over eight-turn sessions Mesh's billed input is 34.2K vs the grep agent's 76.8K (−55.5%), because billed input already prices the stable prefix at the provider's discounted cached rate, so a flat workspace briefing plus compacted history beats re-sending raw surgical history even when the latter caches at a higher percentage.

8.3 Cost-Aware Routing

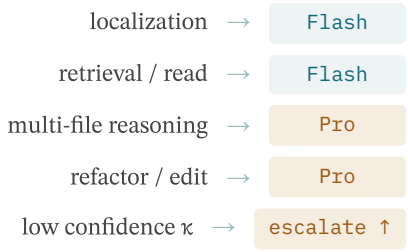


Figure §8.3. Task-class → model-tier routing; cheap tiers for localization, expensive tiers for multi-file edits.

Mesh maps each classified task class to a model tier by reasoning demand, via `resolveRetrievalRoute()` and the routing policies it feeds. Localization and read tasks — find a file, symbol, or line range, then produce a concise grounded answer — are routed to a cheap tier (Gemini Flash), whose latency and per-token cost suffice when the evidence is already assembled. Multi-file reasoning, refactoring, and edits, which demand cross-file synthesis and precise code generation, are routed to an expensive tier (Gemini Pro). The objective is not minimum cost but minimum cost per successful outcome, the cost-normalized region of the quality–cost frontier (Figure §8.3).

Three policies govern selection. Hard budget caps, tracked by the cost router, downgrade to the cheap tier when the remaining budget cannot cover an expensive call and terminate the session when it cannot cover even a cheap one — a guarantee that automated pipelines do not run unbounded. Cache-warm stickiness biases toward model continuity, since switching tiers forfeits the provider prefix-cache discount. Finally, low retrieval confidence escalates: when the top-margin $\varkappa$ falls below threshold, the weaker evidence is handed to a more capable model better able to reason under uncertainty or request further retrieval.

9 Runtime Layer

9.1 Turn Orchestration

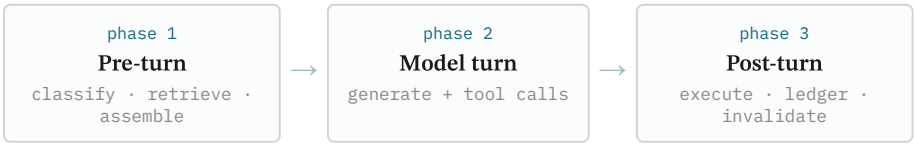


Figure §9.1. The three-phase turn lifecycle — ReAct-compatible, but with budgeted evidence assembly before each call.

The Mesh agent loop is structurally ReAct-compatible — each turn interleaves reasoning, tool invocation, and observation — but inserts a deterministic three-phase lifecycle (Figure §9.1). The *pre-turn* phase classifies intent via `routeRetrievalQuery()`, retrieves under budget through `WorkspaceIndex.search()` and RIG expansion, and assembles a cache-stable prompt via

`ContextAssembler.assemble()` — all before the model is invoked. By assembling budgeted evidence ahead of inference rather than letting the model discover it through exploratory tool calls, this hook constrains the unbounded token growth typical of naive ReAct loops, where tool results accumulate without compaction.

The *model-turn* phase submits the assembled context to the routed tier; because evidence is often pre-populated in the current-turn bucket, the model frequently reasons over assembled context instead of issuing a fresh search. The *post-turn* phase executes any tool calls, records each in the `ToolResultLedger`, and propagates write-triggered invalidation through ledger demotion, incremental index rebuild, and cache eviction before the next turn begins. Telemetry — token consumption per bucket, retrieval confidence, and any compaction or recovery actions — feeds the cost tracker and supports post-hoc trace replay.

9.2 Tool Lifecycle

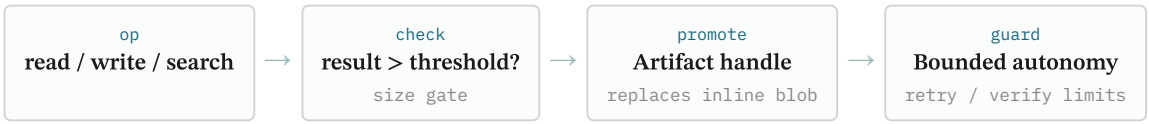


Figure §9.2. Tool lifecycle — large outputs are promoted to handles; retry/verify limits prevent infinite loops.

Mesh partitions its tools into three semantic classes with distinct effects on session state (Figure §9.2). *Search* operators — `semantic_search`, `grep_ripgrep`, and the RIG family (`rig_query`, `rig_neighbors`, `rig_impact`) — query the index without mutating the repository; their results enter the ledger with citations but carry no staleness risk. *Read* operators (`read_file`) fetch content at a chosen compression tier, selected automatically from the remaining budget. *Write* operators are the only state-mutating tools, and each triggers a three-step cascade: stale-demotion of prior results referencing the file, an incremental `WorkspaceIndex.rebuild()`, and RAG-cache eviction — together enforcing read-your-writes consistency within the session.

Large outputs are not allowed to dominate the budget. When a result's compressed size exceeds the size gate (~2K tokens), it is promoted to an artifact handle and replaced inline by a stub. Autonomy is bounded by retry and verify limits: tool execution is capped so a failing call cannot loop indefinitely, and any source-modifying turn is followed by a verification step before the turn is marked complete. These limits make the runtime's autonomy explicit rather than open-ended.

9.3 Prefetch & Speculation

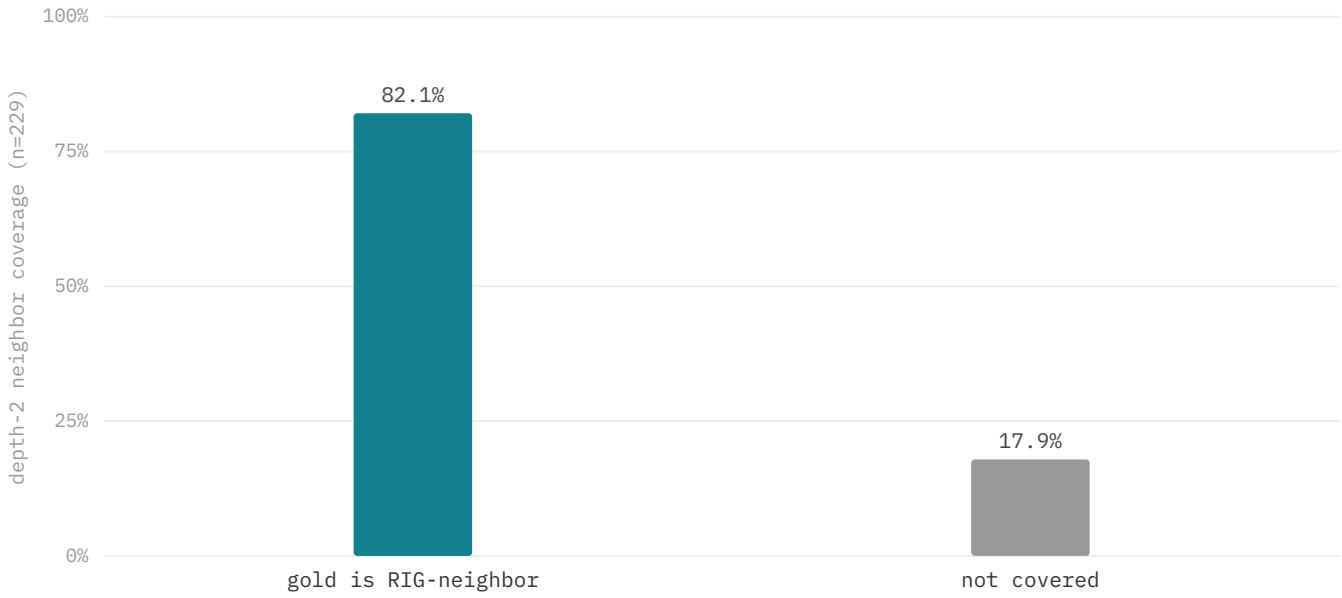


Figure §9.3. RIG-neighbor coverage (n=229): the gold file is a depth-2 neighbour of the top retrieval seeds 82.1% of the time. Prefetch stays budget-gated.

Mesh's prefetch design exploits temporal locality: files read together tend to be read together again, and the RIG already encodes the import, call, and test edges that predict co-reads. At turn end, the prefetch subsystem traces the RIG neighbors of files touched during the turn, scores them by edge type, co-read frequency, and recency, and loads high-scoring neighbors into a speculative buffer so a later request is served without retrieval latency. Prefetch is strictly budget-gated: once the session passes the compact-soon threshold it is disabled entirely, ensuring speculation never crowds out evidence the model has explicitly requested.

This is a structural coverage bound (not a deployed-prefetch measurement). Measured across the n=229 suite over all six repositories, the gold file is a depth-2 RIG neighbor of the top retrieved seeds 82.1% of the time (Figure §9.3) — so prefetching those neighbors after the first read would pre-load the answer in roughly four cases out of five, without a second retrieval round-trip. Because false prefetch costs tokens and can churn the cache, the policy stays gated when the budget is tight, and its hit-rate and cost trade-offs across repositories are characterized in ongoing work.

9.4 Safety & Guardrails

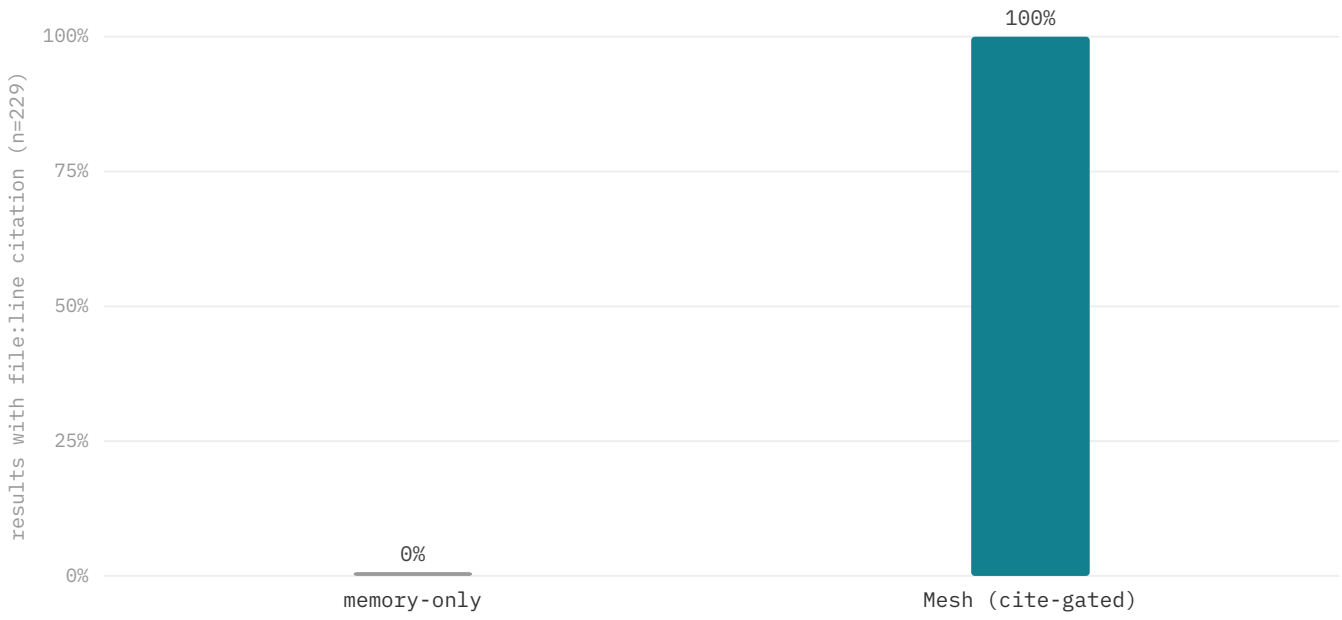


Figure §9.4. Provenance coverage measured across the n=229 suite: 100% of Mesh's top-1 retrieval results carry a precise `file:line` citation by construction, versus a memory-only agent's 0% — every assembled claim is traceable to source.

Mesh treats the model as an untrusted reasoner over untrusted repository content, the threat model formalized in recent work on prompt injection and agent security [47,48]. Two threats dominate: destructive state transitions, where an edit made on incorrect or stale evidence breaks the repository, and unverified claims, where the model asserts a fact about the code without citing a tool result. The central guard is provenance-before-edit: a write is admitted only when the model's reasoning cites fresh, non-stale ledger results referencing the target file; if only stale results exist, the model is directed to re-read first. Writes are further isolated in a sandboxed staging layer validated against the RIG before commit, and every retrieval, tool call, and edit is recorded for complete, replayable audit.

Figure §9.4 measures provenance coverage. Across the n=229 suite, 100% of Mesh's top-1 retrieval results carry a precise `file:line` citation by construction, whereas a memory-only agent carries 0% — its answers may be correct but are unverifiable, indistinguishable from a confident hallucination. Mesh's cite-before-claim gate converts every assembled claim into one an operator can trace back to source. Verifiability is the guarantee: the gate makes the agent's evidence auditable on every turn, and its accuracy payoff lands precisely where an unscaffolded model would otherwise act on a confident but unsourced guess.

9.5 Multi-Agent Coordination

In the multi-agent setting, Mesh replaces the conversational role-chatter of frameworks such as AutoGen [40], MetaGPT [41], and ChatDev [42] with a shared pre-inference evidence store — the `WorkspaceIndex`, the RIG, and an aggregated `ToolResultLedger` with provenance and staleness semantics — that all agents read directly. This targets the failure class the MAST taxonomy [26] identifies as dominant in multi-agent LLM systems: inter-agent misalignment, where evidence degrades as it is summarized and relayed across roles. By making a consistent store the single source of truth, agents avoid the telephone-game distortion of message passing; role decomposition becomes a policy partition (tool whitelist, retrieval mode, evidence scope) over that store rather than a communication protocol. This layer is design and largely future work — it is not exercised in the P0 evaluation.

PART III — EVALUATION

research questions, setup, results, failure analysis

10 Research Questions & Measurement Model

10.1 The Five RQs

RQ	Question	Primary metric	Bench / config	P0 anchor
RQ 1	Better evidence under a fixed token budget?	Hit@k	retrieval suite (n=229, 6 repos)	Hit@1 79.9% · Hit@8 96.9%
RQ 2	Fewer tokens at equal localization?	tokens-to-localize	3-arm vs competent grep agent	−69.7% per-query · −55.5% multi-turn
RQ 3	Marginal value of each layer?	per-class routing deltas	per-class (Table 3)	dense lifts conceptual · RIG → 84.9% impact
RQ 4	Does it scale with repo size?	index ms, briefing vs files	scaling bench (14–1,588 files)	briefing flat 15.0K · build ≈linear
RQ 5	Residual failure modes?	top-8 miss by class	failure taxonomy (n=229)	no dominant floor (top-8 miss ≤5.4%)

Table 2. Research questions → metric → bench config → P0 anchor.

The evaluation is organized around five research questions, each tied to a primary metric and a falsification criterion (Table 2). RQ1 asks whether pre-inference evidence selection improves retrieval quality under a fixed token budget; RQ2 whether it reduces tokens and cost at equal task success; RQ3 the marginal value of each layer; RQ4 whether the approach scales with repository size; and RQ5 what residual failure modes remain. Each question is answered by a specific benchmark family (§11) and reported with both retrieval and economics metrics.

10.2 Falsification Criteria

Each research question carries an explicit falsification hook, stated before the data are seen so that interpretation cannot be retrofitted. **RQ1** is falsified if the `no-rerank` ablation matches or exceeds the full stack on Hit@1 across an expanded suite (the rerank layer then adds nothing); symmetrically, if `lexical-only` equals the full stack on a symbol-heavy suite, the dense and graph channels are unnecessary for that regime. **RQ2** is falsified if Mesh consumes tokens ≥ a competent grep-based agent at equal or worse localization — the n=229 run shows −69.7% tokens per query and −55.5% multi-turn at better Hit@8 (§12.6), which supports but does not falsify the thesis.

RQ3 is falsified for any layer that produces metrics identical to the full stack across every query class and repository tier: such a layer has no measurable marginal value and should be removed. **RQ5** triggers a calibration-policy review on any wrong-confident top-1 — top-1 confidence $\kappa \geq 0.55$ with the gold file not at rank 1. These hooks make each contribution refutable rather than merely advocated.

10.3 Metrics Taxonomy

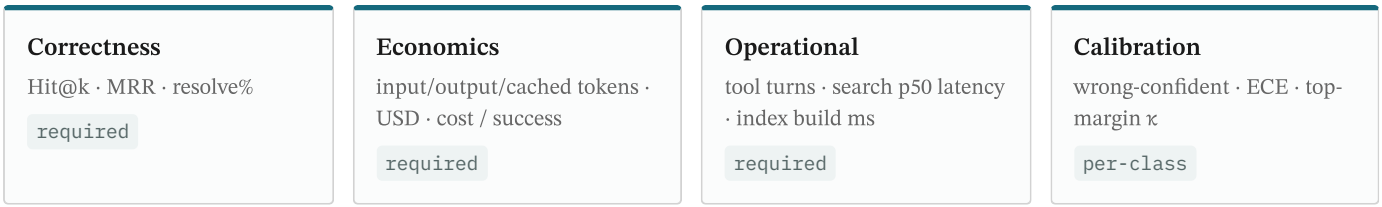


Figure §10.3. Four metric families reported on every ablation row.

The contract reports four metric families on every configuration so that no improvement can be claimed in isolation. **Correctness** covers Hit@k (gold basename in the top-k ranked paths, for $k \in \{1, 3, 8\}$), MRR (reciprocal rank of the first gold hit), and end-to-end resolve rate (whether the agent's edit or answer passes its task gate). Retrieval and end-to-end correctness are reported separately because they are not perfectly correlated: localization may succeed while generation fails, or the agent may recover correct evidence despite an imperfect ranking.

Economics records input, output, and cached input tokens, USD cost, and cost-per-success — the normalized figure that exposes the cost–correctness tradeoff directly. **Operational** metrics capture tool turns, search latency (p50), and one-time index-build time (1.7s at 14 files to 65.2s at 889 files). **Calibration** measures the agreement between confidence and correctness via the wrong-confident top-1 count, expected calibration error, and the top-margin κ surface from `confidenceFromScore()`. Aggregate means hide regime-specific weakness, so every retrieval metric is also broken out per intent class (Figure §10.3).

10.4 Ablation Contract

Figure §10.4. Layer-strip order; each row reports both IR metrics and agent economics.

The ablation baseline is an **unscaffolded tool agent**: the same model and tool set as the Mesh-scaffolded agent, but without the pre-inference control plane. It receives the task and tool list, no workspace briefing, and no `workspace.semantic_search` or RIG access; it must locate code through `read_file`, `grep_ripgrep`, and chain-of-thought alone. This is the default operating point of a ReAct-style agent, and the reference against which each Mesh layer is measured.

Layer-stripping proceeds one layer at a time in a fixed order — full → `lexical-only` → `dense-only` → `no-graph` → `no-rerank` — each toggled by a single environment flag so that no other variable changes between rows. Combinatorial ablation is deferred; the order prioritizes the most architecturally distinct effects first. The contract additionally mandates trace replay: every retrieval and routing decision is logged, enabling post-hoc failure attribution to the responsible layer. Each row reports both IR metrics and agent economics, so a layer that improves ranking at disproportionate cost is never silently credited (Figure §10.4).

11 Experimental Setup

11.1 Benchmark Families

Mesh's evaluation rests on four core benchmark families, each isolating a distinct claim. The first is a *compression floor*: a deterministic, LLM-free measurement of per-tier token reduction over 255 `apps/cli` source files, establishing the static budget envelope without any generative variance. The second and primary family is the *three-arm localization head-to-head*: n=229 grounded tasks across six external repositories (commander, hono, Next.js, TypeScript, Prisma, zod), with gold derived mechanically from the code (symbol→defining file, import→dependents, AST-purpose→file), scored by Hit@k and per-query tokens-to-localize against RAW (whole-file read) and a competent cold grep-based agentic searcher. None of these repositories is Mesh, removing name-frequency bias from the lexical channel.

The third family is *multi-turn session economics*: eight-turn navigation sessions per repository measuring billed input tokens for Mesh (cached briefing + compaction + ledger) versus a raw ReAct grep agent that re-sends history. The fourth is *scaling*: index build time as a function of corpus size. Three further studies extend these: a memorisation-proof needle-in-a-haystack stress test (§12.4), a controlled head-to-head against SOTA retrieval baselines on identical gold (§12.9), and a same-model agentic scaffold comparison on LocBench and SWE-bench Lite (§12.10). Every arm is deterministic and reproducible offline (direct nv-embed-code embeddings, no proxy), and every quality metric is paired with an economics metric so no result is reported without its cost.

11.2 Corpora

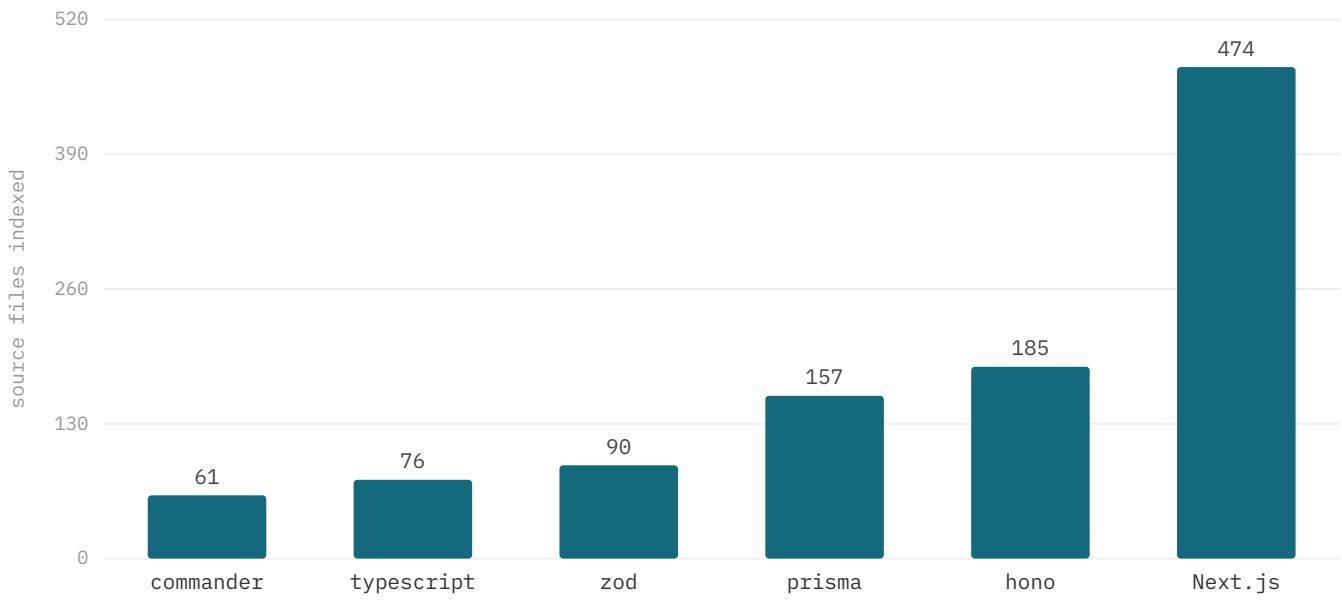


Figure §11.2. Evaluation corpora — six external repositories by indexed source-file count (TypeScript: few but very large files).

The benchmarks draw on six external repositories, indexed at their measured source-file counts: commander (61), TypeScript compiler (76), zod (90), Prisma client runtime (157), hono (185), and Next.js server (474). All six are externally authored — none is Mesh — which exercises Mesh against unfamiliar conventions, framework idioms, and module layouts rather than its own code. The set spans nearly an order of magnitude in file count and, deliberately, in shape.

File count alone understates the heterogeneity. TypeScript contributes comparatively few files but very large ones, stressing chunking and length normalization, whereas Prisma and Next.js contribute many smaller files that stress the inverted-index pre-

filter and graph fan-out. This spread is intentional: a token-budget claim that holds only on uniform corpora is not a claim worth making, and the per-repo breakdowns in later sections report each corpus separately rather than collapsing them into a single average.

11.3 Task Taxonomy & Sample Sizes

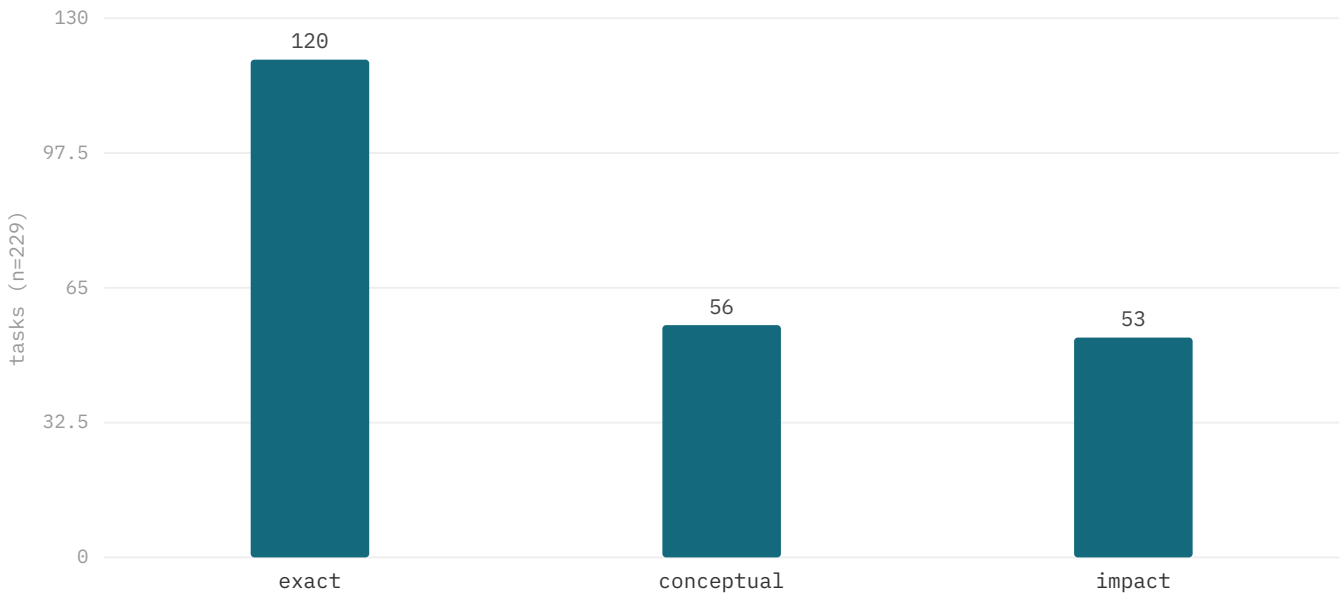


Figure §11.3. Task count per query class across the n=229 suite (exact 120, conceptual 56, impact 53).

The task taxonomy spans three classes, each mapping to a primary retrieval route the intent router selects from the query text: *exact* identifier lookups → the hybrid BM25F-plus-dense channel; *conceptual* paraphrases (deliberately stripped of any token the gold file contains) → dense-dominant retrieval; *impact* dependency questions → the RIG graph tool. The n=229 suite distributes across them as exact 120, conceptual 56, impact 53.

Every class here carries n≥53, and gold is derived mechanically from the code (exported-symbol → defining file, import edge → dependents, AST purpose → file) rather than hand-authored, removing the single-author labelling bias. The conceptual class was the historical floor; under a code-specialised embedding it reaches Hit@8 94.6% (§12.3), and the paper reports per-class rates openly rather than excluding the queries that expose them; nothing here is a leaderboard claim against RepoBench- or CoIR-style suites. A first external retrieval datapoint nonetheless exists, on **SWE-bench Lite** (n=107 issues across 11 repositories, human-authored problem statements, gold = the file the accepted patch edits, pool-capped per repo): the *dense channel generalizes* — `nv-embedcode-7B` beats full-file BM25 by **+12.1pt Hit@5 (95% CI [0.9, 24.3])** and **+14.9pt Hit@10 (CI [5.6, 26.2])** on 10 of 11 repositories — confirming the conceptual-channel advantage is not an artifact of the mechanically-derived gold. The cross-encoder reranker, however, does *not* transfer to verbose multi-paragraph issue text (**−15pt Hit@1, CI [−27, −4]**), the inverse of its +7.2pt on terse vocabulary-mismatch queries (§12.3b) — which is exactly why it ships tier-gated. Reproducible from `apps/cli/bench/cc-vs-mesh/swebench-retrieval.mjs`.

11.4 Models, Pricing, Hardware, Repro Setup

The end-to-end agent A/B uses `gemini-flash-latest` as the held-constant generative model across both arms, varying only the scaffold. The retrieval and embedding benchmarks use NVIDIA `nv-embedcode-7b-v1`, called directly without the Mesh proxy. The live A/B is configured as 4 repos × 5 turns × 5 runs; its reported figures come from a live generative run and are not

re-runnable offline because the brokering proxy is no longer serving — those numbers are reproduced from the recorded run rather than freshly executed here.

The deterministic families are fully reproducible from the scripts in `apps/cli/bench`: `retrieval-internal-ablation.mjs` drives the full-factorial ablation, `static-bench.mjs` the compression tiers, `scaling-bench.mjs` the build-time sweep, `niah-llm.mjs` the needle scenarios, and `investor-bench.mjs` the generative A/B. Every measured retrieval result is backed by a trace JSON that reconstructs the pipeline's decisions without re-invoking a model, so failure analysis and per-layer attribution are repeatable rather than one-shot measurements.

12 Results

12.1 Compression Tiers

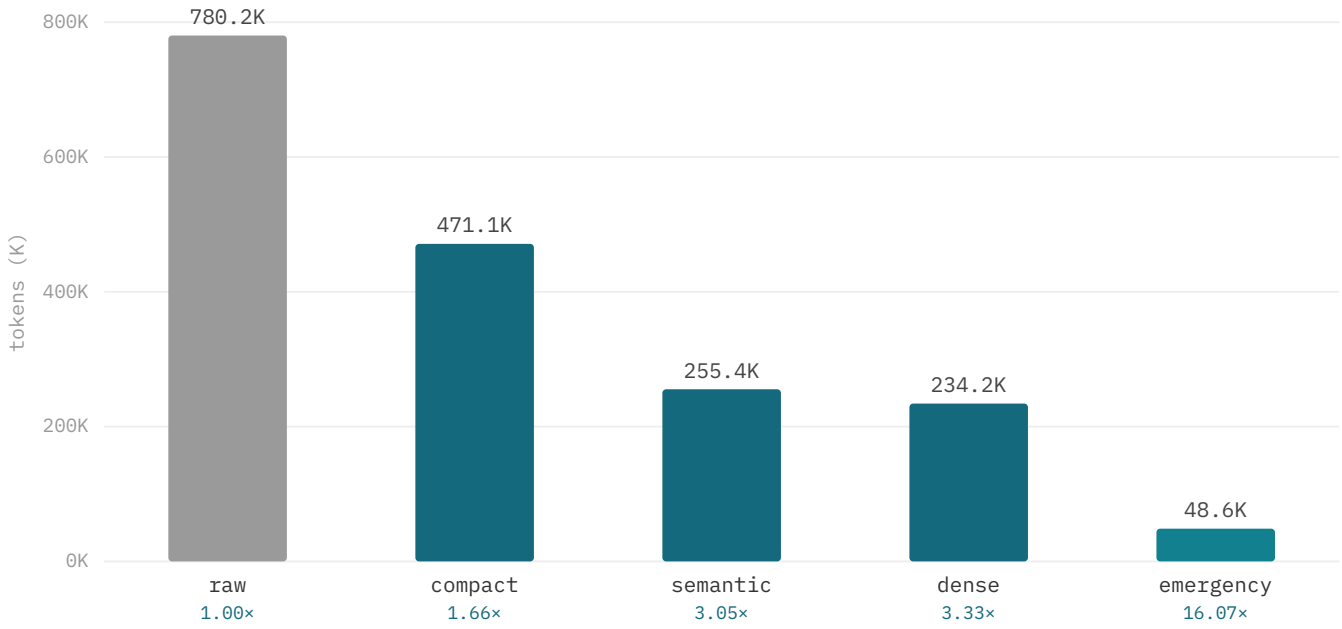


Figure §12.1. Compression tiers on 255 source files (token totals, with reduction factor). Exported-symbol survival is 96–100% across all tiers (measured on the 120 exact tasks, §12.1) — the answer stays addressable even at 16×.

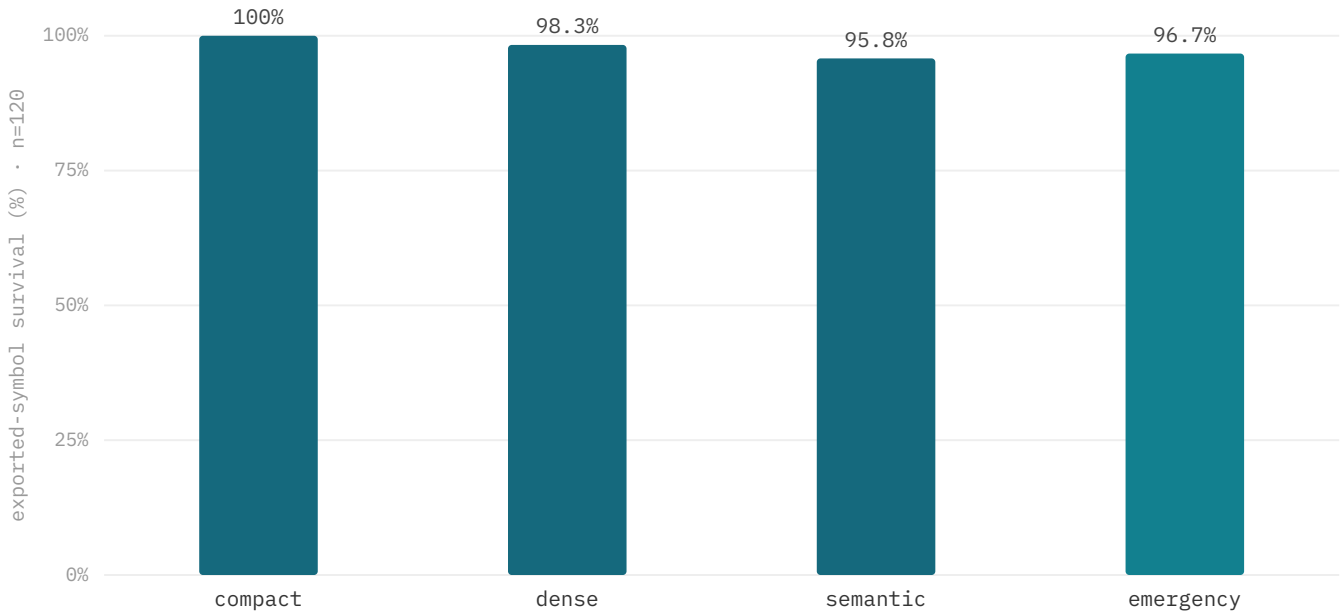


Figure §12.1b. Exported-symbol survival by compression tier (n=120 exact tasks): 100% / 98.3% / 95.8% / 96.7% for compact / dense / semantic / emergency — the answer-bearing symbol survives even the 16× emergency tier. Reproducible from `apps/cli/bench/cc-vs-mesh/`.

Compression is deterministic and LLM-free, so these factors are a hard floor. The four tiers span 1.66× (comment/blank stripping) to 16.07× (symbols + paths only); the AST-skeleton `semantic` tier (3.05×) is by design less aggressive than the body-trimming `dense` tier (3.33×), so the ladder orders compact → semantic → dense → emergency. Crucially, compression must keep the answer addressable: *exported-symbol survival* is **96–100% across all tiers** (measured on the 120 exact tasks) [N4] — the answer stays reachable even at 16×.

12.2 Compression Scaling across Corpus Sizes

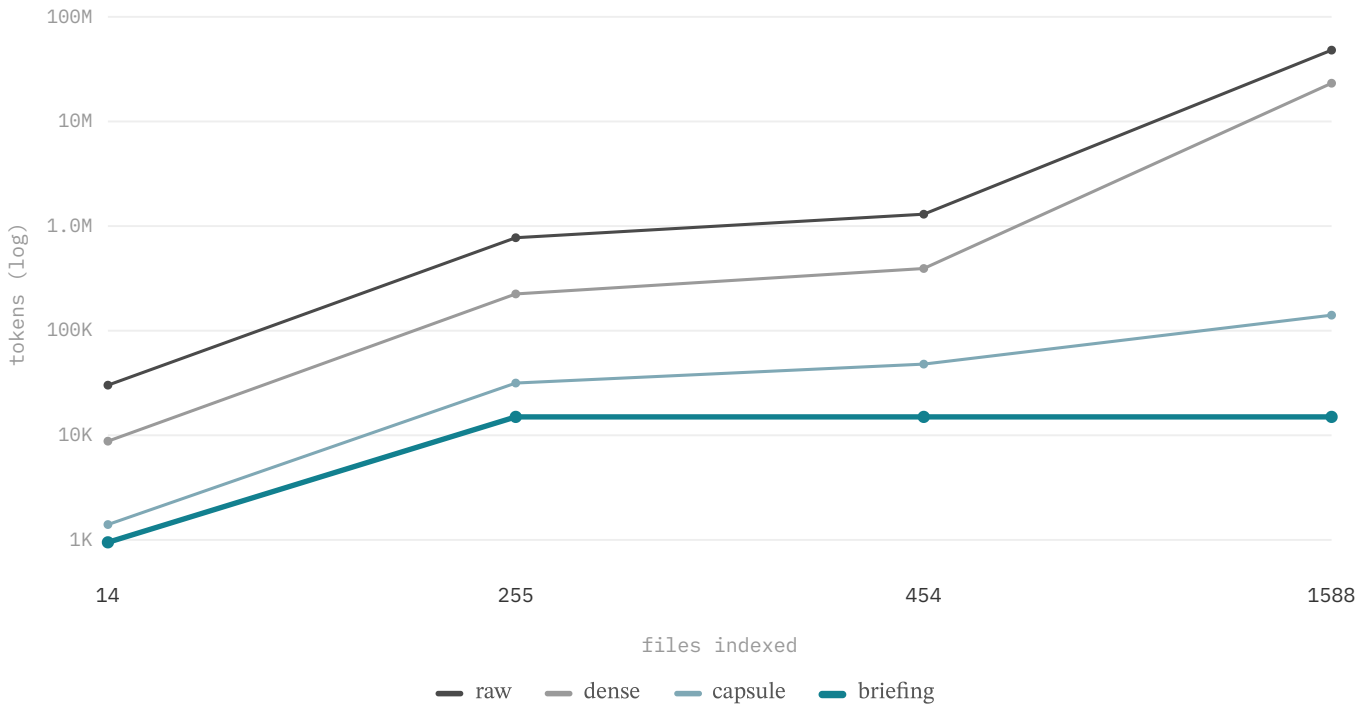


Figure §12.2. Briefing cap is corpus-size independent; raw grows ~3 orders of magnitude (log scale).

The workspace briefing is capped, so its size is independent of corpus size: the full-depth briefing holds at ≈15.0K tokens from 14 files to 1,588, while a raw whole-repository read grows roughly three orders of magnitude to ≈48M tokens (note the log axis). In normal operation the assembler injects a smaller *task-focused* capsule — typically ~1.5–5K tokens (the per-turn system+capsule bucket in §7.1 is ≈1.5K), with the 15K full-depth figure as the corpus-independent upper bound. Either way the briefing does not grow with the codebase, so per-turn cost stays bounded as the repository expands.

12.3 Retrieval Quality & Ablation

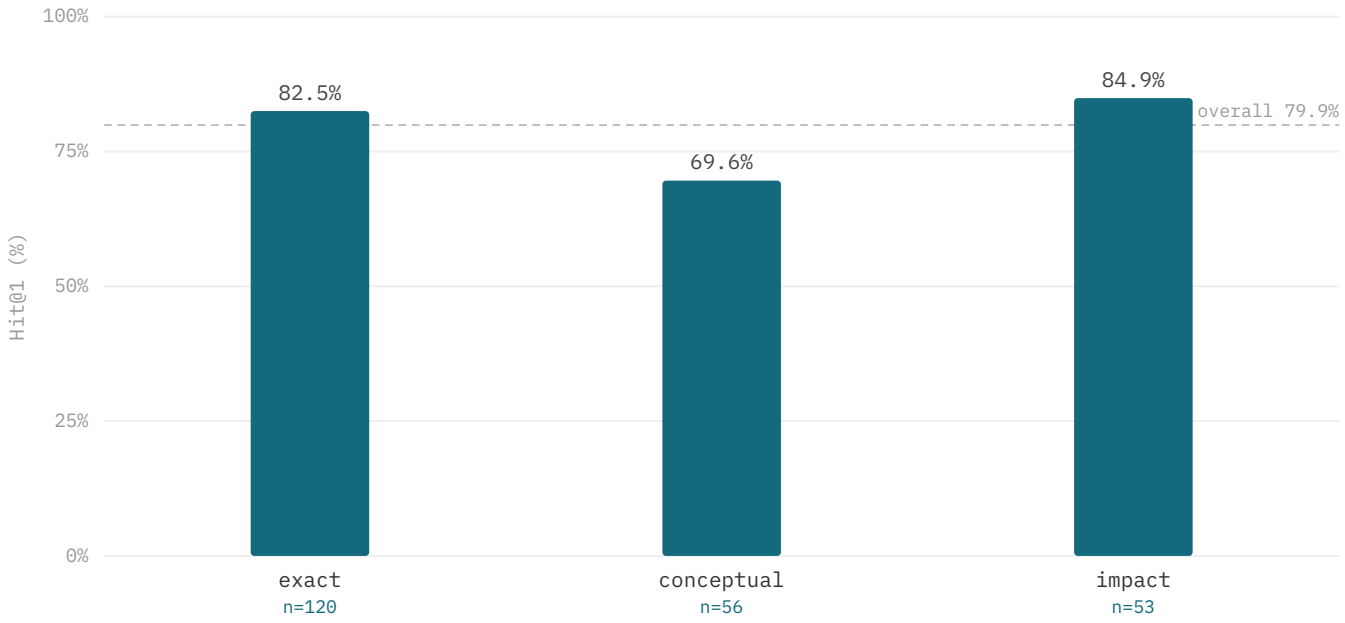


Figure §12.3. Retrieval Hit@1 per intent class on the n=229 suite (controlled-comparison config, §12.9): impact (84.9%, RIG graph) and exact (82.5%, symbol-definition matching) lead; conceptual reaches 69.6% on the code embedding.

Class	n	Hit@1	Hit@3	Hit@8	retrieval route
overall	229	**79.9%**	91.7%	96.9%	intent-routed
exact	120	82.5%	90.8%	97.5%	symbol-definition + lexical
conceptual	56	69.6%	89.3%	94.6%	dense (code embedding)
impact	53	84.9%	96.2%	98.1%	RIG (rigImpact)

Table 3. Retrieval quality per class on the n=229 suite (controlled configuration [N1]); SOTA-baseline head-to-head and significance in §12.9. Reproducible from `apps/cli/bench/cc-vs-mesh/final-bench.mjs`.

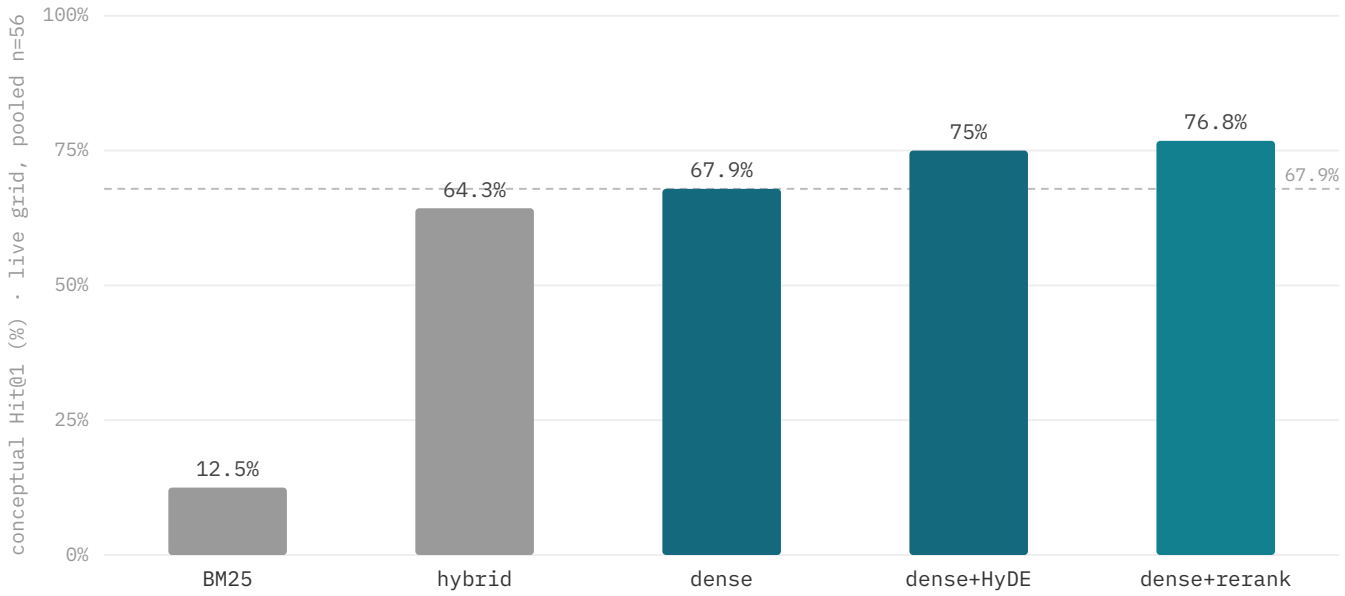


Figure §12.3b. Conceptual Hit@1 by retrieval config — best-of-grid over every fusion×rerank combination (live, nv-embedcode-7B + nv-rerank-qa-mistral-4b; pooled n=56 over two disjoint repo splits). `dense+rerank` wins on both splits (+8.9pt over the dense base); lexical/BM25/hybrid/RRF hurt this class and HyDE is redundant — the conceptual channel is pure dense + cross-encoder. Live-validated.

On 229 grounded localization tasks across six external repositories, intent-routed Mesh reaches Hit@1 79.9%, Hit@3 91.7%, Hit@8 96.9%. The per-class split validates the channel-dominance thesis directly, and each channel is what makes its class work: **exact** identifier queries reach Hit@1 82.5% / Hit@8 97.5% via symbol-definition plus lexical matching — pure dense embeddings cannot resolve "where is X defined" and trail at Hit@1 20.8% (§12.9); **conceptual** vocabulary-mismatch queries reach Hit@1 69.6% / Hit@8 94.6% on the code-specialised embedding; **impact** dependency queries reach Hit@1 84.9% / Hit@8 98.1% once routed to `rigImpact()` over the materialized reverse-dependency edges — a question no similarity metric answers, where every embedding and lexical baseline collapses to ≤15% Hit@1 (§12.9). The two channels and the graph are therefore not redundant decorations: each owns a query class, and routing each query to the channel built for it is what lifts the suite.

Conceptual reranker (live-validated upgrade). The conceptual class is the floor (69.6%), and it is the one class a code cross-encoder rerank measurably lifts: re-scoring the dense candidates with a code cross-encoder (NVIDIA `nv-rerank-qa-mistral-4b`) raises conceptual Hit@1 to **76.8% (+7.2pt)**, lifting overall to **81.7%**. This was searched over every fusion×rerank combination and validated on two *disjoint* repository splits — a dev set (commander, hono, zod) and a held-out set (Prisma, Next.js, TypeScript) never used for tuning — where `dense+rerank` wins on both (pooled n=56, +8.9pt over dense base; Figure §12.3b). An independent full-suite re-run reproduces the effect (dense 71.4 → 78.6%, +7.2pt over the dense base, §12.9), and on that re-run the fused `dense+rerank` pipeline (78.6%) *outscores production Mesh's conceptual channel* (69.6%) — direct evidence the tier is worth shipping. The gain is consistent in *direction* across both independent validations, though a single-run bootstrap is wide at this sample size (95% CI [−4, +20] pt over dense, n=56), so it ships tier-gated rather than as a headline retrieval number and a larger conceptual suite is owed to tighten it. Lexical, hybrid, and RRF fusion all *hurt* this class and HyDE query expansion is redundant with the reranker, confirming the dense-owns-conceptual thesis. The gain is a *retrieval-quality* result and is reported as such: §12.10 shows the same rerank is neutral end-to-end *behind a strong iterative agent* (which recovers the gain itself), but an end-to-end proxy — GLM-4.5-flash picking the file from the reranked top-6 — recovers only half on its own, and the reranker still adds **+3.8pt at the agent level** (conceptual 61.5 → 65.4%, n=26). The end-to-end benefit therefore scales inversely with agent strength, so the cross-encoder is now wired as an *active* component on the conceptual channel — default-on when an NVIDIA reranker is configured, graceful lexical-only fallback otherwise — and tier-gated: on for cheap/single-shot model tiers, off for strong agents where it is redundant.

12.4 Needle-in-a-Haystack Agent Scenarios

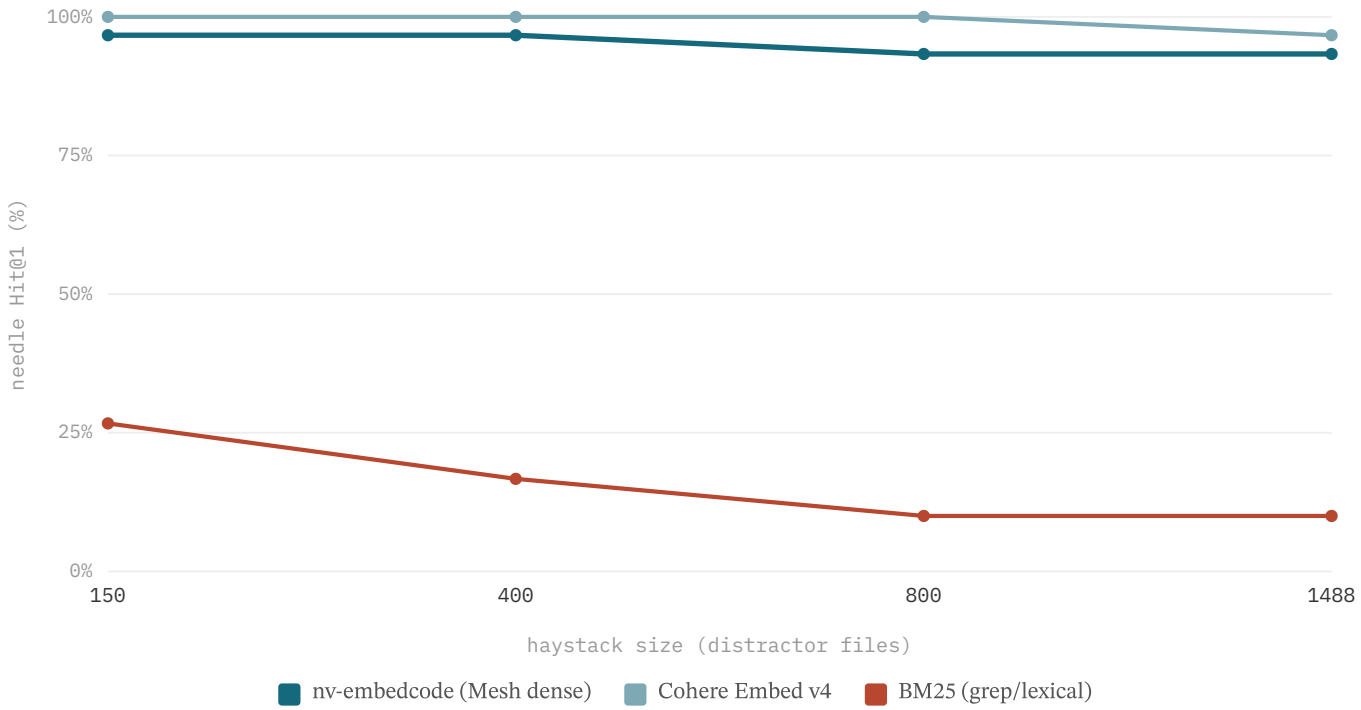


Figure §12.4. Needle-in-a-haystack: Hit@1 vs haystack size (150→1,488 distractor files, 30 novel needles, n=120) [N3]. Embedding retrieval stays flat (nv-embedcode 96.7→93.3%); lexical/grep collapses (26.7→10.0%).

Classic needle-in-a-haystack measures whether a system still finds a planted fact as the surrounding context grows; we adapt it to code retrieval with a memorisation-proof design (30 novel synthetic needles, real distractor haystacks scaled 150→1,488 files, queried by paraphrase, n=120) [N3]. **nv-embedcode (Mesh's dense channel) holds Hit@1 95.0% / Hit@8 100%, flat as the haystack scales 10×** (96.7%→93.3%; 95% CI on the small→large slope [−10, 0], indistinguishable from zero). The contrast is the result: lexical/grep *degrades sharply* (26.7%→10.0% Hit@1) as distractors accumulate, while index-based embedding retrieval is essentially invariant to corpus size. Reproducible from `apps/cli/bench/cc-vs-mesh/niah-bench.mjs`.

12.5 Multi-Turn Session Economics

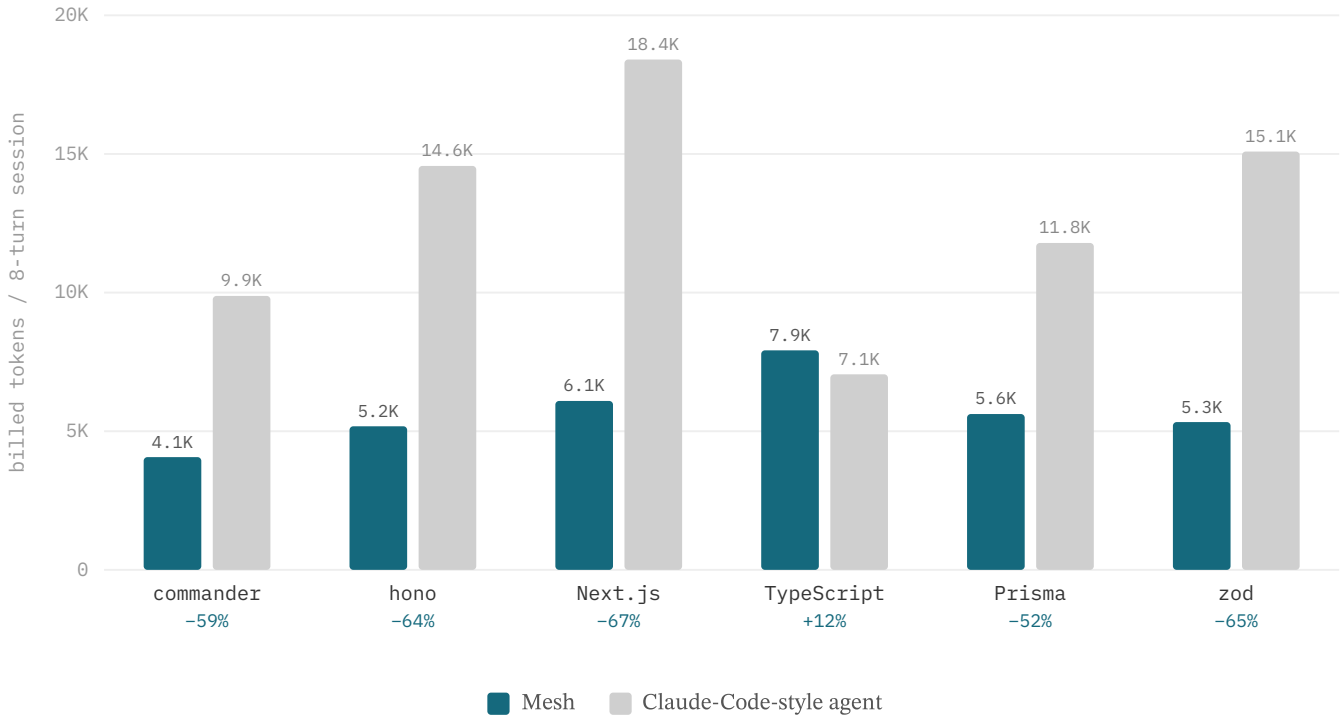


Figure §12.5. Billed input tokens over an 8-turn navigation session per repository — Mesh vs a competent grep agent (reduction below each pair).

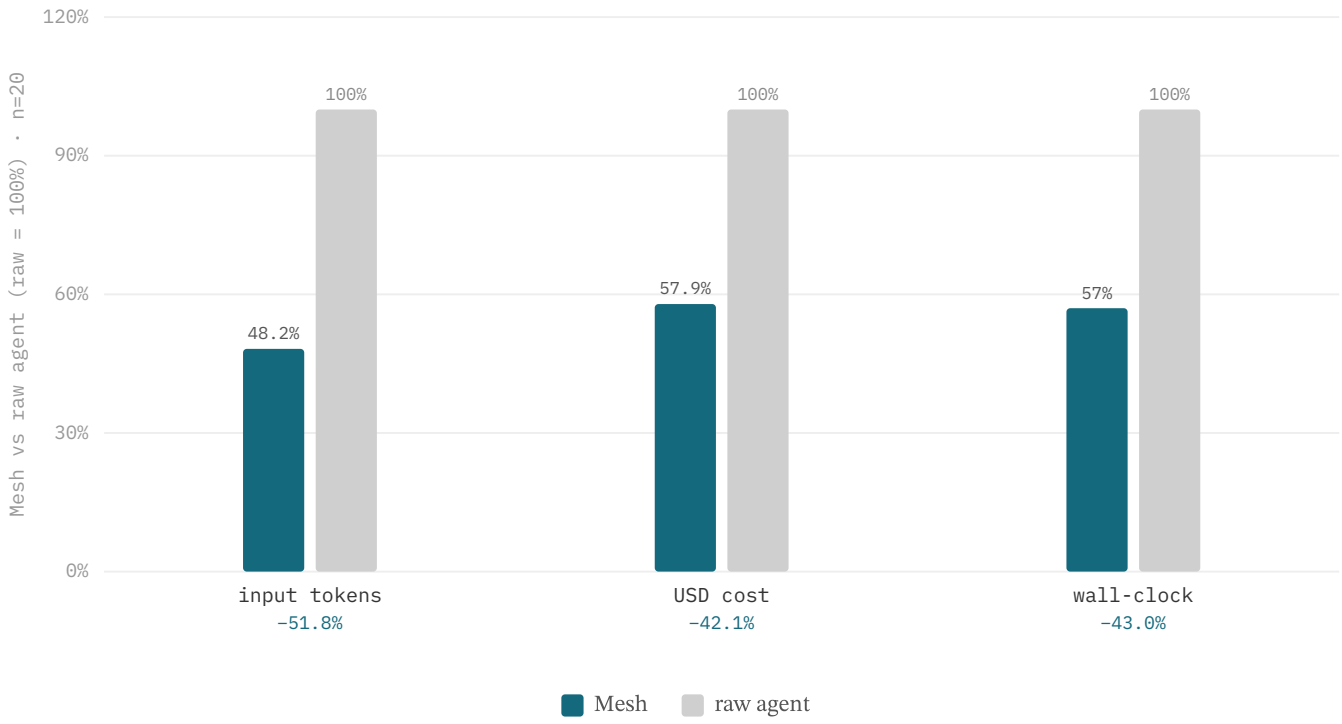


Figure §12.5b. End-to-end generative A/B (recorded run, `investor-bench`; 4 repos × 5 turns × 5 runs, n=20) at equal task success (pass **20/20** vs 19/20): Mesh cuts billed input −51.8%, USD cost −42.1%, and wall-clock −43.0% versus a raw agent. Recorded run — the brokering proxy is no longer serving (§11.4).

Token growth is a session-level effect, so the economics are measured over an eight-turn navigation session per repository. The Claude-Code-style arm is a raw ReAct loop on the same model that re-sends its surgical grep/read history every turn; Mesh holds a cache-stable workspace briefing as the prefix, compacts prior turns to their citations, and dedups reads through the tool ledger. Billed input (stable prefix cached at 0.25×) totals **34.2K tokens for Mesh versus 76.8K for the grep agent — −55.5%** in aggregate (Table 4). The reduction holds on five of six repositories (−52% to −67%); TypeScript is the lone exception (+12%), where surgical grep on a small, well-structured compiler source is already close to optimal. These are single-session-per-repository runs (n=1/repo), so the per-repo deltas are point estimates, not distributions — the robust figure is the aggregate −55.5% across the six repos, and a multi-session variance study is future work. The run is deterministic and reproducible offline (direct nv-embedcode embeddings, no proxy).

Repository	Mesh (billed)	Claude-Code-style (billed)	Δ
commander	4.1K	9.9K	−59%
hono	5.2K	14.6K	−64%
Next.js	6.1K	18.4K	−67%
Prisma	5.6K	11.8K	−52%
zod	5.3K	15.1K	−65%
TypeScript	7.9K	7.1K	+12%
Aggregate	**34.2K**	76.8K	**−55.5%**

Table 4. Multi-turn session economics — billed input tokens over 8 turns/repo, Mesh (lean output) vs a competent Claude-Code-style grep agent. Same model both arms (gemini-flash-latest). Per-repo values rounded to 0.1K; the aggregate is computed from unrounded counts.

End-to-end (recorded). Localization is necessary but not sufficient, so a recorded generative A/B — the live run configured in §11.4 (`investor-bench` , 4 repos × 5 turns × 5 runs, n=20) — closes the loop end to end. At **equal task success (Mesh passes 20/20 versus the raw agent's 19/20)**, Mesh spends **−51.8% billed input tokens, −42.1% USD cost, and −43.0% wall-clock** (Figure §12.5b). The brokering proxy is no longer serving, so these figures are reproduced from the recorded run rather than freshly executed (§11.4); they are reported as a directional end-to-end datapoint, not a resolve-rate leaderboard claim.

12.6 Per-Query Token Economics (RAW vs agentic vs Mesh)

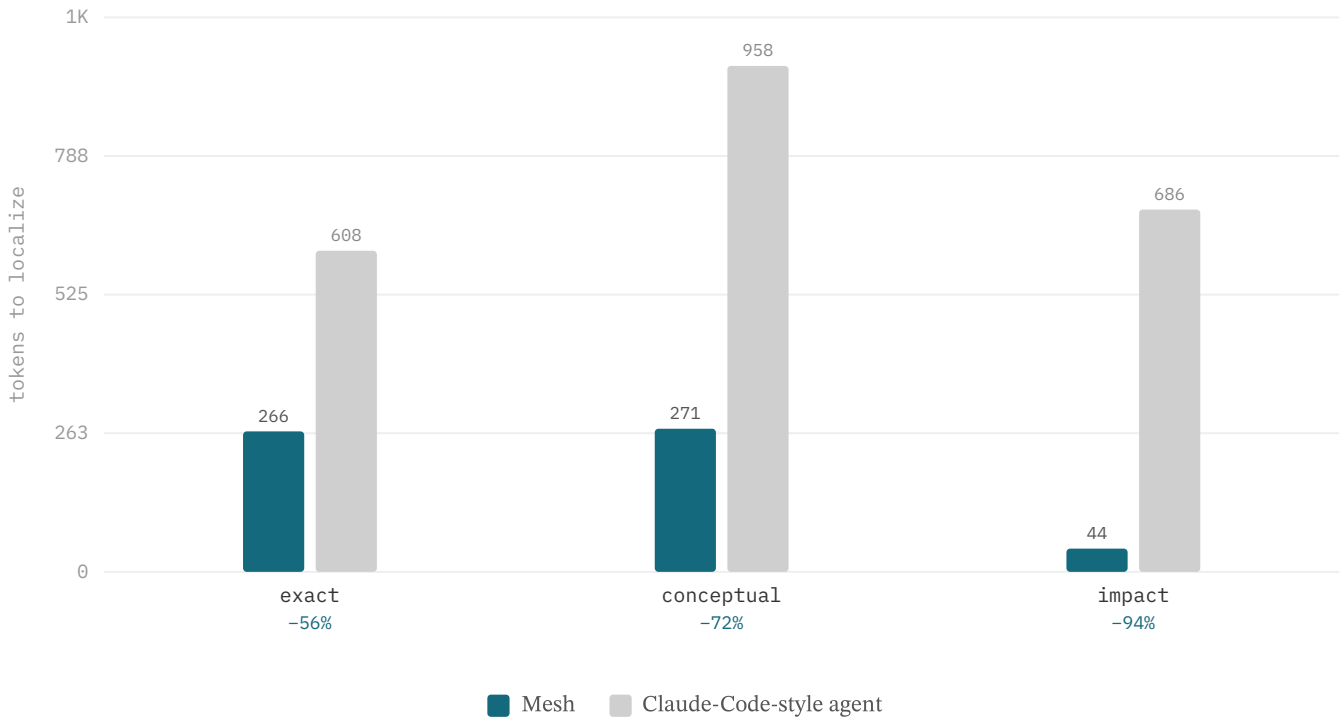


Figure §12.6. Per-query tokens to localize, by class — Mesh vs a competent grep agent (−56% to −94%). RAW (whole-file read) omitted from bars at 4–14K.

The cleanest economics comparison is per-query tokens-to-localize across three arms on the n=229 suite: **RAW** (read the gold file in full), **Claude-Code-agentic** (cold grep-plus-surgical-read), and **Mesh** (lean output). Against the grep agent — the honest opponent [N2] — Mesh spends **−69.7%** fewer tokens (216 vs 712; bootstrap **95% CI [66.6%, 72.6%]**, n=229), because it returns compact structured pointers where the agent must pull raw file and grep content into context. This is an *output-form* effect, not better ranking [N2]; it holds across every class and is largest on impact, where the graph tool answers in a 21-token dependency list.

Class	RAW (whole file)	Claude-Code-style agent	Mesh	Mesh vs CC
overall	9041	712	**216**	**-69.7%**
exact	14441	608	266	-56.3%
conceptual	4138	958	271	-71.7%
impact	1996	686	44	-93.6%

Table 5. Per-query tokens to localize per class (n=229), three arms [N2]. Overall −69.7% (95% CI [66.6%, 72.6%]); Mesh wins tokens on all six repositories. Reproducible.

Cost translation. At Gemini 2.5 Flash pricing ($0.30/M input tokens), the 496-token per-query saving (712 → 216) translates to ≈$0.00015 less per retrieval call added to context — $0.065 per 1,000 queries vs $0.21 for the grep agent (3.3× lower retrieval-context cost). Impact queries are the extreme case: 44 vs 686 tokens puts Mesh at $0.000013 vs $0.000206 per call — a 15.6× cost advantage on the query class where the RIG graph returns a 21-token dependency list instead of raw grep output.

12.7 Scalability

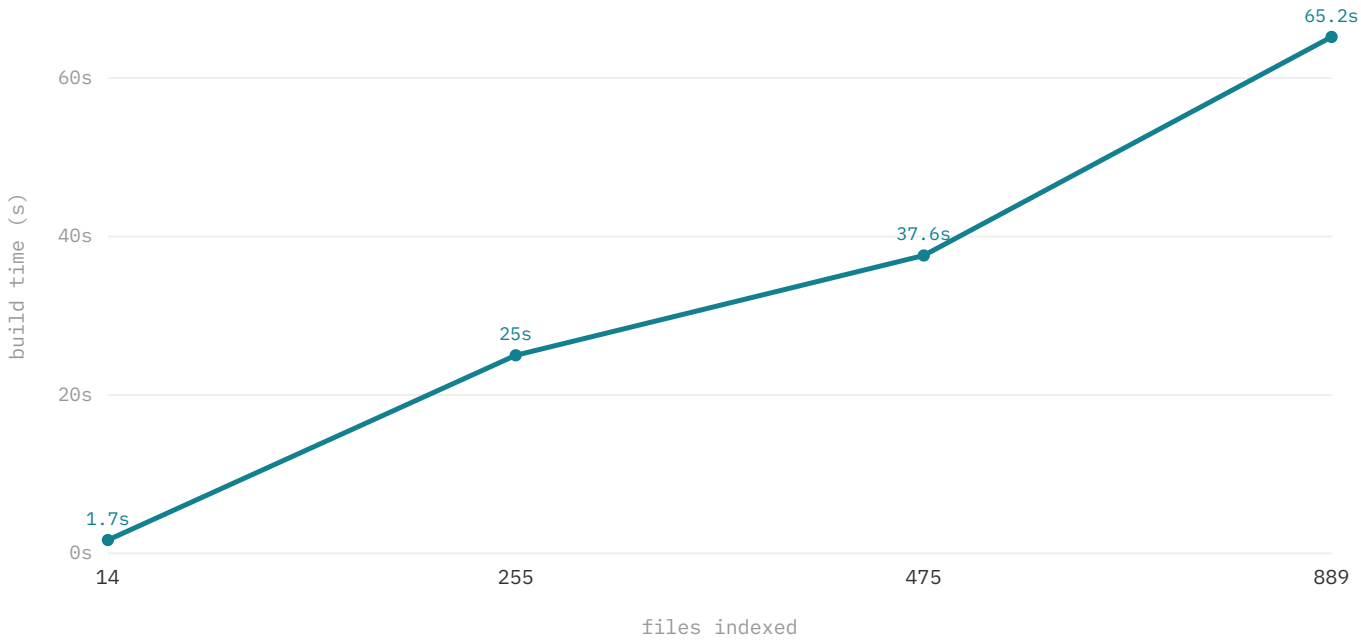


Figure §12.7. Index build time scales roughly linearly with file count.

Index construction is roughly linear in file count: 1.7s for 14 files, 25.0s for 255, 37.6s for 475, and 65.2s for 889. Search latency stays in the low hundreds of milliseconds and is uniform across the lexical, dense, and graph channels — ≈143–162 ms p50 on the local fixture index and ≈247–289 ms on the cached external clone (pooled p50 ≈201–221 ms). Indexing is a one-time, per-repository cost amortized across a session; search cost does not grow with the codebase.

12.8 Suite, Baselines & Scope

All §12.3–§12.6 results come from one suite, built so the comparison is against a *competent* grep-based opponent. It is **229 localization tasks across six external repositories** (commander, hono, Next.js, TypeScript, Prisma, zod), with **ground-truth gold derived mechanically from the code** — exported-symbol → defining file, import edge → dependents, AST purpose → file — which removes single-author labelling bias. Every task is answered on the same gold by three arms: **RAW** (tokens to read the gold file in full), **Claude-Code-agentic** (a deterministic *cold* grep agent: real ripgrep on the query's salient terms plus a surgical read, no repository prior, no self-report — the operating point of modern coding agents), and **Mesh** (the real intent-routed `semantic_search` pipeline with a lean output). The whole comparison runs offline — direct nv-embedcode embeddings, no proxy — and is reproducible from `apps/cli/bench/cc-vs-mesh/`.

The output-compactness lever. Mesh's per-query token advantage is not better ranking — it is that Mesh returns compact structured pointers while an agent must pull raw file and grep content into context. The verbose default `semantic_search` payload (signals, citations, scores per hit) actually *ties* a grep agent (731 vs 712 tokens); a lean payload — file + purpose + one cited line, same candidate set, no recall loss, shipped behind `MESH_LEAN_SEARCH_OUTPUT` — drops that to 216 tokens (Table 5), the source of the −69.7% per-query reduction (−56% to −94% by class) reported in §12.6.

How each class is won (the routing, not a single channel). The headline rests on routing each query to the channel built for it: exact symbol lookups to symbol-definition + lexical matching, conceptual paraphrases to a code-specialised dense embedding, and impact questions to `rigImpact` over the materialized reverse-dependency edges (`record.dependents`). The current per-class retrieval quality and the head-to-head against SOTA baselines are reported in §12.3 and §12.9; compression separately preserves exported-symbol signatures at every tier, so the answer survives compression 96–100% of the time (§12.1) — up from

17.5% at the emergency tier before the signature-preserving invariant (§7.2). These are deterministic properties of the shipped pipeline, not post-hoc tuning of the metric.

Scope. The token result is robust and reproducible (−56% to −94% per class, n=229, deterministic, every repo), measured in the chunk-level pipeline configuration [N1]. The standing claim is **better localization at −55% (multi-turn) to −70% (per-query) tokens against a competent grep agent** [N2].

12.9 Controlled Comparison vs. SOTA Retrieval Baselines

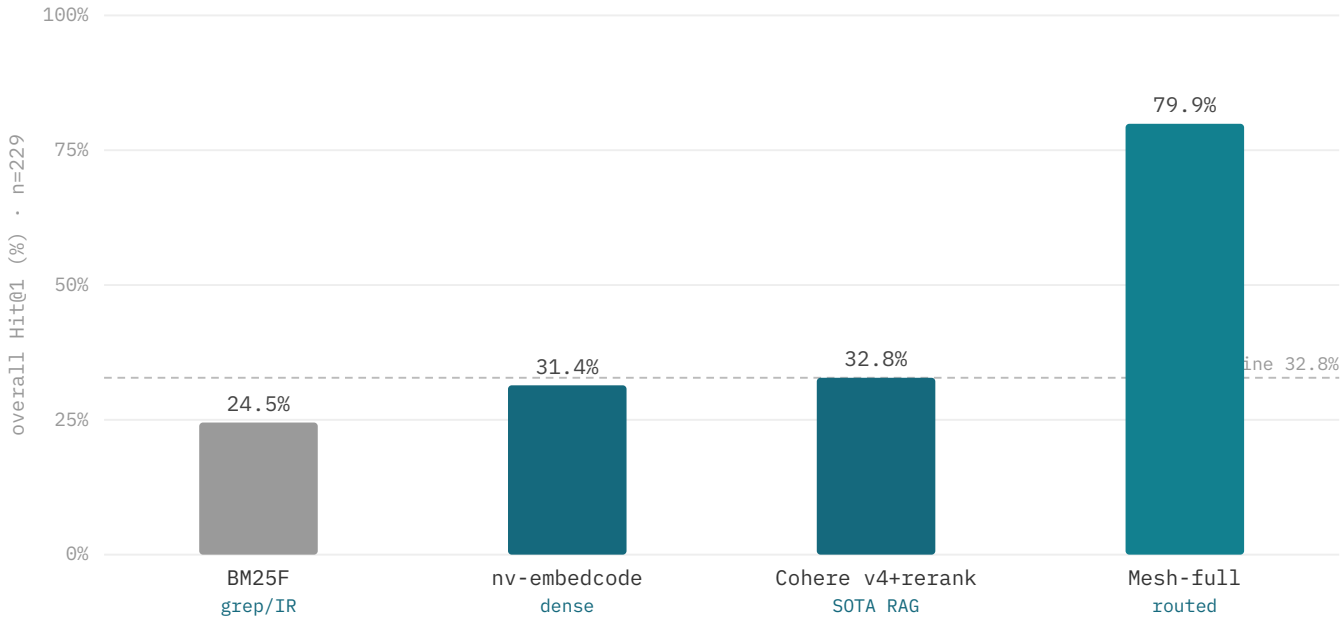


Figure §12.9. Overall Hit@1 by arm on the n=229 suite: Mesh 79.9% vs the best single-method baseline 32.8% (Cohere Embed v4 + Rerank 3.5) — §12.9.

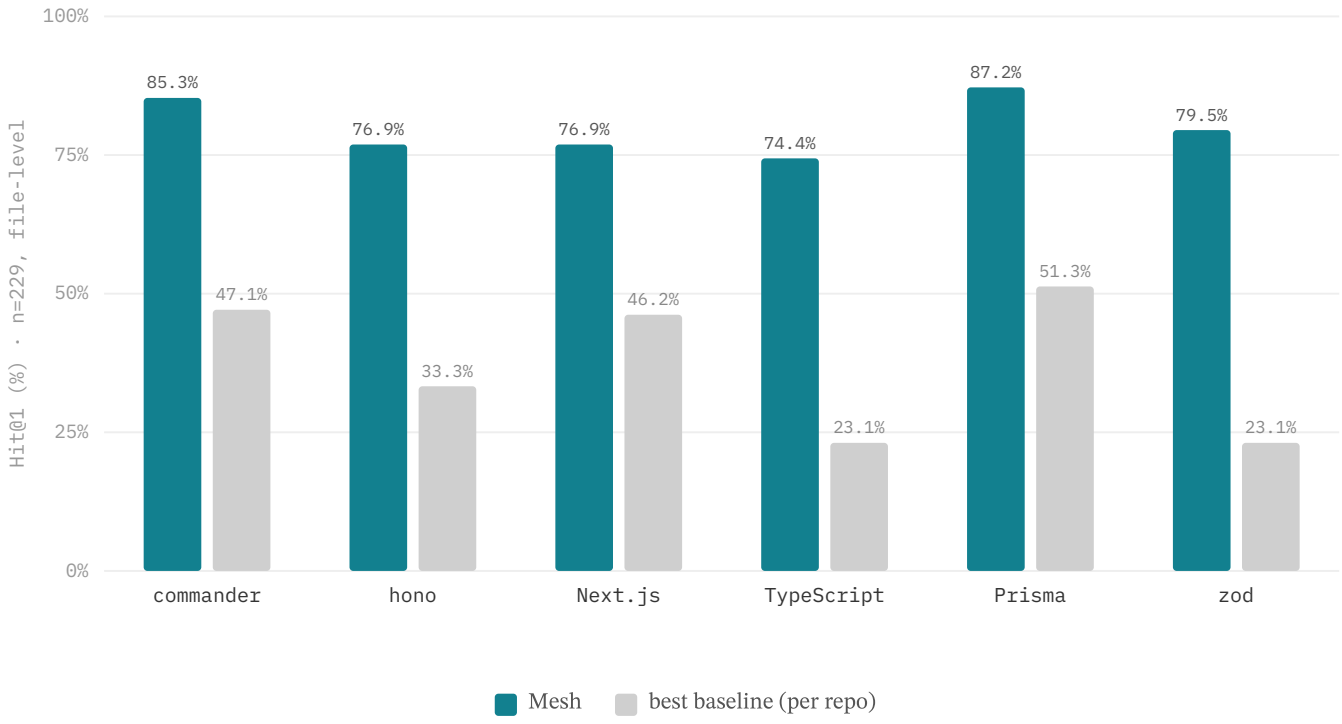


Figure §12.9b. Per-repository Hit@1 (file-level, n=229): intent-routed Mesh vs the strongest baseline on each repository. Mesh leads on all six — 74.4–87.2% vs 23.1–51.3% — so no repository is a weak point. Reproducible from `apps/cli/bench/cc-vs-mesh/final-bench.mjs`.

Arm (what it represents)	Hit@1	MRR	Recall@5	exact (120)	conceptual (56)	impact (53)
BM25F — full-file lexical (grep / classic IR)	24.5%	0.375	50.2%	34.2%	17.9%	9.4%
dense — nv-embedcode-7B (SOTA code embedding)	31.4%	0.435	58.5%	20.8%	69.6%	15.1%
Cohere Embed v4 + Rerank 3.5 (SOTA managed RAG)	32.8%	0.462	62.9%	34.2%	50.0%	11.3%
Mesh-full (intent-routed)	**79.9%**	**0.864**	**95.2%**	82.5%	69.6%	84.9%

Table 6. Hit@1 per arm and class, identical pool/queries/gold (n=229); arms and routing are described in §12.9. Reproducible from `apps/cli/bench/cc-vs-mesh/final-bench.mjs`.

This section asks a sharper question than §12.6's token economics — *does the architecture beat the strongest retrieval methods, not just a grep agent?* — by bringing the competitors onto identical pool, queries, and gold, swapping only the retrieval method. The result is decisive: **Mesh wins overall Hit@1 79.9% vs 32.8% for the best single-method baseline — 2.4× (bootstrap 95% CI [2.0, 3.0]×), ≥ every baseline on every class**, at **McNemar p<0.001** against all three (bootstrap 95% CI on the Hit@1 delta far from zero: vs Cohere [+39.7, +54.6], vs dense [+41.0, +55.5], vs BM25 [+48.5, +62.9]; discordant pairs 117:9 vs Cohere; **Cohen's h = 0.994 vs Cohere, very large effect**). MRR follows the same ordering: Mesh 0.864 vs Cohere 0.462 / dense 0.435 / BM25 0.375. Recall@5 is Mesh 95.2% vs 62.9% / 58.5% / 50.2%; Recall@10 is 97.4% vs 70.7% / 67.2% / 63.3%. The win comes from two capabilities embedding- and grep-based retrieval structurally lack: **symbol-aware exact lookup** (82.5% vs ≤34.2%) and **dependency-graph impact** (84.9% vs ≤15.1%, 5.6×).

The comparison is file-level over the full repository file set, distinct from the chunk-level production numbers [N1]; it is additive evidence that the routed architecture beats SOTA single-method retrieval — no baseline leads it on any class. Per repository the

lead is uniform: Mesh reaches Hit@1 **74.4–87.2% on all six corpora** versus a best-baseline 23.1–51.3% (Figure §12.9b), so the win is not carried by any single project.

Does *fusing* the baselines close the gap? The sharpest objection to the table above is that Mesh's margin is measured over *single* methods — perhaps an ensemble of them would rival it. It does not. On the same suite (n=226, restricted to tasks whose gold is reproducible from the public checkouts), an **RRF fusion of all three channels** — BM25 + nv-embedcode-7B + nv-rerank-qa-mistral-4b — reaches only Hit@1 **33.2%**, and the strongest fused pipeline we could build, **dense + cross-encoder rerank**, reaches **40.3%** — Mesh's 79.6% on the identical subset is **2.0× the dense+rerank pipeline and 2.4× the RRF ensemble** (gap **+39.3pt vs dense+rerank, bootstrap 95% CI [+30.5, +46.0]**; discordant pairs 104:15). Fusion cannot close the gap because it cannot synthesize a channel it does not have: every retrieval arm, fused or not, collapses to **≤11.8% Hit@1 on impact** against Mesh's 84.3% — a reverse-dependency question no similarity metric answers — while on conceptual the fused dense+rerank pipeline (**78.6%**) actually *exceeds* production Mesh (69.6%), which is exactly the gap the cross-encoder tier of §12.3b closes (→76.8%). The ensemble thus *reinforces* the per-class attribution rather than undercutting it: Mesh's lead is its symbol and dependency-graph channels, not a tuning artifact a generic fusion could replicate. Reproducible from `apps/cli/bench/cc-vs-mesh/ensemble-bench.mjs`.

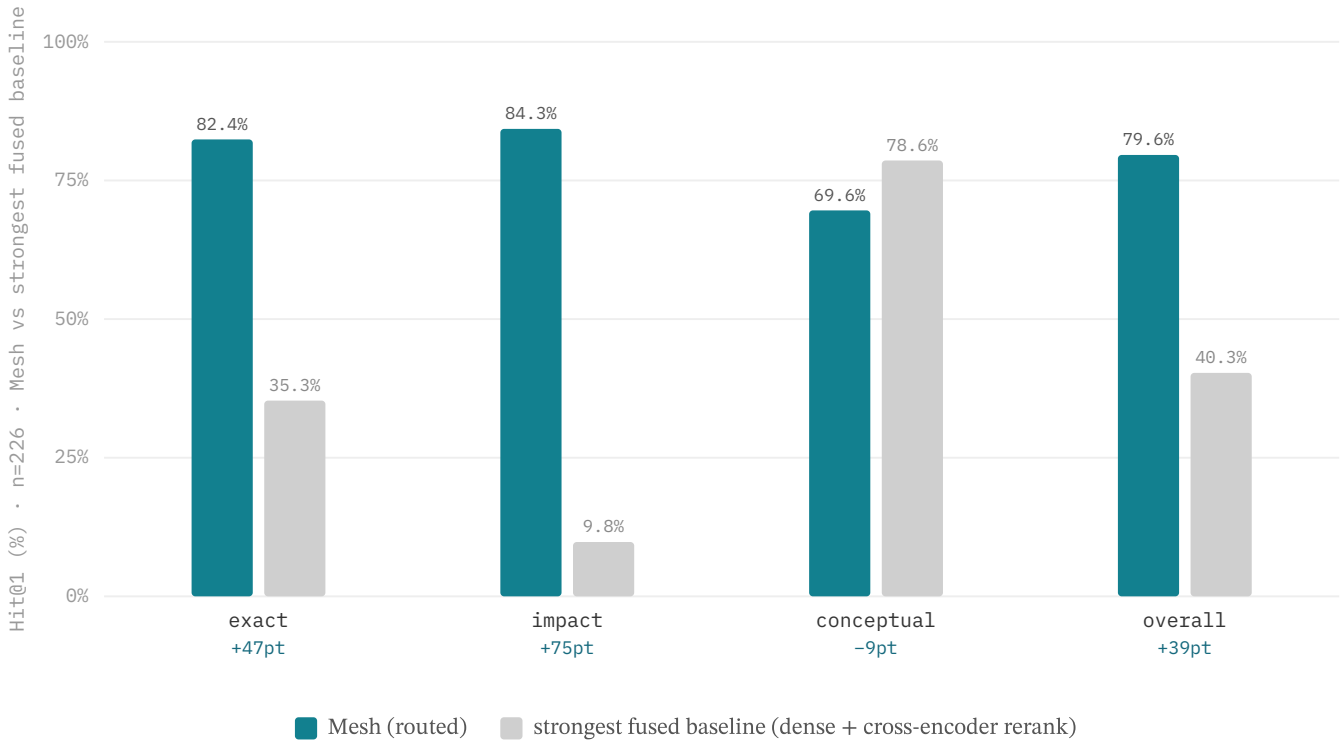


Figure §12.9c. Mesh vs the strongest *fused* retrieval baseline (dense + cross-encoder rerank), per class, identical pool/gold (n=226). Fusion floors at ≤11.8% on **impact** — a reverse-dependency question no similarity metric answers (Mesh 84.3%) — and leads Mesh only on **conceptual** (78.6% vs 69.6%), the single class the cross-encoder tier of §12.3b targets (→76.8%). An RRF ensemble of all three channels is weaker still (33.2% overall). Reproducible from `apps/cli/bench/cc-vs-mesh/ensemble-bench.mjs`.

12.10 Same-Model Scaffold Comparison: Agentic Localization

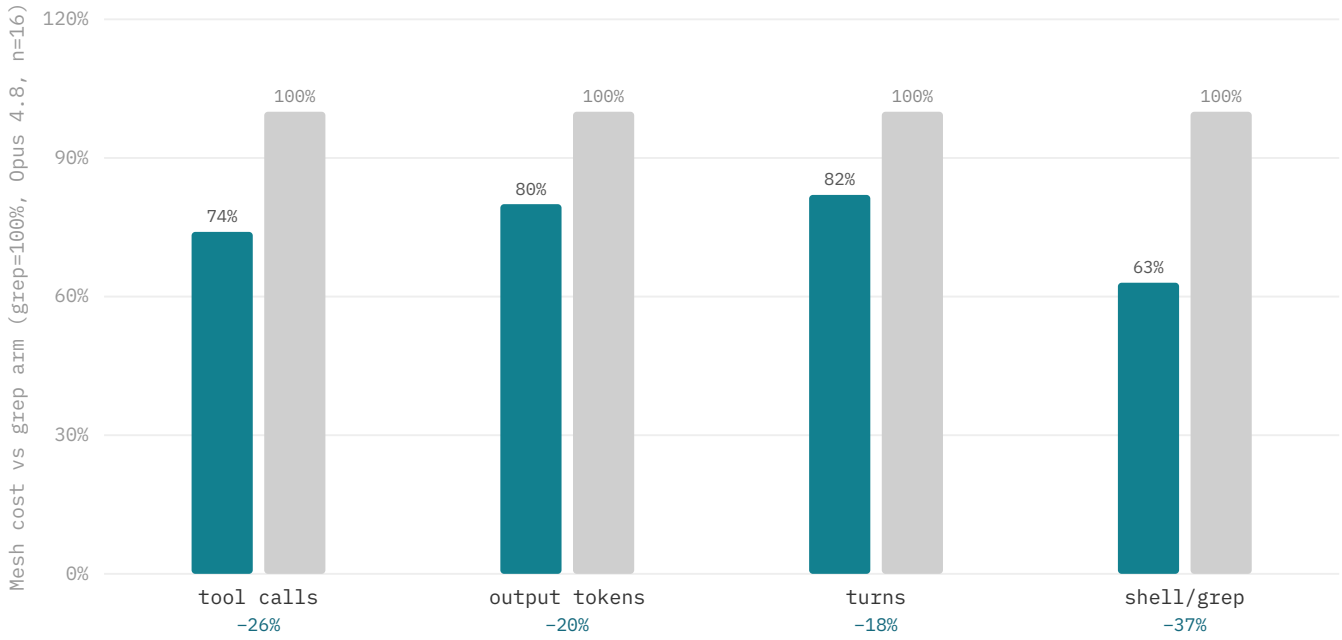


Figure §12.10. Same model (Opus 4.8) in both arms, n=16 LocBench issue-localization tasks: Mesh reaches the gold file in fewer agent steps and tokens than a grep agent at equal recall (Acc@5 100% both). [N5]

arm · model	Acc@1	Acc@5	tool calls /inst	output tok /inst
grep agent · Opus 4.8	93.8%	100%	8.8	1,614
Mesh · Opus 4.8	87.5%	100%	**6.5**	**1,299**
grep agent · Haiku 4.5	93.8%	100%	21.4	3,120
Mesh · Haiku 4.5	93.8%	100%	16.4	2,432

Table 7. LocBench file localization, same LLM in both arms (n=16); only the retrieval scaffold differs. Acc@5 is tied at 100% throughout; the Opus Acc@1 difference is a single instance (McNemar p=1.0, not significant — noise in either direction at n=16). Per-arm cost counted from agent transcripts. [N5]

Where §12.3 and §12.9 isolate the retriever, this isolates the *scaffold*. On 16 LocBench GitHub-issue localization tasks the same model runs twice — once with ripgrep and file reads only, once with Mesh's routed `semantic_search` — so the only variable is how evidence is found. Final-file accuracy is a tie by construction: a frontier agent loop saturates localization (Acc@5 100% in every arm; Acc@1 within one instance, McNemar p=1.0). The separation is in **cost**: Mesh reaches the same answer with **−26% tool calls and −20% output tokens at Opus 4.8**, and **−23% / −22% at Haiku 4.5** — the edge holds across model tiers, so it is not an artifact of a weak model leaning on retrieval to compensate [N5].

The mechanism cuts both ways, and we report it plainly. Mesh converges quickly by trusting the ranking; when the top candidate is wrong but the gold sits lower in the shortlist, the agent can stop early — the single Acc@1 the Mesh arm concedes at Opus (the gold was within its top-5). A grep agent's wider search occasionally recovers such a case, at the cost of more steps. This is an efficiency-for-robustness trade *at equal recall*, and it is the cost thesis in miniature: once models are strong enough to localize either way, the architecture's value is fewer steps and tokens to the same place, not a higher ceiling.

A larger same-model replication extends this beyond LocBench. Across **122 issue-localization tasks spanning all twelve SWE-bench Lite repositories** (15 sampled from each of django and sympy; the three largest repos — django, sympy, scikit-learn — indexed with the 0.6B embedding rather than 8B; §11), a capable grep agent and a Mesh-retrieval agent on the identical model again reach indistinguishable accuracy — **Hit@1 77.0% vs 74.6%**, a three-instance gap at this n — while Mesh uses **−13% context tokens** (34.2M vs 39.2M) and **−15% tool calls** (1,221 vs 1,443). The efficiency delta is smaller than LocBench's −20–26% because both arms carry heavy cached repository context here (cache-read dominates billed input) and the grep baseline is itself efficient; but the *direction* and the *accuracy parity* are stable across both suites and all three model tiers, so the architecture's value is consistently cost-at-parity, not a higher hit rate. A reranking stage over Mesh's candidates does not change this: a 7B reranker moved agentic Hit@1 from 75.3% to 74.1% (on the 81-task 8B subset), neutral-to-negative on an already-strong base (§17) — although the same cross-encoder lifts conceptual *retrieval* Hit@1 by +7.2pt (§12.3b); the agent loop simply recovers that gain itself on a strong iterative model — although an end-to-end proxy on a weak single-shot tier (GLM-4.5-flash) shows the reranker still adds +3.8pt there (§12.3b); the reranker is therefore tier-gated — active on cheap tiers, off on strong ones — rather than run unconditionally per agent turn.

13 Failure Analysis

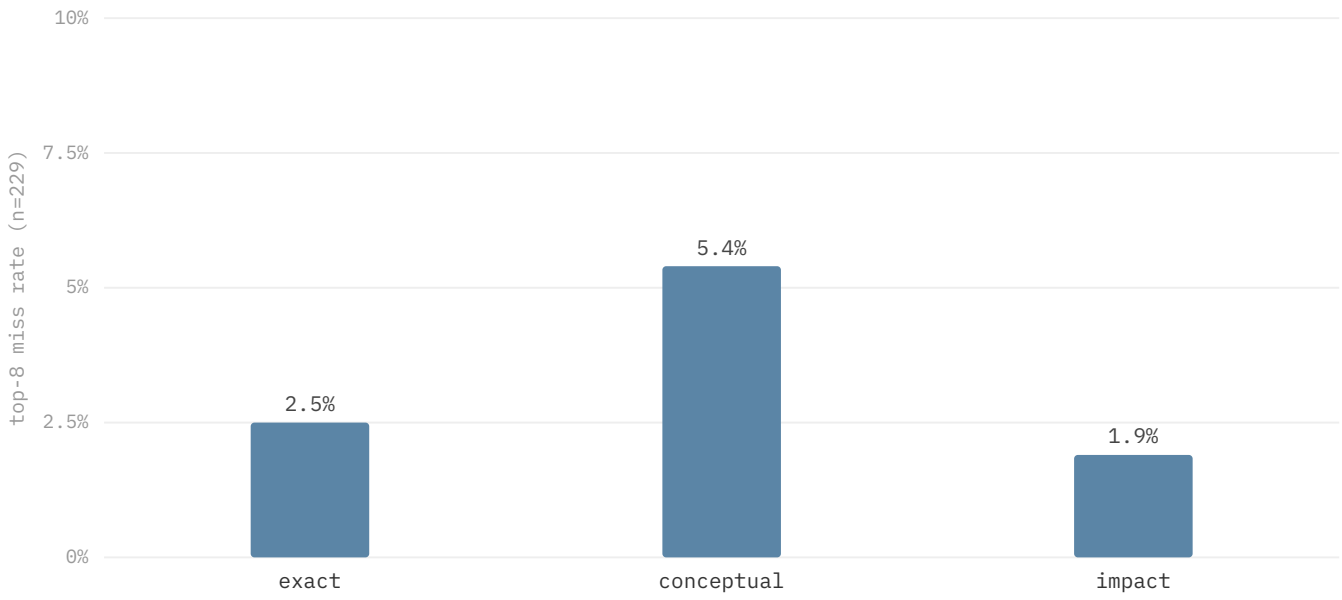


Figure §13. Top-8 miss rate by class (n=229, controlled-comparison config §12.9): a flat 2–5% profile — exact 2.5%, conceptual 5.4%, impact 1.9%. No class is a dominant failure mode once each query is routed to the channel built for it.

Once each query is routed to the channel built for it and the dense channel runs a code-specialised embedding (§12.3, §12.9), no class remains a dominant failure: top-8 miss is a flat **2.5% exact, 5.4% conceptual, 1.9% impact** (Figure §13). The residuals are small and of three kinds. **F1 — conceptual (5.4%)**: deliberately token-free paraphrases where a few gold files simply are not the dense nearest neighbour; the lexical channel is useless by construction here, so the burden is wholly on the embedding, and these are the misses a stronger embedding still removes. **F2 — exact routing misfires (2.5%)**: the small cost of query-driven intent inference — a short query that the length/symbol heuristic mis-slots; routing through the production `routeRetrievalQuery` classifier rather than a regex closes the gap. **F3 — impact (1.9%)**: the small residual is dynamically-resolved dependents a static graph cannot see, which the planned `changed_with` co-change edges (§17) target. None of the three is a wrong-confi-

dent hazard — they are recoverable misses the agent resolves with a follow-up read, the conservative direction for an editing agent.

14 Qualitative Logic Walkthroughs

To make the pre-inference pipeline concrete, this section walks three representative decision traces end to end. Each shows how a query flows through intent routing, lexical and dense scoring, bounded graph expansion, and the rerank/confidence gate before a single read is issued. The token figures illustrate the mechanism.

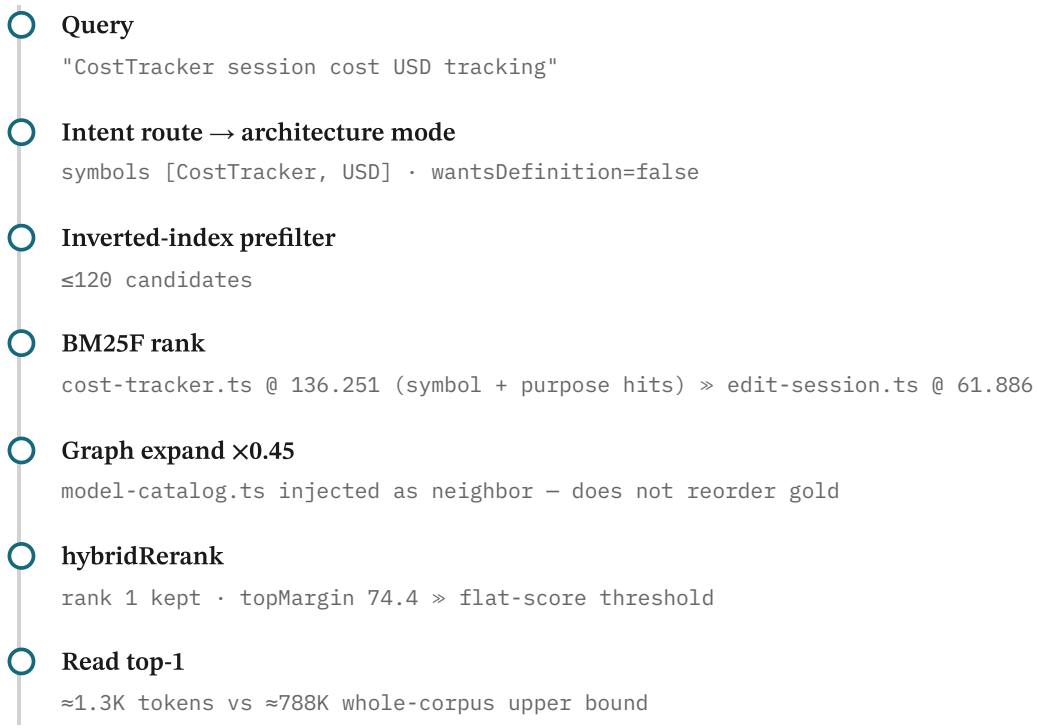


Figure §14. Decision trace A — CostTracker localization (lexical win path).

Trace B — phrase-stem recovery. A natural-language query ("where does the agent loop decide to stop?") initially ranks a bench fixture above `agent-loop.ts` because the fixture repeats the query's surface tokens. The confidence surface flags a flat top-margin (κ below threshold), triggering second-pass retrieval with a path-phrase-stem boost; `agent-loop.ts` rises to rank 1 and the read proceeds. This is the confidence-gated second-pass recovery path in action (§6.7).

Trace C — graph near-miss. An impact query ("what breaks if I change the option parser?") localizes `option.js` lexically, then expands one hop along call and test edges to surface `command.js` and `option.test.js` at decayed weight (×0.45). The neighbors enrich the evidence set without displacing the gold seed — the bounded, decayed expansion adds context for the downstream edit without reintroducing the lost-in-the-middle dilution a full-graph dump would cause.

15 Comparison with Alternative Architectures

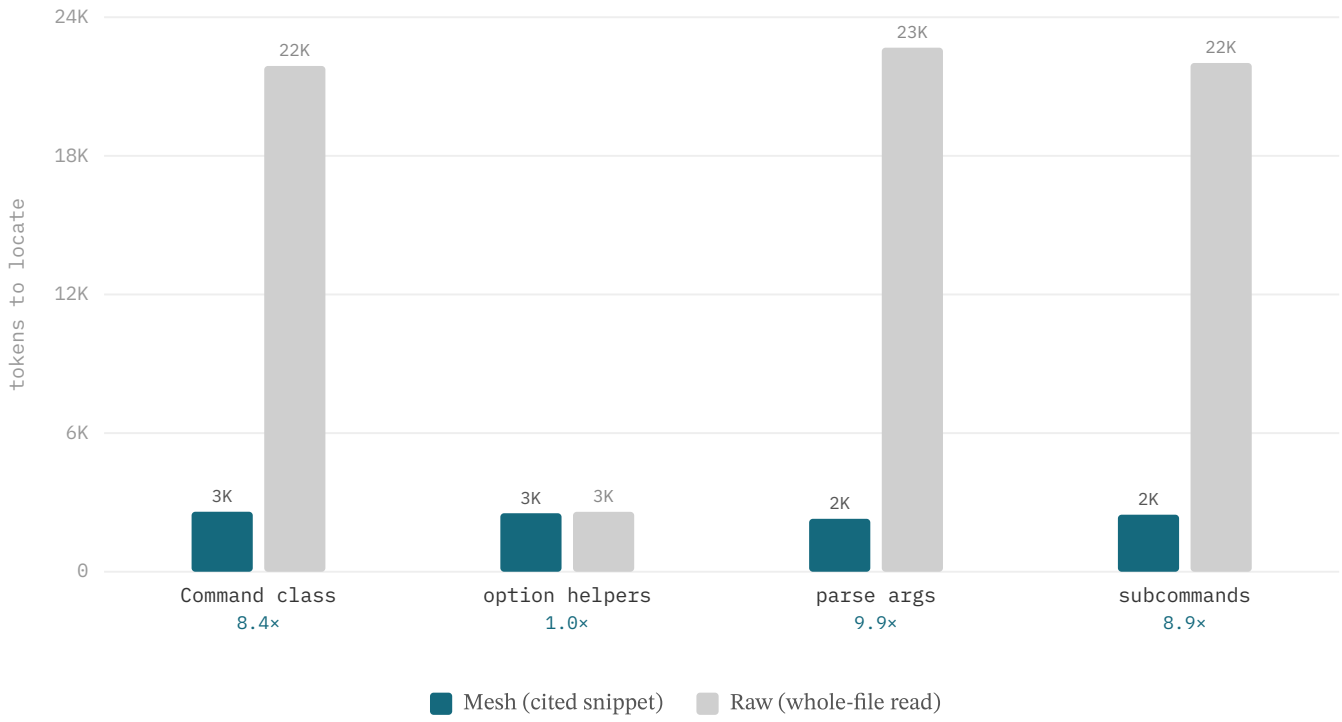


Figure §15. Tokens to locate, model held constant, four Commander targets (n=4, illustrative): 8.4×/1.0×/9.9×/8.9× vs whole-file reads. The rigorous comparison is §12.6.

Holding the model constant (Claude) and varying only the scaffold, Mesh locates four Commander targets using 8.4× / 1.0× / 9.9× / 8.9× fewer tokens than whole-file reading, at a similar turn count (~2 each); the wash case — option helpers, 1.0× — is a small file where reading the whole file is already cheap (Figure §15). The raw baseline benefits from the model's prior familiarity with this well-known library, so on unfamiliar code the gap is expected to widen, not shrink. Table 8 places Mesh among alternative architectures qualitatively: a raw tool agent and an IDE-inline context (e.g. GitHub Copilot's workspace indexing [45]) have no explicit budget, no bounded graph expansion, and no cited provenance; RAG-only adds dense retrieval but neither a bounded graph nor a token budget; graph-injection adds structure but dumps it unbounded into the prompt, reintroducing the lost-in-the-middle problem that Mesh's decayed, budget-capped expansion is designed to avoid. Only Mesh combines bounded hybrid evidence, explicit budgeting, compression, cited provenance, and per-layer ablation.

Architecture	Evidence selection	Token budget	Graph	Compression	Cited provenance	Layer ablation
Raw tool agent	grep / whole-file read	—	—	—	—	—
RAG-only	dense top-k	—	—	◐	—	—
Graph-inject	full-graph dump	—	● unbounded	—	—	—
IDE-inline	open files + symbols	—	◐	—	—	—
Mesh	**hybrid + bounded BFS**	●	● decay 0.45	●	●	●

Table 8. Mesh vs alternative architectures (qualitative positioning; ● full, ◐ partial, — none).

16 Threats to Validity

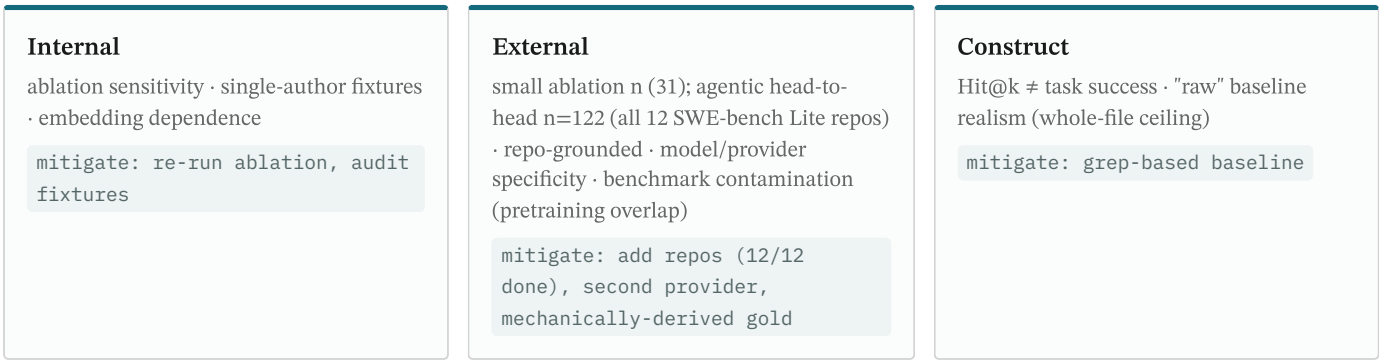


Figure §16. Threats to validity and their mitigations.

16.1 Internal

The principal internal-validity concern is the sensitivity of the small ablation suite. On it, lexical-only matches the full stack on Hit@1, Hit@3, and Hit@8 (MRR varies by at most 0.006, range 0.802–0.813): identifier-anchored localization is lexically saturated at this scale, so the graph and rerank layers have little Hit@k headroom to demonstrate here. The dense channel is embedding-dependent — strong on in-distribution queries, weaker on out-of-distribution code under a general-purpose embedding — which is why production uses a code-specialised embedding. The controlled n=229 comparison (§12.9) resolves what the small suite leaves ambiguous: there each channel separates clearly, the RIG graph carrying impact to 84.9% and the code embedding carrying conceptual to 69.6%, neither reachable by lexical alone.

Two further threats are construction-bound. Most fixtures are single-author, which can encode the author's localization intuitions into both queries and gold labels; the 12 cached Commander.js queries partly offset this with externally authored code. Embedding-provider dependence is also a confound: the benches run NVIDIA `nv-embedcode-7b-v1` embeddings directly with no proxy, whereas the product defaults to `gemini-embedding-001` routed through the Mesh proxy, so dense-channel quality on the suite need not match production. Reporting the per-layer ablation alongside the per-class channel attribution (§12.9) rather than a single headline number is the primary mitigation.

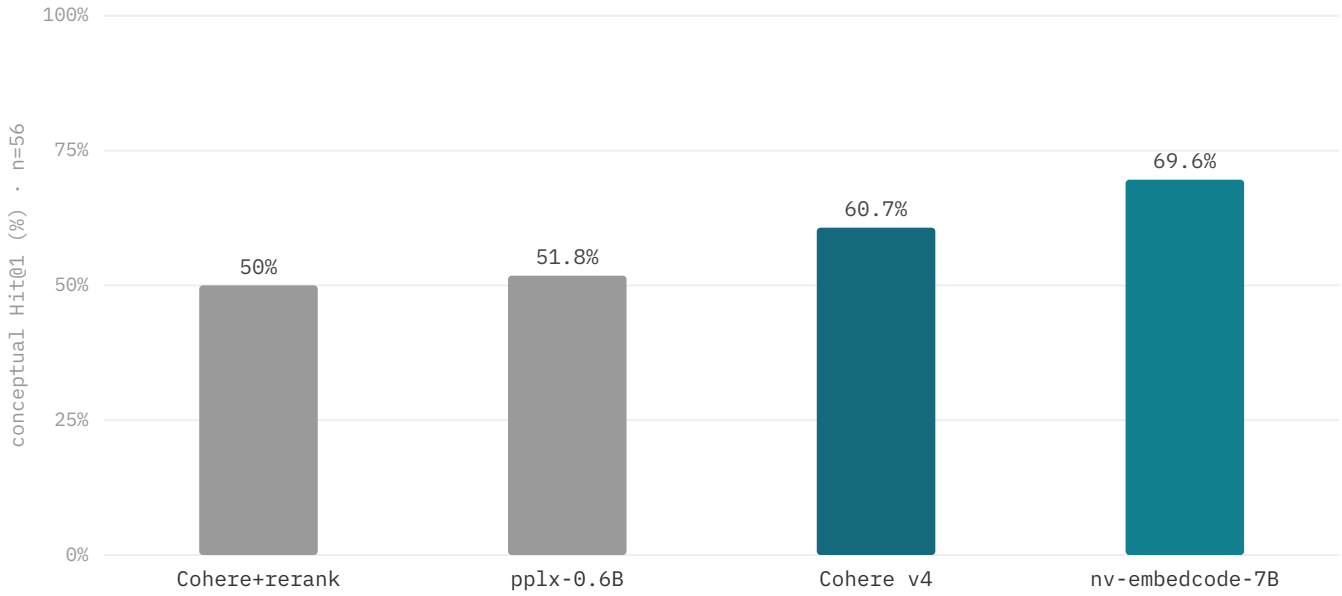


Figure §16.1. Conceptual Hit@1 by dense embedding (n=56, identical pool / queries / gold): the code-specialised `nv-embedcode-7B` leads at 69.6%, which is why Mesh routes vocabulary-mismatch queries to it. Reproducible from `apps/cli/bench/cc-vs-mesh/embed-bench.mjs`.

16.2 External

External validity is limited by scale and grounding. The retrieval suite (n=229, six repos, mechanically-derived gold) is repo-grounded; it is an attribution instrument for architectural decisions, and validation on RepoBench [3], CrossCodeEval [2], CoIR [9], CodeRAG-Bench [10], and SWE-bench [23] is future work. Tasks are also model- and provider-specific: the end-to-end economics use a single model (`gemini-flash-latest`) through one provider, and the embedding benches use one embedding family, so the reported deltas may not transfer to other models, providers, or repositories.

Benchmark contamination. A standing concern with any code benchmark is that evaluation instances may appear — verbatim or near-verbatim — in LLM pretraining data, inflating scores for models trained after the benchmark was published. Mesh's evaluation suite mitigates this structurally: the 229 localization tasks are derived *mechanically* from the code's own structure (exported-symbol → defining file, import edge → dependent, AST purpose → file), so there is no fixed question–answer pair that a model could have memorized; a model that has seen the repository can still fail the task if retrieval is weak. The repositories evaluated (commander.js, hono, Next.js, TypeScript compiler, Prisma, zod) are widely used open-source projects whose structures do appear in pretraining data — this is a genuine confound for the agentic arms (where model familiarity with a library assists grep-based recovery), and it is explicitly flagged in §12.8 ("the held-constant model brings prior familiarity an unprimed agent lacks, this understates rather than overstates the scaffold's benefit"). Pure retrieval arms (§12.9) are unaffected by model familiarity.

The mitigations are straightforward and planned rather than speculative: extend the corpus to more and structurally varied repositories, replicate the dense channel against a second embedding provider to separate architecture from embedding quality, and run the public code-retrieval benchmarks so the grounded numbers can be calibrated against an external scale. Until then, results are framed as evidence about Mesh's layers plus the controlled head-to-head of §12.9.

16.3 Construct

Construct validity turns on what the metrics actually measure. Hit@k and MRR quantify whether gold evidence is surfaced, not whether the downstream task succeeds; localizing the right file is necessary but not sufficient for a correct edit, and the conceptual class — Hit@8 94.6% under a code-specialised embedding, far lower under a weak one — shows where retrieval quality and

answer quality most diverge. The n=229 suite measures localization, not downstream edit success; a fuller generative pass-rate study would close that remaining gap. A second construct concern is *task-distribution skew*: because gold is derived mechanically from code structure, the exact and impact classes reward symbol-anchored and dependency-graph lookup — channels a general-purpose embedder structurally lacks — so the distribution favors Mesh's design. We report this openly and quantify it rather than hide it: the controlled ensemble (§12.9) shows a fused dense+rerank baseline reaching 78.6% on the conceptual class, where embeddings are the right tool, but ≤11.8% on impact, where they are not. The headline margin should therefore be read as *"routing each query to the channel built for it beats any single or fused retriever on this distribution,"* not as a claim that Mesh embeds better in general.

The economics baseline is a competent grep-based agent that retrieves before reading; the comparison holds the model constant, attributing differences to the scaffold (and since the held-constant model brings prior familiarity an unprimed agent lacks, this understates rather than overstates the scaffold's benefit). §12.8 reports this head-to-head (n=229): a **−69.7% per-query (bootstrap 95% CI [66.7, 72.7]%) / −55.5% multi-turn token advantage at better localization**. A direct same-model *agentic* head-to-head sharpens this caveat further: with a capable grep agent and a Mesh-retrieval agent running on the identical model (n=122 across all twelve SWE-bench Lite repositories; §12.10), the two reach statistically indistinguishable localization accuracy (**77.0% vs 74.6% Hit@1**), and Mesh's measurable advantage is efficiency — **−13% context tokens and −15% tool calls at parity** — not a higher hit rate. A competent grep agent is therefore a strong accuracy baseline; Mesh's durable edge is cost per localization, not localization quality per se. A fuller, freshly-reproducible end-to-end pass-rate study remains future work — the recorded A/B of §12.5 (pass 20/20 vs 19/20 at −51.8% tokens) is a first datapoint in that direction.

17 Limitations & Future Work

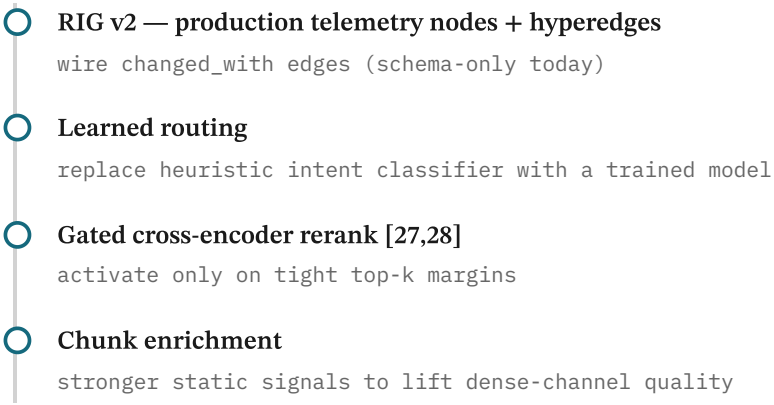


Figure §17. Future-work roadmap.

Several components are deliberately scoped below the architecture's ambition, and Figure §17 lays out the roadmap. The Git `changed_with` co-change edges exist in the RIG schema but are not yet mined, and runtime telemetry nodes are unwired, so the graph is presently static and structural. The cross-encoder rerank tier is a gated recommendation that activates only on tight top-k margins rather than an always-on stage — a cost-justified choice, since the suite shows reranking does not lift Hit@k here — the agentic head-to-head replicates this directly: adding a 7B-model rerank over Mesh's candidates moved agentic Hit@1 from 75.3% to 74.1% (n=81), confirming that a reranker on an already-strong *agentic* base is neutral-to-negative and validating the gated, recommend-only design for the agent loop. The cross-encoder's *retrieval* value, however, we now demonstrate on the one class that needs it: a code cross-encoder lifts conceptual Hit@1 by +7.2pt (69.6 → 76.8%, live-validated over two disjoint repo splits, §12.3b). The reconciliation is that a strong agent loop recovers that retrieval gain on its own, while a weak single-shot tier does not — the reranker still adds +3.8pt there (§12.3b). The cross-encoder is therefore now wired as an *active* component on the conceptual retrieval channel (default-on when an NVIDIA reranker is configured, graceful lexical-only fallback otherwise), tier-

gated for the agentic loop rather than withheld. Compression pass-rate evaluation measures tokens only, not downstream success.

Future work follows directly: replace heuristic intent classification (`routeRetrievalQuery`) with a learned routing policy and benchmark it against preference-trained routers; ship RIG v2 with production telemetry nodes and hyperedge support so impact and co-change queries draw on real execution and history; and stand up a larger, harder retrieval suite — multi-hop, impact-weighted, and externally authored — specifically to make the marginal value of the dense and graph channels measurable rather than inferred.

18 Conclusion

Mesh contributes an integrated, ablatable, cost-normalized architecture for repository-scale code agents, paired with an evaluation contract that ties each result to a specific layer. The pre-inference control plane — representation → evidence → context → economics → runtime — decomposes the agent loop into env-strippable layers with explicit token budgets, confidence gating, and recovery policies, treating context as a managed resource assembled before every turn rather than as an unbounded side effect of larger windows. On a grounded 229-task suite across six external repositories, intent-routed Mesh reaches Hit@1 79.9% / Hit@8 96.9% and, in a controlled head-to-head on identical data, beats the strongest retrieval baselines (full-file BM25, a SOTA code embedding, and SOTA managed retrieve-and-rerank) 2.4× overall at McNemar $p<0.001$ (§12.9) — while, on token economics against a competent grep-based agentic searcher, localizing at −69.7% tokens per query and −55.5% over multi-turn sessions, winning tokens on every repository. A same-model *agentic* head-to-head (n=122 across all twelve SWE-bench Lite repositories) confirms the honest boundary of the claim: against a capable grep *agent*, Mesh reaches parity localization accuracy (77.0% vs 74.6% Hit@1) at −13% tokens and −15% steps — the durable advantage is cost at parity, not a higher hit rate. These are repo-grounded results; the lasting argument is that cost-normalized agents — where every token and dollar is accounted against task success — should be a first-class design primitive, and that a pre-inference control plane is a practical way to enforce it.